

Dieter W. Heermann

Computer Simulation Methods

in Theoretical Physics

Second Edition

With 30 Figures

Springer-Verlag Berlin Heidelberg New York
London Paris Tokyo Hong Kong

Professor Dr. Dieter W. Heermann

Institut für Theoretische Physik der Universität Heidelberg,

Abteilung Vielteilchen Physik, Philosophenweg 19,

D-6900 Heidelberg, Fed. Rep. of Germany

ISBN 3-540-52210-7 2. Auflage Springer-Verlag Berlin Heidelberg New York
ISBN 0-387-52210-7 2nd edition Springer-Verlag New York Berlin Heidelberg

ISBN 3-540-16966-0 1. Auflage Springer-Verlag Berlin Heidelberg New York
ISBN 0-387-16966-0 1st edition Springer-Verlag New York Berlin Heidelberg

Library of Congress Cataloging-in-Publication Data. Heermann, Dieter W. Computer simulation methods in theoretical physics / Dieter W. Heermann. – 2nd ed. p. cm. Includes bibliographical references. ISBN 0-387-52210-7 (U.S.) 1. Mathematical physics—Mathematical models. 2. Mathematical physics—Data processing. I. Title. QC20.H47 1990 530.1'01'13—dc20 90-9573

This work is subject to copyright. All rights are reserved, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in other ways, and storage in data banks. Duplication of this publication or parts thereof is only permitted under the provisions of the German Copyright Law of September 9, 1965, in its current version, and a copyright fee must always be paid. Violations fall under the prosecution act of the German Copyright Law.

© Springer-Verlag Berlin Heidelberg 1986 and 1990
Printed in Germany

The use of registered names, trademarks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

Coverdesign: W. Eischenschink, D-6805 Heddeshcim
Printing: Weibert-Druck GmbH, D-6100 Darmstadt
Binding: J. Schäffer GmbH & Co. KG., D-6718 Grünstadt

This text was prepared using the PSTM Technical Word Processor
2154/3150-543210 – Printed on acid-free paper

*Dedicated to
A. Friedhoff and my parents*

Preface to the Second Edition

The new and exciting field of computational science, and in particular simulational science, has seen rapid change since the first edition of this book came out. New methods have been found, fresh points of view have emerged, and features hidden so far have been uncovered. Almost all the methods presented in the first addition have seen such a development, though the basics have remained the same. But not just the methods have undergone change, also the algorithms. While the scalar computer was in prevalent use at the time the book was conceived, today pipeline computers are widely used to perform simulations. This brings with it some change in the algorithms. A second edition presents the possibility of incorporating many of these developments. I have tried to pay tribute to as many as possible without writing a new book.

In this second edition several changes have been made to keep the text abreast with developments. Changes have been made in the style of presentation as well as to the contents. Each chapter is now preceded by a brief summary of the contents and concepts of that particular chapter. If you like, it is the chapter in a nutshell. It is hoped that by condensing a chapter to the main points the reader will find a quick way into the presented material.

Many new exercises have been added to help to improve understanding of the methods. Many new applications in the sciences have found their way into the exercises. It should be emphasized here again that it is very important to actually play with the methods. There are so many pitfalls one can fall into. The exercises are at least one way to confront the material.

Several changes have been made to the content of the text. Almost all chapters have been enriched with new developments, which are too numerous to list. Perhaps the most visible is the addition of a new section on the error analysis of simulation data.

It is a pleasure to thank all students and colleagues for their discussions, especially the students I taught in the summer of 1988 at the University of Lisbon.

Wuppertal, Heidelberg
September 1989

D.W. Heermann

Preface to the First Edition

Appropriately for a book having the title "Computer Simulation Methods in Theoretical Physics", this book begins with a disclaimer. It does not and cannot give a complete introduction to simulational physics. This exciting field is too new and is expanding too rapidly for even an attempt to be made. The intention here is to present a selection of fundamental techniques that are now being widely applied in many areas of physics, mathematics, chemistry and biology. It is worth noting that the methods are applicable not only in physics. They have been successfully used in other sciences, showing their great flexibility and power.

This book has two main chapters (Chaps. 3 and 4) dealing with deterministic and stochastic computer simulation methods. Under the heading "deterministic" are collected methods involving classical dynamics, i.e. classical equations of motion, which have become known as the molecular dynamics simulation method. The second main chapter deals with methods that are partly or entirely of a stochastic nature. These include Brownian dynamics and the Monte-Carlo method. To aid understanding of the material and to develop intuition, problems are included at the end of each chapter. Upon a first reading, the reader is advised to skip Chapter 2, which is a general introduction to computer simulation methods.

The material presented here is meant as a one-semester introductory course for final year undergraduate or first year graduate students. Accordingly, a good working knowledge of classical mechanics and statistical mechanics is assumed. Special emphasis is placed on the underlying statistical mechanics. In addition, the reader is assumed to be familiar with a programming language.

I would like to express my thanks to K. Binder, D. Stauffer and K. Kremer for discussions and critical reading of the manuscript, without which the book would not have taken its present form. It is also a pleasure to acknowledge discussions with members of the condensed matter group at the University of Mainz and to thank them for creating a warm working environment. In particular I would like to mention I. Schmidt and B. Dünweg. Finally, I thank I. Volk and D. Barkowski for proofreading the manuscript.

Special thanks are due to the Institut für Festkörperforschung of the Kernforschungsanlage Jülich for its hospitality, not only during part of the preparation of the book. Financial support from the Max-Planck-Institut für Polymerforschung (Mainz) and the Sonderforschungsbereich 41 is also gratefully acknowledged.

Contents

1. Introductory Examples	1
1.1 Percolation	1
1.2 A One-Particle Problem	4
Problems	6
2. Computer-Simulation Methods	8
Problems	12
3. Deterministic Methods	13
3.1 Molecular Dynamics	13
Integration Schemes	17
Calculating Thermodynamic Quantities	22
Organization of a Simulation	26
3.1.1 Microcanonical Ensemble Molecular Dynamics	27
3.1.2 Canonical Ensemble Molecular Dynamics	35
3.1.3 Isothermal-Isobaric Ensemble Molecular Dynamics	42
Problems	48
4. Stochastic Methods	51
4.1 Preliminaries	51
4.2 Brownian Dynamics	55
4.3 Monte-Carlo Method	61
4.3.1 Microcanonical Ensemble Monte-Carlo Method	73
4.3.2 Canonical Ensemble Monte-Carlo Method	78
4.3.3 Isothermal-Isobaric Ensemble Monte-Carlo Method	94
4.3.4 Grand Ensemble Monte-Carlo Method	96
Problems	100
Appendix	104
A1. Random Number Generators	104
A2. Program Listings	113
References	137
Subject Index	143

1. Introductory Examples

The concepts of deterministic and stochastic simulation methods are introduced. Two examples are used: the percolation theory, providing an example of a stochastic method and the simulation of a particle in a force field, providing an example of a deterministic method.

1.1 Percolation

A problem lending itself almost immediately to a computer-simulation approach is that of *percolation*. Consider a lattice, which we take, for simplicity, as a two-dimensional square lattice. Each lattice site can be either occupied or unoccupied. A site is occupied with a probability $p \in [0, 1]$ and unoccupied with a probability $1-p$. For p less than a certain probability p_c , there exist only finite clusters on the lattice. A *cluster* is a collection of occupied sites connected by nearest-neighbour distances. For p larger than or equal to p_c there exists an infinite cluster (for an infinite lattice, i.e., in the thermodynamic limit) which connects each side of the lattice with the opposite side. In other words, for an infinite lattice the fraction of sites belonging to the largest cluster is zero below p_c and unity at and above p_c .

Analytic results for the percolation threshold p_c , i.e., where an infinite cluster appears for the first time, are only available for two and infinite dimensions. The question arises whether one can obtain an approximation for the percolation threshold by computer simulations for dimensions higher than two and complicated lattice structures. To keep the computer-simulation approach transparent we stay with the two-dimensional square lattice.

By its nature the problem suggests a stochastic approach. Suppose one could generate a lattice filled with a given probability and check whether a percolating structure occurs, using this probability. To be sure that one is definitely above or below the percolation threshold an average over many such samples must be taken. Running through a range of p values the percolation threshold is narrowed down until sufficient accuracy is established.

Algorithmically the problem can be attacked as follows. We set up a two-dimensional array, for example in a FORTRAN program. Initially all elements are set to zero. The program now visits all sites of the lattice,

either by going successively through all rows (columns) or by choosing an element at random, until all sites have been visited. For each element of the array the program draws a uniformly distributed random number $R \in [0, 1]$. If R is less than an initially chosen p , then the element is set to one. After having visited all elements, one *realization* or configuration is generated.

A computer program producing a realization might look as shown in Algorithm A1. We assume that a main program exists which sets the lattice to zero and assigns a trial percolation probability p . The procedure "percolation" is then called, generating a configuration by going through the lattice in a typewriter fashion, i.e. working along each row from left to right, from the top to the bottom.

Algorithm A1

```

subroutine percolation (lattice,L,p)
real p
integer L, lattice (1:L,1:L)
  do 10 j = 1,L
    do 10 i = 1,L
      R = uniform()
      if ( R .lt. p ) lattice (i,j) = 1
10 continue
return
end

```

The function *uniform* is supposed to generate uniformly distributed random numbers in the interval $[0, 1]$. Examples of configurations generated by the Algorithm A1 are displayed in Fig.1.1. For a value of p equal to 0.1 (Fig.1.1 top) there are only scattered finite clusters. There is no path from one side to the opposite side. Choosing p equal to 0.6 (Fig.1.1. bottom) a large cluster exists connecting two opposite sides. In addition to the spanning cluster, there exist a number of finite clusters.

After having called the subroutine, the main program performs an analysis of the configuration for an infinite cluster. Two possibilities arise: either a spanning cluster exists, as in the case $p = 0.6$, then p is a candidate for being greater than or equal to the percolation threshold, or the opposite case is true. Calling the subprogram many times with the same p and averaging over the results determines whether, indeed, p is above or below the threshold.

To see where some of the difficulties lie in connection with such simulations, we step up one dimension ($d=3$) (Fig.1.2). The first difficulty is the size dependence of the results for the percolation probability. Intuitively we expect that it should be easier for clusters in a small lattice to connect opposite sides, thus the percolation threshold would be shifted to smaller p values. Indeed, the results on the percolation probability for the three-dimensional lattice displayed in Fig.1.2 show this behaviour. We use as a criterion for the percolation that a spanning cluster exists along at least one of the possible directions. We note further that no sharp transition occurs for the finite small lattices. The percolation threshold is smeared out and difficult to estimate.

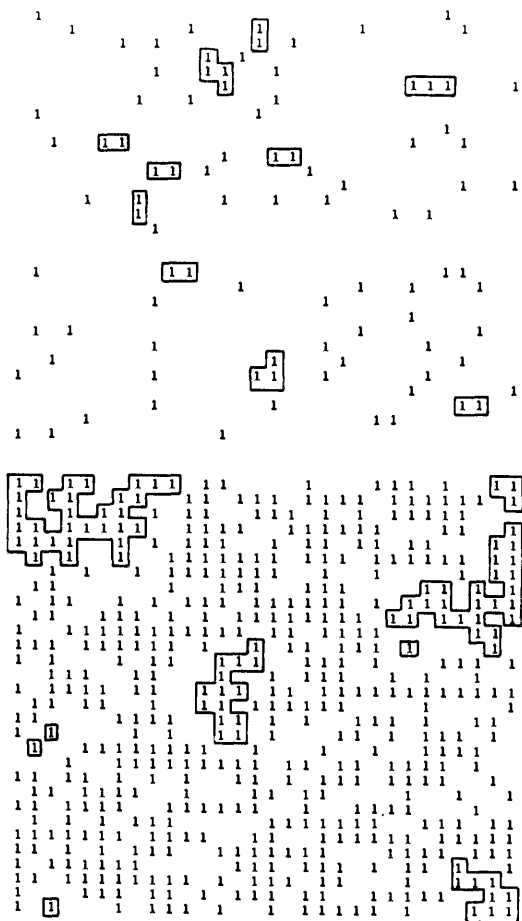


Fig.1.1. Configurations generated by a stochastic computer simulation of the percolation problem. The top figure shows a configuration generated with an occupation probability $p = 0.1$. The realization shown in the bottom picture was generated using $p = 0.6$. Some clusters are marked by contours. Lattice sites taken by zeros are not shown

The second difficulty is the number of samples. For an accurate determination of p_c quite a large number of samples have to be taken to reduce the statistical uncertainty. This holds true for other such direct simulations.

The third difficulty concerns the random numbers. A problem arises if the random numbers have some built-in correlations. Such a correlation is extremely dangerous since it biases the results and is only detectable if some aspects of the problem are known from different methods or the results show some extreme anomaly.

The approach described above to determining the percolation threshold is an example of a *stochastic simulation method*, in particular of a *Monte-Carlo simulation*. As the name suggests, such simulations are intricately connected with random numbers. In the percolation problem the random

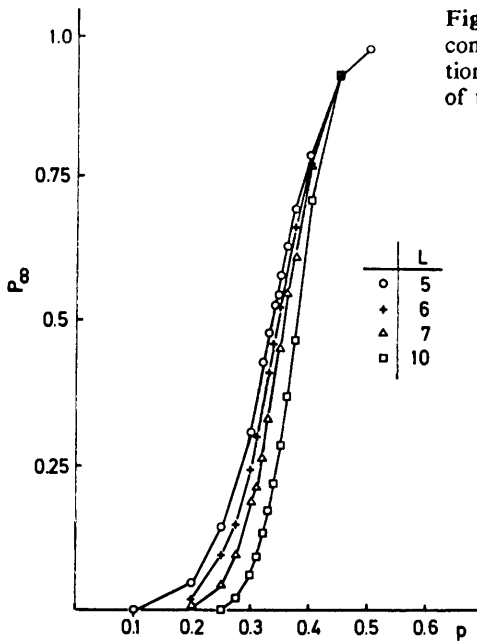


Fig.1.2. Shown is the dependence of the computer simulation results of the percolation problem (three dimensional) on the size of the lattice

numbers are exploited directly, whence such simulations are called *direct Monte-Carlo simulations*.

1.2 A One-Particle Problem

The one-particle system moving in a spring potential, i.e., a one-dimensional harmonic oscillator, supplies another illustrative example where a solution is obtainable by computer simulation. Although the system is trivial to solve analytically, it nevertheless provides insight into possible ways to attack problems involving a collection of interacting particles not readily solvable with analytic methods.

The nature of the problem is deterministic, in the sense that we start from a Hamiltonian description of the particle moving under the influence of the force exerted by the spring, i.e.,

$$\mathcal{H} = \frac{p^2}{2m} + \frac{1}{2} kx^2, \quad (1.1)$$

where p is the momentum, m the mass of the particle, k the spring constant and x the position. In addition to the Hamiltonian, we need to specify the initial conditions $(x(0), p(0))$. There is no coupling to an external system and the energy E is a conserved quantity. The particle will follow a trajectory on a surface of constant energy given by

$$\frac{p^2}{2mE} + \frac{kx^2}{2E} = 1 . \quad (1.2)$$

Having written down the Hamiltonian we have some options as to the form of the equations of motion. We may reformulate the problem in terms of a Lagrangian and derive the equation of motion or cast the problem in the form of the Newtonian equation. The algorithm for numerical solution and its properties depend on the choice. Here we take the Hamiltonian form of the equation of motion

$$\frac{dx}{dt} = \frac{\partial \mathcal{H}}{\partial p} = \frac{p}{m} , \quad \frac{dp}{dt} = - \frac{\partial \mathcal{H}}{\partial x} = - kx . \quad (1.3)$$

We would like to compute some properties of the system as it moves along a path $(x(t), p(t))$ in phase space. In general, the complexity of the equations of motion defy an analytic treatment, and one resorts to a numerical integration. We go about solving the problem by approximating the continuous path by a polygon, i.e., to first order the differential is taken as

$$\frac{df}{dt} = \frac{1}{h} [f(t+h) - f(t)] , \quad (1.4)$$

h being the *basic time step*. With such an approximation a solution of the equations of motion can be obtained only at times which are multiples of the basic unit h . Note that if the basic time step is finite there will always be a certain error, i.e., the generated path will deviate from the true one. Inserting the *discretization* for the differential into the equations of motion, we obtain the following recursion formulae for the position and the momentum:

$$\frac{dx}{dt} \simeq \frac{1}{h} [x(t+h) - x(t)] = \frac{p(t)}{m} , \quad \frac{dp}{dt} \simeq \frac{1}{h} [p(t+h) - p(t)] = - kx(t) \quad (1.5)$$

or

$$x(t+h) = x(t) + hp(t)/m , \quad p(t+h) = p(t) - hkx(t) . \quad (1.6)$$

Given an initial position $x(0)$ and momentum $p(0)$ consistent with a given energy, the trajectory of the particle is simulated. Starting from time zero, the position and momentum at time h are computed via the above equations; then at $t = 2h, 3h$, etc. Any properties one is interested in can be computed along the trajectory that is generated by the recursion relation. Two examples of trajectories based on the polygon method are shown in Fig.1.3. The total energy was set equal to unity. The first choice of the basic time step is clearly too large, since the trajectory does not follow an ellipse. The path is a spiral, indicating that energy is absorbed by the particle. The second choice gives a more reasonable trajectory.

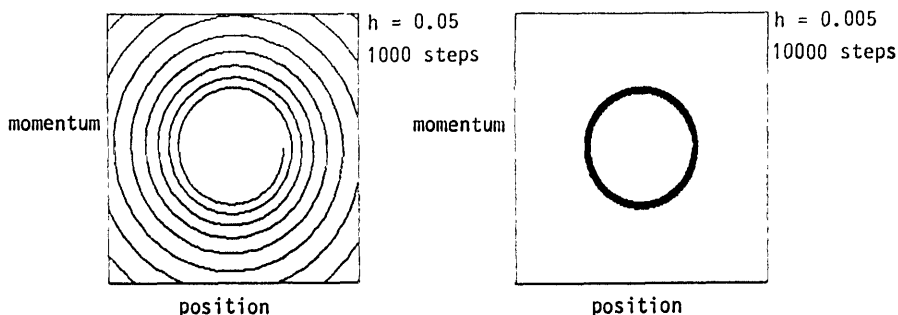


Fig.1.3. Trajectories in phase space for the spring problem as generated with a simple algorithm. In the left picture the result indicates that the energy was not conserved. The right picture shows an almost conserved energy

For a collection of particles we are faced with similar problems. Due to the finite step size the trajectory departs from the true one. We are interested in developing algorithms of high order in h to keep the error as small as possible. On the other hand, the basic time unit determines the simulated real time. A very small time step may have the consequence that the system does not reach equilibrium during a run.

The kind of approximation made to solve the spring problem numerically on a computer is but one type. In the following, more sophisticated schemes will be presented for *deterministic simulations*. Such methods are known as *molecular dynamics*.

In many cases, computer-simulation methods offer a surprisingly simple and elegant approach to the solution of a problem. In the following chapters we study some most commonly used methods.

Problems

- 1.1 Write a computer program for the two-dimensional percolation problem. You may watch for percolation, i.e., for a spanning cluster, either by inspection (printing out the realizations) or by writing a routine. You should not be discouraged if you do not come up immediately with an efficient algorithm to identify clusters (see [1.1 or 2] for an algorithm). (Answer: $p_c = 1/2$ for $d = 2$.)
- 1.2 **Bond Percolation.** In this chapter we were concerned with the site percolation problem. Equally well we can study the bond percolation problem. Instead of occupying the lattice sites we connect sites by choosing a probability p_{bond} and if the outcome of a comparison with a random number is positive we occupy the bond between the selected two sites. Revise your algorithm for this problem. Do you think that the site and the bond percolation problem are equivalent?
- 1.3 **Random Walk.** Consider a lattice on which a particle can move. The particle is only allowed to go from one site to the nearest neighbour site. Start from a site which is considered the origin. Choose a random

number which determines to which of the q nearest neighbour sites the particle moves. Continue to do so for n moves. To establish the diffusion law, develop an algorithm which generates walks of length n . Determine the mean-square displacement for each walk and average the result. Plot the mean-square displacement as a function of n .

- 1.4 **Growth of Clusters** (Eden cluster [1.3]). One starts from one occupied site on a lattice as a seed for a cluster. At every "time step" one additional randomly selected perimeter site is occupied. A perimeter site is an empty neighbour of an already occupied site. Write a computer program to simulate the growth of the cluster as a function of "time".
- 1.5 **Aggregation.** A nice example of a direct method is the generation of aggregates. Take a lattice with one or more sites of the lattice designated as aggregation sites. Draw a wide circle or sphere around the potential aggregation sites. Introduce a new particle at a randomly picked site on the circle or sphere. The particle is now performing a random walk (cf. previous exercise) until it walks outside the valid region, or moves to one of the surface sites of the aggregation site. If the particle has come to the surface site of the aggregation site it sticks and the potential aggregation surface has grown. Inject a new particle into the valid region and continue until the aggregate has grown substantially. The valid region is then changed to give room for the walker to move. Why is this a direct probabilistic simulation of aggregation? Can you invent variations of the problem?
- 1.6 Develop an algorithm and write a computer program for the spring problem. To actually solve the equations numerically it is most convenient to scale the equations such that they become dimensionless. Check the energy during the simulation. Is it conserved? Vary the time step.
- 1.7 Develop an algorithm and write a computer program for the pendulum:

$$d^2\phi/dt^2 = -(g/\ell)\sin\phi ,$$

where ϕ is the angle, ℓ the length of the string and g the gravitational constant. Is there a stability problem?

2. Computer-Simulation Methods

Computer-simulation methods are by now an established tool in many branches of science. The motivations for computer simulations of physical systems are manifold. One of the main motivations is that one eliminates approximations. Usually, to treat a problem analytically (if it can be done at all) one needs to resort to some kind of approximation; for example, a mean-field-type approximation. With a computer simulation we have the ability to study systems not yet tractable with analytical methods. The computer simulation approach allows one to study complex systems and gain insight into their behaviour. Indeed, the complexity can go far beyond the reach of present analytic methods.

Because they can be used to study complex systems, computer-simulation methods provide standards against which approximate theories may be compared. At the same time, they allow the comparison of models with experiment, and provide a means of assessing the validity of a model.

There is yet another feature. Computer simulations can fill the gap between theory and experiment. Some quantities or behaviours may be impossible or difficult to measure in an experiment. With computer simulations such quantities can be computed.

At the outset of a simulation stands a well-defined model of a physical system. We are interested in computing properties of the physical system. Our point of view is that the properties or observables appear as averages over some sample space. For example, in the percolation problem (Sect.1.1) the threshold p_c is the average probability of percolation over the space of all configurations. In the spring problem (Sect.1.2) the temperature is computed as the average kinetic energy along the generated path.

For the main part we shall assume that a system under consideration has a model Hamiltonian $\mathcal{H}$. We denote a state of the system by $\mathbf{x} = (x_1, \dots, x_n)$, where n is the number of degrees of freedom. The set of states constitutes the available phase space Ω . The property A to be calculated will be a function of the states of the system. As mentioned above, our point of view is statistical mechanical. What we need to specify in order to compute the property A is a distribution function $f(\cdot)$. The quantity A is then given by

$$\langle A \rangle = Z^{-1} \int_{\Omega} A(\mathbf{x}) f(\mathcal{H}(\mathbf{x})) d\mathbf{x}, \quad (2.1)$$

where

$$Z = \int_{\Omega} f(\mathcal{K}(\mathbf{x})) d\mathbf{x} .$$

This is the *ensemble average* with the partition function Z . The distribution function f specifies the appropriate ensemble for the problem at hand.

The ensemble average is, however, not accessible in computer simulations. In such simulations the quantity A is evaluated along a path in phase space. Take the spring problem. We are not going to evaluate the temperature for a large number of similar systems, rather, we propagate the particle along a trajectory in phase space and evaluate the kinetic energy along the path. What we are computing is a *time average*

$$\bar{A}_t = (t-t_0)^{-1} \int_{t_0}^t A(\mathbf{x}(\tau)) d\tau . \quad (2.2)$$

The question arising is: Are the two averages the same? For this we must invoke *ergodicity*, allowing the replacement of ensemble averages by time averages

$$\langle A \rangle = \bar{A}_{\infty} . \quad (2.3)$$

At this point, one of the two major limitations of computer-simulation methods arises. Clearly a computer simulation cannot follow a path over an infinite time. The *observation time is limited* to a finite path length so that actually the available phase space is sampled. One has to be content with

$$\bar{A} \simeq \langle A \rangle . \quad (2.4)$$

For some problems the finite observation time may be considered infinite. Consider, for example, the computation of a molecular system where the observation time is much larger than the molecular time. What we also have to take into account is the statistical error [2.1-3].

We are led to the question of how we are going to propagate the system through phase space. This is the point where we distinguish two methods. The approaches developed here are:

- (i) deterministic methods, and
- (ii) stochastic methods.

We look first at the deterministic methods. The idea behind these is to use the *intrinsic dynamics* of the model to propagate the system. One has to set up equations of motion and integrate forward in time. For a collection of particles governed by classical mechanics this yields a trajectory $(\mathbf{x}^N(t), \mathbf{p}^N(t))$ in phase space for the fixed initial positions $\mathbf{x}_1(0), \dots, \mathbf{x}_N(0)$ and momenta $\mathbf{p}_1(0), \dots, \mathbf{p}_N(0)$.

Stochastic methods take a slightly different approach. Clearly, what is required is to evaluate only the configurational part of the problem. One can always integrate out the momentum part. The problem which is posed then is how to induce transitions from one configuration to another, which in the deterministic approach would be established by the momenta. Such transitions in stochastic methods are brought about by a probabilistic evolution via a *Markov process*. The Markov process is the probabilistic analogue to the intrinsic dynamics. This approach has the advantage that it allows the simulation of models which have no intrinsic dynamics whatsoever.

As well as the finite observation time, simulational physics is faced with a second major limitation: *finite system size*. In general, one is interested in the computation of a property in the thermodynamic limit, i.e., the number of particles tends to infinity. Computer simulations allow, however, only system sizes small compared to the thermodynamic limit so that there are possible *finite-size effects*. In order to reduce the finite-size effects an approximation is made that has thus far been suppressed, namely the introduction of boundary conditions. Boundary conditions clearly affect some properties.

Let us follow up the points made above. In deterministic as well as in stochastic computer simulation methods the successive configurations are correlated [2.4-8]. What does this mean if we calculate the time average of an observable A, which by necessity can cover only a finite observation time? Let us consider the statistical error for n successive observations A_i^* , $i = 1, \dots, n$:

$$\langle (\delta A)^2 \rangle = \langle [n^{-1} \sum_{i=1}^n (A_i - \langle A \rangle)]^2 \rangle. \quad (2.5)$$

In terms of the autocorrelation function for the observable A

$$\phi_A(t) = \frac{\langle A(0)A(t) \rangle - \langle A \rangle^2}{\langle A^2 \rangle - \langle A \rangle^2} \quad (2.6)$$

and the characteristic correlation time

$$\tau_A = \int_0^\infty \phi_A(t) dt \quad (2.7)$$

we can rewrite the statistical error as

$$\langle (\delta A)^2 \rangle \simeq \frac{2\tau_A}{n\delta t} [\langle A^2 \rangle - \langle A \rangle^2] \quad (2.8)$$

where δt is the time between observations, i.e., $n\delta t$ is the total observation time τ_{obs} .

We notice that the error does not depend on the spacing between the observations but on the total observation time. Also the error is not the one which one would find if all observations were independent. The error is enhanced by the characteristic correlation time between configurations. Only an increase in the sample size and/or a reduction in the characteristic correlation time τ_A can reduce the error.

Now that we know how the statistical error for an observable A depends on the finite observation time, we can ask for the dependence on the finite system size. For this we define

$$\Delta(n, L) = \sqrt{(\langle A^2 \rangle_L - \langle A \rangle_L^2)/n} . \quad (2.9)$$

Here L is the linear dimension of the system. Note that we write $\langle \cdot \rangle_L$ for the average. This is meant as the average with respect to the finite system size. How does this error depend on L ?

Recall that for thermodynamic equilibrium, for a system of infinite size one observation suffices to obtain A. In other words, if $L \rightarrow \infty$ then $\Delta(n, L)$ must go to zero, regardless of n . Or, if we increase the system size then the effective number of observations should increase. Let L be the system size and L' the new one which we obtain by a scale factor b with $b > 1$: $L' = bL$. The number of effective observations will change to $n' = b^{-d}n$ where d is the dimensionality. More formally we can express the idea by

$$\Delta(n, L) = \Delta(n', L') = \Delta(b^{-d}n, bL) . \quad (2.10)$$

We can work out this expression using the definition of Δ and find

$$\langle A^2 \rangle_L - \langle A \rangle_L^2 \propto L^{-x}, \quad 0 \leq x \leq d . \quad (2.11)$$

In the case where $x = d$ we call the observable A *strongly self-averaging* and in the cases $0 < x < d$, *weakly self-averaging*. As we increase L , Δ tends to a finite value, independent of L .

The problem of non-self-averaging arises not only at the critical point, where such quantities as the susceptibility or the specific heat exhibit a lack of self-averaging. It occurs also in non-equilibrium situations.

The finite size of the system has an advantage. It enables one to compute second-order thermodynamic properties. In a system of finite size the intensive properties describing the system deviate from their mean values, i.e., they fluctuate around a mean value. These fluctuations depend on the ensemble, of course. Let us take as an example the fluctuations in temperature. We assume that we work with the microcanonical ensemble, as we do in some deterministic methods. It is of interest to relate the fluctuation in the temperature to the specific heat C_V , which in thermodynamics is computed from the second derivative of the free energy F :

$$C_V = - \frac{\partial}{\partial T} \left[T^2 \frac{\partial(F/T)}{\partial T} \right]_V .$$

The fluctuations in temperature are related to the specific heat [2.9, 10] by

$$\frac{\langle T^2 \rangle - \langle T \rangle^2}{\langle T \rangle^2} = \frac{3}{2N} \left[1 - \frac{3kN}{2C_V} \right] . \quad (2.12)$$

Similar relations can be obtained that relate the fluctuations in magnetization in a canonical ensemble to the isothermal susceptibility. The interest in fluctuations stems from the fact that the free energy is difficult to compute in a computer simulation.

Although we are jumping ahead somewhat, it seems appropriate to discuss ensembles at this point. The natural ensemble for the molecular dynamics method is the microcanonical one, where the energy is a constant of motion. Nevertheless, we would like to study systems where the temperature and/or the pressure is a constant of motion. In such a situation the system is not closed and is in contact with a bath. The contact is, however, only conceptual. The approach taken will be to constrain some degrees of freedom. Let us take the case of a constant temperature. For a constant temperature the mean kinetic energy is an invariant. This suggests that an algorithm could be devised such that the mean kinetic energy is constrained to a given value. Due to the constraint we are not really working with a canonical ensemble. Rather, we reproduce only the configurational part of the ensemble. The approach is valid as long as the constraint does not destroy the Markovian character of the transitions from one state to another. The dynamical properties may, however, be influenced by the constraint. In the following we shall always have at the back of our minds that if we impose an ensemble on a system it may be only the configurational part that we evaluate.

Problems

- 2.1 Assume a Gaussian distribution $P(\delta A)$ for the statistical error. Work out the averaging behavior for $(\delta A)^2$ and $(\delta A)^4$.
- 2.2 Show that the susceptibility and the specific heat are non-self-averaging at the critical point.
- 2.3 Can you work out an expression for the statistical error which incorporates the behaviour of $r_A L^3$ and averaging behaviour at the critical point?

3. Deterministic Methods

The kind of systems we are dealing with in this chapter are such that all degrees of freedom are explicitly taken into account. We do not allow stochastic elements representing, for example, an interaction of the system with a heat bath. The starting point is a Newtonian, Lagrangian or Hamiltonian formulation within the framework of classical mechanics. What we are interested in is to compute quantities for such systems, for example, thermodynamic variables, which appear as ensemble averages. Due to energy conservation the natural ensemble is the microcanonical one. However, sometimes it is desirable to compute a quantity in a different ensemble. To allow such calculations within the framework of a Newtonian, Lagrangian or Hamiltonian description, the formulation has to be modified. In any case, the formulation leads to differential equations of motion. These equations will be discretized to generate a path in phase space, along which the properties are computed.

3.1 Molecular Dynamics

The mathematical background to the molecular dynamics method is presented. Some approximation schemes for the differential equations are discussed. Basic notions such as the computational cell, boundary conditions and the minimum image convention for the calculation of the force are introduced.

The starting point for the *Molecular Dynamics* (MD) method [3.1-7] is a well-defined microscopic description of a physical system. The system can be a few- or many-body system. The description may be a Hamiltonian, Lagrangian or expressed directly in Newton's equations of motion. In the first two cases the equations of motion must be derived by applying the well-known formalisms. The molecular dynamics method, as the name suggests, calculates properties using the equations of motion, and one obtains the static as well as the dynamic properties of a system. As we shall see in Sect.4.3, the Monte-Carlo method yields the configurational properties, although there is also a dynamic interpretation [3.8].

The approach taken by the MD method is to solve the equations of motion numerically on a computer. To do so, the equations are approxim-

ated by suitable schemes, ready for numerical evaluation on a computer. Clearly there will be an error involved due to the transition from a description in terms of continuous variables with differential operators to a description with discrete variables and finite difference operators. The order of the entailed error depends on the specific approximation, i.e., the resulting algorithm. In principle, the error can be made as small as desired, restricted only by the speed and memory of the computer.

Definition 3.1

The molecular dynamics method computes phase space trajectories of a collection of molecules which individually obey classical laws of motion. □

Note that the definition includes not only point-particle systems but also collections of "particles with subunits" [3.9]. Indeed, an algorithm exists that allows systems to have internal constraints as, for example, a system of polymers [3.10-17]. Also possible are constraints such as the motion in a specific geometry [3.18].

Early simulations were carried out for systems where the energy is a constant of motion [3.1-7]. Accordingly, properties were calculated in the microcanonical ensemble where the particle number N , the volume V , and energy E are constant. However, in most situations one is interested in the behaviour of a system at constant temperature T . This is partly due to the fact that the appropriate ensemble for certain quantities is not the microcanonical but the canonical ensemble. Significant advances in recent years now allow computation within ensembles other than the microcanonical. In Sects. 3.1.2 and 3 we will see how the equations of motion are modified to allow such calculations without introducing stochastic forces.

The general technique is not restricted to deterministic equations of motion. Rather, equations of motion involving stochastic forces can be simulated. Algorithms covering such problems will be discussed in Chap.4; however, some of the material presented here also applies to non-deterministic dynamics.

What we have to deal with are equations of the form

$$\frac{du(t)}{dt} = K(u(t), t), \quad (3.1)$$

where u is the unknown variable, which might be, for example, a velocity, an angle or a position, and K is a known operator. The variable t is usually interpreted as the time. We shall not restrict ourselves to a deterministic interpretation of (3.1) but allow $u(t)$ to be a random variable. For example, we might be interested in the motion of a Brownian particle and (3.1) takes on the form of the *Langevin equation*

$$\frac{dv(t)}{dt} = -\beta v(t) + R(t). \quad (3.2)$$

Since the fluctuation force $R(t)$ is a random variable, the solution $v(t)$ to the *Stochastic Differential Equation* (SDE) will be a random function.

We may distinguish four types of the Equation (3.1):

- 1) K does not involve stochastic elements and the initial conditions are precisely known;
- 2) K does not involve stochastic elements but the initial conditions are random;
- 3) K involves random force functions; or
- 4) K involves random coefficients.

We treat types 1 - 3 in this text. In the case of types 1 and 2 the task of solving (3.1) reduces to an integration. For type-3 problems, special precautions have to be taken, since the properties of the solution are developed through probabilistic arguments.

For simplicity, we assume for the remainder of this chapter that we are dealing with monatomic systems so that the molecular interactions do not depend on the orientation of the molecules. Furthermore, we will always deal with pairwise additive central-force interactions. To stress once again the point made earlier, the technique is not restricted to such systems. The inclusion of orientation-dependent interactions and the constraints of connectivity would unnecessarily complicate the exposition. In general, the system will be described by the Hamiltonian

$$\mathcal{H} = \frac{1}{2} \sum_i \frac{p_i^2}{m_i} + \sum_{i<j} u(r_{ij}) , \quad (3.3)$$

where r_{ij} is the distance between the particles i and j . For ease of reference, we abbreviate the configurational internal energy as

$$U(r) = \sum_{i<j} u(r_{ij}) . \quad (3.4)$$

Let the system consist of N particles. Since we restrict ourselves to properties of the bulk at a specific density ρ we must introduce a volume, the *MD cell*, to retain a constant density. If the system is in thermal equilibrium, the shape of the volume is irrelevant [3.19]. This is true, of course, for gases and liquids in the limit where the volume is large enough. For systems in a crystalline state the shape does make a difference.

For liquid or gaseous states we take a cubic volume for computational simplicity. Let L be the linear size of the MD cell with volume $V = L^3$. The introduction of the box creates six unwanted surfaces. Particles hitting these surfaces would be reflected back into the interior of the cell. Especially for systems with a small number of particles, important contributions to any property would come from the surfaces. To reduce the effect of the surfaces we impose *periodic boundary conditions* (pbc), i.e., the basic cell is identically repeated an infinite number of times. Mathematically this is stated as follows. For any observable A we have

$$A(\mathbf{x}) = A(\mathbf{x} + \mathbf{nL}) \quad \text{with} \quad \mathbf{n} = (n_1, n_2, n_3) \quad (3.5)$$

for all integers n_1, n_2, n_3 . The computational implementation is that if a particle crosses a surface of the basic cell it re-enters through the opposite wall with unchanged velocity. With the periodic boundary conditions we have eliminated the surfaces and created a quasi-infinite volume to represent the macroscopic system more closely. The assumption involved is that the small volume is embedded in an infinite block.

Each component of a position vector is represented by a number between zero and L . If particle i is at $\mathbf{r}_i$, there is a set of *image particles* at positions $\mathbf{r}_i + \mathbf{nL}$, $\mathbf{n}$ being an integer vector. Due to the periodic boundary conditions the potential energy is affected because we have

$$U(\mathbf{r}_1, \dots, \mathbf{r}_N) = \sum_{i < j} u(r_{ij}) + \sum_{\mathbf{n}} \sum_{i < j} u(|\mathbf{r}_i - \mathbf{r}_j + \mathbf{nL}|) . \quad (3.6)$$

In order to avoid the infinite summation in the second term on the right-hand side we introduce a convention about how the distances are computed [3.20, 21]

Convention 3.2. Minimum Image

The distance r_{ij} between particle i at $\mathbf{r}_i$ and particle j at $\mathbf{r}_j$ is $r_{ij} = \min \{|\mathbf{r}_i - \mathbf{r}_j + \mathbf{nL}|\}$ over all $\mathbf{n}$. □

A particle in the basic cell interacts only with each of the $N-1$ other particles in the basic cell or their nearest images. In effect, we have cut off the potential by the condition

$$r_c < L/2 . \quad (3.7)$$

The price to be paid is that we neglect the background. It would be more realistic to include the interaction of each particle with all the image particles. An elegant procedure for doing so has been worked out by *Ewald* [3.22, 23]. The question of how the properties under computation are influenced is not yet fully understood and remains to be investigated more closely. Better understood are the boundary conditions applied within the Monte-Carlo method. The value of L should be chosen so large that the forces that would occur for distances larger than $L/2$ are negligibly small, to avoid finite-size effects.

A cubic volume is, of course, not the only possible geometry to confine the system and to conserve the density (Problem 3.1). Some applications, for example crystallization, require different choices [3.24-27]. In any case, there is a danger that the periodic boundary conditions impose a particular lattice structure [3.28, 29].

Integration Schemes

From a numerical-mathematics point of view the MD method is an *initial value problem*. A host of algorithms have been developed [3.30, 31] for this problem, which are, however, not all applicable in the context of physical problems. The reason is that many schemes require several evaluations of the right-hand side of (3.1), storage of previous evaluations and/or iterations. Specifically, assume that (3.1) was derived from the Hamiltonian (3.3), i.e., the equations of motion are

$$m \frac{dr_i}{dt} = p_i, \quad \frac{dp_i}{dt} = \sum_{i < j} F(r_{ij}). \quad (3.8)$$

Each evaluation of the right-hand sides for the N particles takes $N(N-1)/2$ quite time-consuming operations. To avoid this, simpler schemes are employed which suffice in accuracy for most applications. The conservation properties are also a problem, as we shall discuss below.

To solve the equations of motion on a computer we construct a finite difference scheme for the differential equations to the highest possible order. From the difference equations we then derive recursion relations for the positions and/or velocities (momenta). These algorithms perform in a step-by-step way. At each step approximations for the positions and velocities are obtained, first at time t_1 then at $t_2 > t_1$, etc. Hence, the integration proceeds in the time direction (*time-integration algorithms*). The recursion relation must clearly allow efficient evaluation. In addition, the scheme must be numerically stable.

The most straightforward discretization of the differential equation stems from the Taylor expansion. The idea is to base the algorithm on a discrete version of the differential operator. With suitable assumptions we can expand the variable u in a Taylor series

$$u(t+h) = u(t) + \sum_{i=1}^{n-1} \frac{h^i}{i!} u^{(i)}(t) + R_n, \quad (3.9)$$

where R_n gives the error involved in the approximation. However, it is convenient to use the *big-oh* notation. In general, the notation $O(f(z))$ stands for any quantity $g(z)$ such that $g(z) < M < f(z)$ whenever $a < z < b$. Here M is an unspecified constant. The following calculus holds:

$$\begin{aligned} f(z) &= O(f(z)), \\ cO(f(z)) &= O(f(z)), \\ O(f(z)) + O(f(z)) &= O(f(z)), \\ O(O(f(z))) &= O(f(z)), \\ O(f(z)) \times O(g(z)) &= O(f(z)g(z)). \end{aligned} \quad (3.10)$$

Using the big-oh calculus we find the error entailed in (3.9) of the order $O(h^n)$. Equation (3.9) allows an immediate construction of a difference scheme (symmetric difference approximation) with a *discretization error* of the order h . Let $n = 2$, then

$$\frac{du(t)}{dt} = h^{-1}[u(t+h) - u(t)] + O(h) \quad (3.11)$$

$$= h^{-1}[u(t) - u(t-h)] + O(h) . \quad (3.12)$$

These are the simplest schemes; the first we are familiar with from the example in Chap.1. Equation (3.11) is called the *forward difference quotient* and (3.12) the *backward* one. Using the forward difference we get the *Euler algorithm* [3.31] for the solution of the general problem (3.1) with the initial value u_t at the starting time t , i.e.,

$$u(t) = u_t , \quad u(t+h) = u(t) + hK(u(t), t) . \quad (3.13)$$

The Euler algorithm represents a typical example of a *one-step method*. Such methods use the previous value as the only input parameter to determine a new value. We shall now derive the error introduced by employing the algorithm. We have to distinguish between the local and the global error. Let $z(t)$ be the exact solution of

$$\frac{dz(t)}{dt} = K(z(t), t) . \quad (3.14)$$

Define a function

$$\mu(u, t, h) = \begin{cases} \frac{z(t+h) - u}{h} , & h \neq 0 , \\ K(u, t) , & h = 0 , \end{cases}$$

which is the difference quotient of the exact solution. The difference

$$\tau(u, t, h) = \mu(u, t, h) - K(u, t) \quad (3.15)$$

measures the *local discretization error*. If

$$\tau(u, t, h) = O(h^p) , \quad (3.16)$$

then the *method* is of order p . The Euler algorithm has $p = 1$. We can go even further and ask for the *global discretization error*. It can be shown [3.31] that the global error is equal to the local error for the one-step method.

So far we have considered only one-step methods. A more sophisticated scheme yielding a two-step method is immediately derived by using (3.9) with $n = 3$:

$$u(t+h) = u(t) + h \frac{du(t)}{dt} + \frac{1}{2} h^2 \frac{d^2 u(t)}{dt^2} + R_3 , \quad (3.17)$$

$$u(t-h) = u(t) - h \frac{du(t)}{dt} + \frac{1}{2} h^2 \frac{d^2 u(t)}{dt^2} + R_3^* .$$

Note that $R_3 \neq R_3^*$. Subtracting the second equation from the first, we get

$$u(t+h) = u(t-h) + 2h \frac{du(t)}{dt} + R_3 - R_3^* .$$

The error analysis shows that it is $O(h^3)$. Hence

$$\frac{du(t)}{dt} = \frac{1}{2h} [u(t+h) - u(t-h)] + O(h^2) . \quad (3.18)$$

Using the same idea we obtain for the second derivative

$$u^{(2)}(t) = h^{-2} [u(t+h) - 2u(t) + u(t-h)] + O(h^2) . \quad (3.19)$$

Multi-step methods allow the construction of algorithms of high order. Typical members of this class employed in simulational physics are those developed by *Gear* [3.33-35], *Beeman* [3.36], and *Toxvaerd* [3.37]. Such methods (including the one-step) have the general form

$$u(t+rh) + \sum_{\nu=0}^{r-1} a_{\nu} u(t+\nu h) = hG(t; u(t+rh), \dots, u(t); h) , \quad (3.20)$$

where G is some function of K , for example

$$G = \sum_{\nu=0}^r b_{\nu} K(u(t+\nu h), t+\nu h) .$$

We distinguish between *predictor* and *corrector schemes*. In a predictor scheme G does not depend on $u(t+rh)$ whereas it does in a corrector scheme. The Beeman algorithm, for example, is a third-order predictor-corrector scheme.

Most predictor-corrector methods require far more storage than the one- or two-step methods. Due to the present-day limitation of computer memory, only certain algorithms are applicable to physical systems. In addition, some methods require iterations to solve for the implicitly given variable. In the following we shall not use sophisticated predictor-corrector schemes. The reader is directed to [3.30-37] for more information.

Now that we have derived some algorithms to solve the equations of motion numerically, the question arises as to the choice of the basic time

step h (*MD step*). It determines the accuracy of the computed trajectory. Consequently, h affects the accuracy of the computed properties, in addition to the statistical error. But the choice of h is also important with regard to the simulated real time. For many problems it is desired to simulate a fairly long real time. The question is how large can the time step be? Consider, for example, an argon system of N particles, which will be the standard example in this chapter. The interaction between the particles is assumed to be of the Lennard-Jones type. For the argon system a time step $h \propto 10^{-2}$ was found sufficient in most regions of the phase diagram [3.6, 7]. Here h is a dimensionless quantity and the real time equivalent is roughly 10^{-14} s. Hence, a simulation lasting 1000 steps yields a real time equivalent of 10^{-11} s.

In connection with the number of MD steps carried out, h determines how much of the phase space is sampled. Naturally one would like to make h as large as possible to sample large portions. However, h determines the time scale, and we have to consider the time scale(s) on which changes in the system occur. Some systems have several different scales. A molecular system may have one time scale for intramolecular modes and another for intermolecular modes. Unfortunately, there exists no criterion for a choice of h . There is only a very general rule of thumb [2.3]: The fluctuations in the total energy should not exceed a few percent of the fluctuations in the potential energy. It is not clear in how far this carries any meaning. One would need to calculate the correlation functions for all observables of interest. Often the relaxation times are different. Hence looking at just the energy can be misleading.

One reason for energy fluctuations is the potential cut-off to be described later. A second reason is the error entailed by the approximation. No matter how high the order of an algorithm, the system will eventually depart from the true trajectory, as long as h is finite. A drift in the energy δE is caused by the finite time step, though the drift might be small.

From a more general point of view we can ask for the conservation properties of the algorithms. The energy, and the linear and angular momenta should be conserved during the course of a molecular-dynamics simulation. One way to establish conservation is to constrain the system artificially [3.37]. There is, however, a rigorous way of enforcing conservation [3.38-40]: Instead of using forces to calculate the motion one should use the potentials. It can be shown [3.39, 40] that with this approach the energy, and the linear and angular momenta remain constant if the algorithm is set up in a special form. Nevertheless, there is still the discretization error so that the computed trajectory is not the "true" one, even though the energy is conserved. The system will follow an alternative path on the constant-energy surface. It is also required that the potential is the true one. This is, however, not the case for a system enclosed by a finite box. In addition, we may ask for the time-reversal properties. Interestingly enough, only the one-step method is invariant under time reversal if we require that the equations define a canonical transformation [3.41, 42].

We return to the reason for energy fluctuations. Such fluctuations may be produced due to the finite arithmetic of the computer as well as the fi-

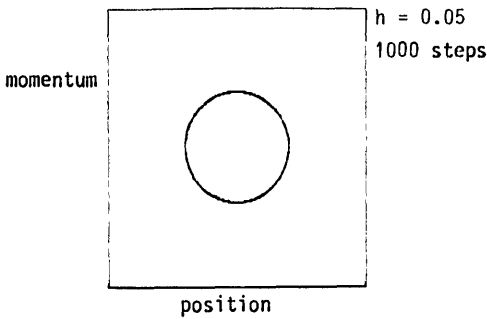


Fig.3.1. Phase space diagram for the spring problem obtained with an order two algorithm

nite step width. Though rounding errors usually play a less important role than the other phenomena, they nevertheless deserve consideration. Associated with each arithmetic operation is a round-off error [3.43]. The result due to an addition is obtained with finite precision so that the last digit is not the true one. Rather, it is the result of rounding. An error is also created on adding two quantities with quite different orders of magnitude (note that on a computer the associative property of addition does not hold!). This can occur in the calculation of the force acting on a particle. Imagine that at least one particle exerts a strongly repelling force, some particles are near the potential minimum giving only a negligible contribution, and the others are far away. Adding the smaller contributions to the dominant repelling force will result in a loss of accuracy of some digits. However, if the summation is carried out by first sorting the contributions according to their magnitude and then summing, beginning with the smallest terms, significant digits are secured.

Let us consider the spring problem from Chap.1 to demonstrate the points made above. Figure 1.3 shows the resulting trajectories using the simple algorithm of order one. In the first case, where $h = 0.05$, the energy is not conserved. The initial total energy was one, and after 1000 steps had reached a value of 12.14! The time step $h = 0.005$ with 10^4 steps already yields a better result, $E = 1.28$. Under the same initial conditions an order-two algorithm gives (Fig.3.1)

$$h = 0.05, \quad 10^3 \text{ steps}, \quad E = 0.9999457,$$

$$h = 0.005, \quad 10^4 \text{ steps}, \quad E = 0.9998397.$$

Though the results are far better, the smaller time step, which should give the better result, actually has less accuracy. This is due to the round-off errors. The calculation was carried out on a personal computer with single precision, yielding 7 significant digits. The order-two algorithm involves a multiplication of h^2 with some other quantity and round-off errors occur. Performing the calculation in double precision gives all significant digits:

$h = 0.05$, 10^3 steps , $E = 0.99995860$,

$h = 0.005$, 10^4 steps , $E = 0.99999956$.

Calculating Thermodynamic Quantities

In computer simulations of physical systems the ensemble average has to be replaced by the time average. In conventional MD simulations the number of particles, N , and the volume V are fixed. Strictly speaking the total linear momentum is another conserved quantity. The total linear momentum is set to zero to avoid motion of the system as a whole. From the equations of motion, given the initial positions $\mathbf{r}^N(0)$ and momenta $\mathbf{p}^N(0)$, a MD algorithm generates the trajectory $(\mathbf{r}^N(t), \mathbf{p}^N(t))$. Assuming that the energy is conserved and that the trajectories spend equal time in all equal volumes with the same energy, the *trajectory average*, defined as

Definition 3.3. Trajectory Average

$$\bar{A} = \lim_{t' \rightarrow \infty} (t' - t_0)^{-1} \int_{t_0}^{t'} dt A(\mathbf{r}^N(t), \mathbf{p}^N(t); V(t)) ,$$

is equal to the microcanonical ensemble average

$$\bar{A} = \langle A \rangle_{NVE} . \quad \square \quad (3.21)$$

In the following we shall always denote ensemble averages by $\langle \cdot \rangle$ and trajectory averages by an overscore. For later application we include a non-constant volume in the definition. For now, the volume does not change in time and has a definite value determined by the number of particles and the density.

The total energy is a conserved quantity for an isolated system. Along any trajectory generated by a molecular-dynamics simulation the energy should remain constant, i.e., $E = \bar{E}$. At this point we have to consider the range of interaction. In general, the range will be longer than the length L of the side of the MD cell and is cut off at $r_c < L/2$. This natural cut-off is, however, not the only one. For computational reasons the potential is usually truncated at a convenient range to reduce the time spent in computing the potential energy. Indeed, if no special precautions are taken 99% of the total execution time required for one MD step can go into the computation of the potentials, i.e., the forces required to propagate the particles.

The cut-off introduces a δ -function singularity in the forces at the point of cut-off if the potential is not smoothly continued to zero. If the potential is given in a tabulated form, this is readily implemented. But the effects of the truncation on the properties of the system must be considered. In non-equilibrium situations, for example metastable states occurring at first-order phase transitions, the range is extremely important. It affects the relaxation of the non-equilibrium into the equilibrium state [3.44, 45].

The cut-off and the approximations made for the differential equations of motion, together with numerical round-off errors, introduce a drift in the energy. The trajectories are then not time reversible either.

The kinetic energy E_k and the potential energy U are not conserved quantities for an isolated system. Their values vary from point to point along the generated trajectory, and we have

$$\bar{E} = \lim_{t' \rightarrow \infty} (t' - t_0)^{-1} \int_{t_0}^{t'} E_k(v(t)) dt , \quad (3.22)$$

$$\bar{U} = \lim_{t' \rightarrow \infty} (t' - t_0)^{-1} \int_{t_0}^{t'} U(r(t)) dt .$$

Let us first look at the kinetic energy. The path generated is not continuous and we have to take the average of the kinetic energy evaluated at the discrete points ν in time

$$\bar{E}_k = \frac{1}{n - n_0} \sum_{\nu > n_0}^n E_k^\nu , \quad (3.23)$$

where

$$E_k^\nu = \sum_i \frac{1}{2} m(v_i^2)^\nu .$$

From the mean kinetic energy we can compute the temperature of the system. As will become apparent later, the temperature is an important quantity to monitor, especially during the initial stages of a simulation.

Recall that we are interested in the computation of observables in the thermodynamic limit. In this limit all ensembles are equal, and we can apply the equipartition theorem.

Theorem 3.4. Equipartition Theorem

If the Hamiltonian is given as in (3.3) we have

$$\frac{1}{2} m v_i^2 = \frac{1}{2} k_B T . \quad \square$$

Since the system has three degrees of freedom per particle (for the moment we ignore constraints such as zero total linear momentum), we obtain

$$\overline{E}_k = \frac{3}{2} N k_B T . \quad (3.24)$$

Assume that the potential has been cut off at r_c . The average internal configurational energy is then given by

$$\overline{U} = \frac{1}{n - n_0} \sum_{\nu > n_0}^n U^\nu , \quad (3.25)$$

where

$$U^\nu = \sum_{i < j} u(r_{ij}^\nu) .$$

Due to the cut-off the total energy and the potential energy entail an error. To estimate the necessary corrections we note that the potential energy is, in general, given by

$$U/N = 2\pi\rho \int_0^\infty u(r)g(r)r^2 dr , \quad (3.26)$$

where $g(r)$ is the pair correlation function and measures the time-independent correlations among the particles. To be precise, $g(r)dr$ is the probability that a particle is found in the volume element dr surrounding r when there is a particle at the origin $r=0$. Let $n(r)$ be the average number of particles situated at a distance between r and $r+\Delta r$ from a given particle, then

$$g(r) = \frac{V}{N} \frac{n(r)}{4\pi r^2 \Delta r} . \quad (3.27)$$

The pair correlation function is easily computed during a simulation. All the distances are available anyway from the calculation of the forces. Since $g(r)$ is time independent one can perform a time average. Figure 3.2 shows $g(r)$ for argon at two points of the phase diagram, as obtained by MD simulations. The pair correlation function is only meaningfully calculated for distances roughly less than half of the linear size of the MD cell.

In (3.25) all the internal configurational energies are summed up to the cut-off distance. For the *tail correction* we can take

$$U_c = 2\pi\rho \int_{r_c}^\infty u(r)g(r)r^2 dr . \quad (3.28)$$

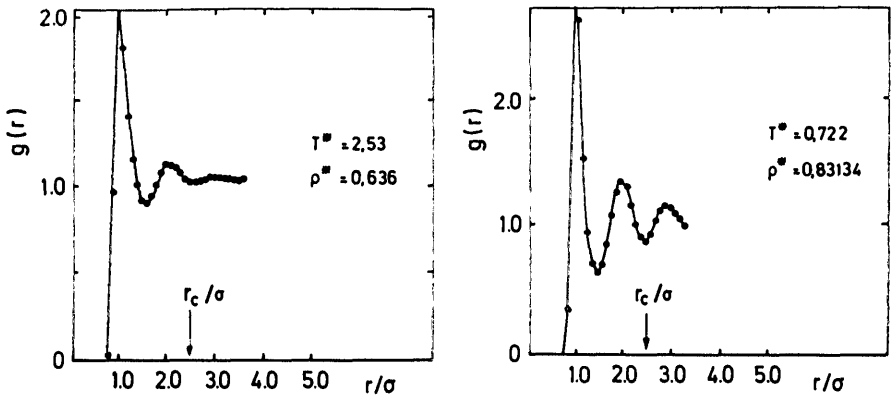


Fig.3.2. Shown are pair correlation functions for two parameter sets as obtained from simulations. The left picture shows a high temperature state $T^* = 2.53$ with a density ρ^* . The right shows the pair correlation function for $T^* = 0.722$ and $\rho^* = 0.831$ (c.f. Example in the next section)

Instead of actually taking $g(r)$ as computed during a simulation, one can also assume that the pair correlation function is identical to unity. The error made in such an approximation will be small if the potential cut-off was not chosen too small. For the results shown in Fig. 3.2 the potential was a Lennard-Jonesian one with a cut-off to the right of the second peak, as indicated by the arrows. There, the pair correlation function is not too far off unity and will stay so for the range extending to infinity.

A tail correction is also necessary for other quantities. As an example we take the calculation of the pressure P for which the virial equation of state holds [3.19].

Theorem 3.5. Virial Equation of State

$$P = \rho k_B T - \frac{\rho^2}{6} \int_0^\infty g(r) \frac{\partial u}{\partial r} 4\pi r^3 dr .$$

As for the computation of the potential energy, we split the integral into a term due to the contributions within the interaction range and a term to correct for the truncation:

$$P = \rho k_B T - \frac{\rho}{6N} \sum_{i < j} r_{ij} \frac{\partial u}{\partial r_{ij}} + P_c . \quad (3.29)$$

The long-range correction is

$$P_c = \frac{\rho^2}{6} \int_{r_c}^\infty g(r) \frac{\partial u}{\partial r} 4\pi r^3 dr . \quad (3.30)$$

In the example of the next section the significance of the corrections to the various quantities will be appreciated. They can amount to several per-cent.

Organization of a Simulation

The actual computer simulation of a molecular system can be broken up into three parts:

- (i) Initialization.
- (ii) Equilibration.
- (iii) Production.

The first part of a simulation is the assignment of the initial conditions. Depending on the algorithm, different sets are required. An algorithm may need two sets of coordinates, one at time zero and one for the previous time step. For the moment assume that to start an algorithm we need the positions and the velocities. The problem one is faced with immediately is that, in general, the initial conditions are not known. Indeed, this is the starting point for a statistical-mechanics treatment! For the computer-simulation approach there are various possible assignments. For definiteness let the initial positions be on a lattice and the velocities drawn from a Boltzmann distribution. The precise choice of the initial conditions is irrelevant, since ultimately the system will lose all memory of the initial state.

A system set up, as outlined above, will not have the desired energy. Secondly, most probably the state does not correspond to an equilibrium state. To promote the system to equilibrium we need the equilibration phase. In this phase, energy is either added or removed until the energy has reached the required value. Energy may be removed or added by stepping the kinetic energy down or up. The system is now allowed to relax into equilibrium by integrating forward the equations of motion for a number of time steps. Equilibrium is established if the system has settled to definite mean values of the kinetic and potential energies.

We can identify at least two potential problems arising in the first two steps. One problem concerns the relaxation time of the system. The basic time step h determines the real time of the simulation. If the intrinsic relaxation time is long, many steps are required in order for the system to reach equilibrium. For some systems the number of time steps may be prohibitively large for the present speed of computers. However, it is possible in some circumstances to circumvent the difficulty by an appropriate scaling of the variables. Examples of where this is possible are systems near second-order phase transitions.

In connection with the relaxation time one has to face the possibility that the system is trapped in a metastable state. Long-lived metastable states may not show an appreciable drift in the kinetic or potential energy. Especially for systems investigated near two-phase coexistence, say between liquid and gas, this danger arises.

The second potential problem is that the system might have been set up in an irrelevant part of the phase space. This problem can be handled by

performing simulations with different initial conditions and different lengths.

The actual computation of the quantities is done in the third part of the simulation. In the production part all quantities of interest are computed along the trajectory of the system in phase space.

In the following we shall study particular algorithms. First we look at methods to deal with the constant energy, constant particle number and constant volume cases. Then we study ways of incorporating into the equations of motion constraints allowing a simulation of a constant temperature rather than of a constant energy. This will follow a discussion of how to compute properties in a constant pressure ensemble.

3.1.1 Microcanonical Ensemble Molecular Dynamics

The basic molecular dynamics algorithm with conserved energy is introduced.

We proceed by developing a computational method to propagate a system along a path of constant energy in the phase space. The starting point is the Hamiltonian describing the interaction of N particles. For simplicity, we assume, as before, a two-body potential with spherical symmetry

$$\mathcal{H} = \frac{1}{2} \sum_i \frac{1}{m} p_i^2 + \sum_{i < j} u(r_{ij}) ,$$

where r_{ij} denotes the distance between particle i and particle j . time does not enter explicitly into the equations. We are considering a system where $\mathcal{H} = E$ is a constant of motion. In addition, we have a constant particle number N and the fourth constraint of zero total linear momentum P .

In classical mechanics the Hamiltonian leads to various forms of the equations of motion. Depending on the choice, the algorithm to solve the equations will have certain features. Though the equations of motion are mathematically equivalent they are not numerically equivalent. Here we start with the Newtonian form

$$\frac{d^2 r_i(t)}{dt^2} = \frac{1}{m} \sum_{i < j} F_i(r_{ij}) . \quad (3.31)$$

Analytically, the solution of the system of second-order differential equations is obtained by integrating twice from time zero to t , to obtain first the velocities and then the positions. Not only the initial positions are required but also the initial velocities. The initial positions fix the contribution of the potential energy to the total energy, and the velocities determine the kinetic-energy contribution. With the specification of the initial conditions the system moves along a path of constant energy in phase space.

To solve the differential equations numerically we use the discretization (3.19) for the second-order differential operator on the left-hand side of (3.31) to get the *explicit central difference method*

$$\frac{d^2 r_i}{dt^2} = \frac{1}{h^2} [r_i(t+h) - 2r_i(t) + r_i(t-h)] = \frac{1}{m} F_i(t). \quad (3.32)$$

This equation provides a prescription for obtaining the positions of the particles at time $t+h$ from the positions at two immediately preceding time steps t and $t-h$ and the forces acting at time t . Solving for the positions at time $t+h$ we get

$$r_i(t+h) = 2r_i(t) - r_i(t-h) + F_i(t)h^2/m. \quad (3.33)$$

Let

$$\begin{aligned} t_n &= nh, \\ r_i^n &= r_i(t_n), \\ F_i^n &= F_i(t_n). \end{aligned}$$

Then (3.33) assumes a more algorithmic form

$$r_i^{n+1} = 2r_i^n - r_i^{n-1} + F_i^n h^2/m. \quad (3.34)$$

Starting from positions r_i^0 and r_i^1 all subsequent positions are determined by the above recursion relation. In other words, the positions of the particles at time $n+1$ are *extrapolated* or *predicted* from the two immediately preceding positions (two-step method).

In the above form the recursion relation produces only the positions. The velocities, however, are needed for the calculation of the kinetic energy and, for example, the velocity auto-correlation function to study transport properties. Following the line of approach used so far, the velocities are computed as, see (3.18),

$$v_i^n = (r_i^{n+1} - r_i^{n-1})/2h. \quad (3.35)$$

Notice that at the $(n+1)$ th step the computed velocities are those of the previous time, i.e., the n th step! Hence, the kinetic energy is one step behind the computed potential energy.

Equations (3.34, 35) together with the initial positions constitute the so-called *Verlet algorithm* [3.6, 7].

Algorithm A2. NVE Molecular Dynamics

- (i) Specify positions r_i^0 and r_i^1 .
- (ii) Compute the forces at time step n : F_i^n .
- (iii) Compute the positions at time step $n+1$ as in (3.34): r_i^{n+1} .
- (iv) Compute the velocities at time step n as in (3.35): v_i^n . □

One advantage of the above algorithm is its time reversibility. Running the system backwards in time leads to the same equations. This is true only in principle. Due to inevitable round-off errors of the finite precision arithmetic, the trajectories depart from their original paths. At each time step there is an addition of the form $O(1)+O(h^2)$, introducing a round-off error. Further, the trajectory departs from the true one because of the finite step size.

In the form of the Verlet algorithm (Algorithm A2) the method is not self-starting. Not only the initial positions must be supplied but also one more set of positions. Sometimes this comes in handy if one sets up a lattice for the initial positions of the N particles and then perturbs it. If the positions and the velocities are initial conditions, the following procedure can be used to calculate the positions at t_1^1 :

$$r_i^1 = r_i^0 + h v_i^0 + \frac{1}{2m} h^2 F_i^0 . \quad (3.36)$$

From then on the algorithm proceeds as presented.

The Verlet algorithm can be reformulated in such a way as to give a numerically more stable method [3.30, 46]. Define

$$z_i^n = (r_i^{n+1} - r_i^n)/h . \quad (3.37)$$

The equations

$$\begin{aligned} r_i^n &= r_i^{n-1} + h z_i^{n-1} , \\ z_i^n &= z_i^{n-1} + m^{-1} h F_i^n \end{aligned} \quad (3.38)$$

are (mathematically) equivalent to (3.34) and are called the *summed form*. A further reformulation yields the *velocity form* of the Verlet algorithm.

Algorithm A3. NVE MD Velocity Form

- (i) Specify the initial positions r_i^1 .
- (ii) Specify the initial velocities v_i^1 .
- (iii) Compute the positions at time step $n+1$ as
 $r_i^{n+1} = r_i^n + h v_i^n + \frac{1}{2} m^{-1} h^2 F_i^n$.
- (iv) Compute the velocities at time step $n+1$ as
 $v_i^{n+1} = v_i^n + h(F_i^{n+1} + F_i^n)/2m$.

The above algorithm is superior to the original one in many ways. Notably, we have succeeded in having the positions and the velocities for the same time step; secondly, the numerical stability is enhanced, which is extremely important for long runs. Yet another feature will show up when we discuss algorithms for the constant temperature ensemble.

In general, one does not know the precise initial conditions corresponding to a given energy. To adjust the system to a given energy, reason-

able initial conditions are supplied and then energy is either drained or added. The procedure is carried out until the system reaches the desired state. For the equilibration phase in the Verlet algorithm, or its variant velocity forms, this is accomplished by an *ad hoc scaling* of the velocities [3.47]. Such a scaling can introduce large changes in the velocities. To eliminate possible effects the system must be given time to establish equilibrium again. Algorithmically the equilibration phase looks like

- (i) Integrate the equations of motion for some time steps.
- (ii) Compute the kinetic and potential energies.
- (iii) If the energy is not equal to that desired, then scale the velocities.
- (iv) Repeat from step 1 until the system has reached equilibrium.

The success of the procedure depends on the initial positions and the distribution of the velocities. A common practice is to set up the system on a lattice and assign velocities according to a Boltzmann distribution. Sometimes, instead of the velocities being scaled, they are all set to zero. In any case, one has to check the velocity distribution after the equilibration phase has been reached to make sure that it has the equilibrium Maxwell-Boltzmann form.

Example 3.1

We study a monatomic system of particles in which the total energy is fixed. In particular, we assume that the interaction between the particles is well represented by a two-body central force interaction of the Lennard-Jones type

$$u(r_{ij}) = 4\epsilon[(\sigma/r_{ij})^{12} - (\sigma/r_{ij})^6], \quad (3.39)$$

where $-\epsilon$ is the minimum of the potential (ϵ specifies the units of energy), which occurs when the distance r is equal to $2^{1/6}\sigma$ (σ specifies the units of length). To be more specific we choose the values of ϵ and σ appropriate for argon (Problems 3.5, 6). We shall take $N = 256$ particles in a box of volume V . Periodic boundary conditions are imposed to conserve the density. This implies that we have to use the minimum image convention (3.2). The volume and the number of particles together with the energy completely specify the point in the phase diagram we wish to study.

To advance the particles inside the MD cell we need to know the forces acting on each particle. For the force in the x direction on the i th particle exerted by the j th particle we obtain from (3.39)

$$F_x(r_{ij}) = 48 \left[\frac{\epsilon}{\sigma^2} \right] (x_i - x_j) \left[\left(\frac{\sigma}{r_{ij}} \right)^{14} - \frac{1}{2} \left(\frac{\sigma}{r_{ij}} \right)^8 \right] \quad (3.40)$$

and similarly for the y and z components. This form of the potential and the force is, however, not suitable for a computer simulation. All quantities are conveniently expressed in a scaled form. Time and positions are scaled by

$$(m\sigma^2/48\epsilon)^{1/2} \quad \text{and} \quad \sigma, \quad (3.41)$$

respectively (m is the mass of the argon atom). This renders the equations dimensionless. Substituting the values for the argon atom into (3.41) the time unit is $3 \cdot 10^{-12}$ s. To ensure a reasonable numerical stability the basic time increment is taken to be $h = 0.064$ or $2 \cdot 10^{-14}$ s. The actual real time will be fairly small since only a limited number of integration steps are possible.

We shall study the argon system at two points in the phase diagram: $(T^*, \rho^*) = (2.53, 0.636)$ and $(0.722, 0.83134)$. For these values of the reduced densities the linear MD cell sizes are $L = 7.38$ and $L = 6.75$, respectively. With these specifications the program can be set up. We use the summed form of the Verlet algorithm to advance the positions. A sample listing of a program is included in Appendix A2.

Initially we assign a face-centred-cubic lattice to the positions of the atoms. To start the algorithm, velocities are drawn from a Maxwell distribution for the appropriate temperature. In Appendix A1 some possible methods for generating such a distribution are listed. Starting with a different distribution, say by assigning random velocities, does not impede the equilibration of the system. Since the MD cell should not move, we must assure a zero total linear momentum. This removes three degrees of freedom from the system and must be taken into account in the calculation of the temperature.

At this point, consideration must be given to the computation of the forces and the potential. To avoid the use of Ewald sums and to speed up the calculation we truncate the potential. To study the effect of the cut-off we use two values, $r_c = 2.5$ and 3.6 . The impact of the truncation on the execution time is quite large. In going from $r_c = 2.5$ to 3.6 the execution time doubles for the densities considered here! The cut-off can further be appreciated by noting that for $r_c = 2.5$ roughly 80% of the total execution time goes into the computation of the forces (this is only true for the algorithm given in the appendix).

Having assigned initial positions and velocities, the system is equilibrated so as to obtain the desired average temperature. The equilibration process is performed by integrating the equations of motion for a certain number of MD steps (here 50). The integration process is then stopped and energy removed or added by an ad hoc readjustment of the velocities. This is done by scaling all the velocities with

$$\beta = \left[T^*(N-1) / 16 \sum_i v_i^2 \right]^{1/2}. \quad (3.42)$$

The procedure is repeated until the desired energy, or equivalently an average temperature, is reached. In these particular simulations the equilibration was performed during the first 1000 MD steps. Figure 3.3 shows the evolution of the kinetic energy for the case $r_c = 2.5$, $T^* = 2.53$, $\rho^* = 0.636$ and for $r_c = 2.5$, $T^* = 0.722$, $\rho^* = 0.83134$. After a few hundred steps the kinetic

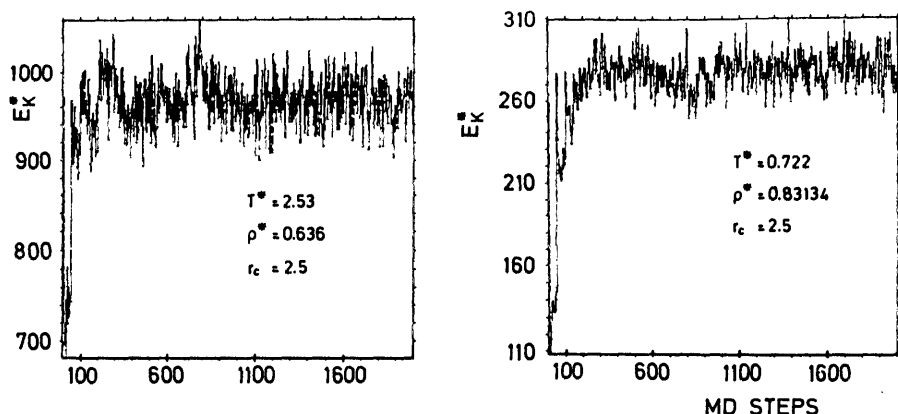


Fig.3.3. Evolution of the kinetic energy during a molecular dynamics simulation. During the first 1000 MD steps the velocities were scaled every 50'th step so as to give the desired temperatures. All quantities are given in reduced units

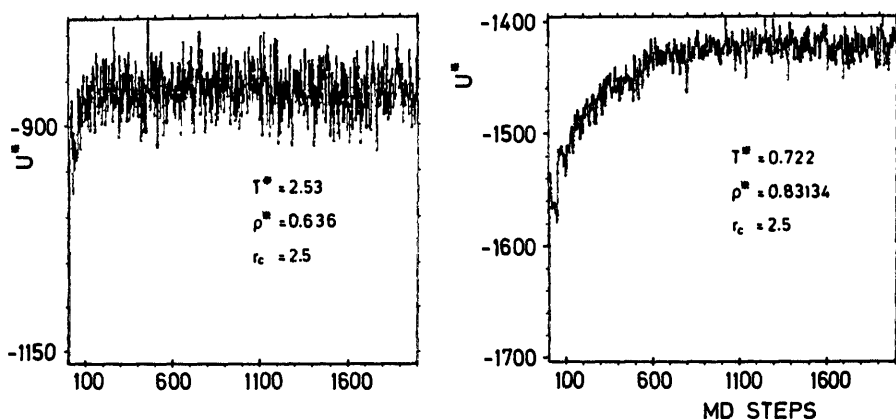


Fig.3.4. Time dependence of the potential energy (in units of ϵ)

energy has already settled to a definite average. However, looking at the potential energy (Fig.3.4) we observe that the overall relaxation of the system is much slower in the case $T^* = 0.722$.

Figure 3.5 shows the evolution of the total energy. At each rescaling of velocities the energy jumps to a different value. Between scaling of velocities when the system is not tampered with, the energy remains almost constant. That the energy does not remain exactly constant is due to the fact that we did not smoothly continue the potential to zero. A fluctuation in energy is introduced each time a particle comes within the cut-off range. In addition, there are the unavoidable round-off errors due to the finite precision of the host machine. These kinds of errors are kept to a minimum due to the numerically more stable algorithm.

To check that the system does indeed behave as expected, we look at the distribution of the kinetic-energy values encountered during the sim-

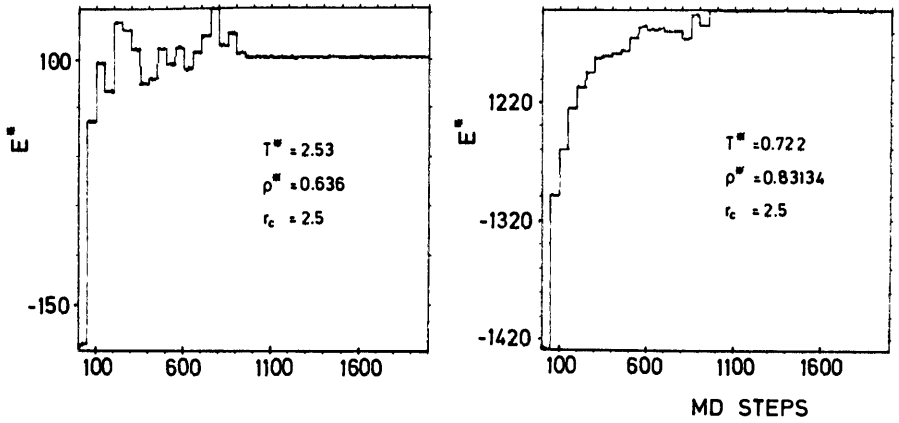


Fig.3.5. Evolution of the total energy. The jumps in the energy indicate the scaling of the velocities

ulation. We also monitor the average speed of the particles. Figure 3.6 shows the distribution of the kinetic energies for the case $r_c = 2.5$, $T^* = 2.53$ and $\rho^* = 0.636$.

For the average speed we must have

$$v = 1.13 \sqrt{2k_B T/m}$$

or in reduced units

$$v' = 1.13 \sqrt{T^*/24},$$

i.e., $v' = 0.3668$ and the simulation result is $v' = 0.3654$ (see Table 3.1 for the other results). The agreement is quite good considering that the system is composed of only 256 particles. Also, the percentage of particles having a speed larger than the mean speed agrees with the expected 46.7%. These results are fairly insensitive to the actual cut-off. Within the statistical error

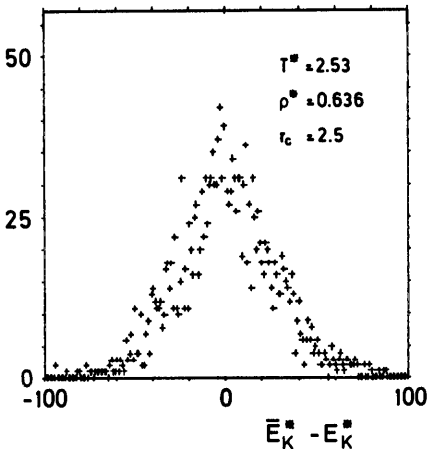


Fig.3.6. The figure shows the distribution of the kinetic energy values during the simulation

Table 3.1. Results from the molecular dynamics simulation of argon. The quantities are averages over 1000 MD steps. The errors are the standard deviations

Equilibration to $T^* = 2.53$ and $\rho^* = 0.636$			
r_c	E_k^*	U^*	E
2.5	966.58 ± 22.1	-864.78 ± 22.4	101.79
3.6	972.15 ± 22.6	-920.10 ± 22.9	52.05
r_c	T^*	v'	% above v'
2.5	2.53 ± 0.06	0.3654 ± 0.007	46.33
3.6	2.54 ± 0.06	0.3667 ± 0.007	46.71
Equilibration to $T^* = 0.722$ and $\rho^* = 0.83134$			
r_c	E_k^*	U^*	E
2.5	279.13 ± 9.57	-1421.98 ± 20.15	-1142.92
3.6	275.11 ± 9.72	-1496.45 ± 21.61	-1221.38
r_c	T^*	v'	% above v'
2.5	0.7297 ± 0.025	0.1965 ± 0.003	47.08
3.6	0.7192 ± 0.025	0.1949 ± 0.003	46.42

they are the same. However, other quantities may be sensitive to the range of interaction.

The dependence on the interaction range is, of course, most readily seen in the potential energy itself. For the high-temperature state the results are

$$U^* = -864.78, \quad r_c = 2.5;$$

$$U^* = -920.10, \quad r_c = 3.5.$$

The smaller range yields an average internal configurational energy of 94% of that for the larger. Recall that we discussed the correction necessary for the potential energy if the potential has been truncated. If we apply the result (3.28) with the Lennard-Jones potential, setting the pair correlation function to unity, we get the corrections

$$U_c^* = -83.94, \quad r_c = 2.5;$$

$$U_c^* = -25.99, \quad r_c = 3.6.$$

The corrections are quite significant; they are 9.7% and 2.8%, respectively. $\square$

The results of the simulation in Example 3.1 represents one particular realization out of a multitude of possible ones. Starting from a different set of initial positions and velocities the system would have followed an alter-

native path on the constant energy surface. Contenting ourselves with one path, we rely on (3.21), i.e., that the trajectory average is equal to the ensemble average. In principle we should have followed the path for an infinitely long time to ensure that the system spends equal time in all equal volumes of the phase space. The limitation of the finite computer time does not permit this. We sampled some region in phase space, so it follows that there will be an error involved in the results. It might be that the path sampled an irrelevant part of the phase space. For example, the initial conditions might be such that the system is set up in an irrelevant part. If the duration of the simulation is too small, the system does not leave the irrelevant part, or only just enters the relevant part. Simulations of different lengths must be made in order to assess the error, i.e., to determine whether the asymptotic behaviour has set in. These remarks also apply to the other methods presented in this text.

Apart from the accuracy and numerical-stability considerations, an important factor in molecular dynamics simulations is the calculation of the force acting on the particles. The integration steps require of order N operations. For two-body additive central forces one has to evaluate $\frac{1}{2}N(N-1)$ terms at each step. To reduce the computational complexity we can exploit the fact that most of the terms in the evaluation turn out to be zero if the potential has a cut-off. Only those terms where the particles are within the cut-off range r_c give contributions.

By choosing a suitable radius r_m we can ensure that only after n time steps does the number of particles inside this sphere change [3.3,7,48] (Problem 3.7). Hence, producing a list of nearest neighbours reduces the evaluation of the force term (3.40). Only those particles in the list of a given particle contribute. Every n th step the table must be updated. The trade-off is, of course, computer storage.

The "*Verlet table*" has been used successfully on general purpose computers. For vector machines the technique has, however, drawbacks. A further discussion of time-saving techniques is deferred to the appendix.

3.1.2 Canonical Ensemble Molecular Dynamics

Various possibilities are discussed for a simulation with a constant temperature, instead of constant energy. Presented are the velocity scaling, isokinetic and damped force methods.

In the previous subsection we saw how the MD method solves the equations of motion numerically. The system under consideration was isolated, i.e., conservative, so the trajectory always stayed on a surface of constant energy in phase space. In many circumstances it is desirable to investigate a system along an isotherm rather than along a line of constant energy. Since the equations of motion allow propagation only on the constant-energy surface we have to modify the equations. The modification has to be such that the system will be conceptually coupled to a heat bath. The heat bath intro-

duces the energy fluctuations which are necessary to keep a fixed temperature.

Generally, observables appear as averages over an appropriate ensemble of similar systems. The appropriate ensemble here, representing equilibrium of a system in a heat bath, is the *canonical ensemble*, where the particle number N , the volume V and the temperature T are fixed, and there is zero total linear momentum P . Since the total energy is not a conserved quantity for constant temperature, schemes have to be devised to introduce fluctuations in the total energy E . However, the average kinetic energy is a constant of motion due to its coupling with the temperature, see (3.24). Any scheme has to satisfy the requirement that the *average* properties computed along a trajectory must be equal to the ensemble average

$$\langle A \rangle_{NVT} = \lim_{t' \rightarrow \infty} \frac{1}{t' - t_0} \int_{t_0}^{t'} A(\mathbf{r}^N(t), \mathbf{p}^N(t); V(t)) dt. \quad (3.43)$$

One way of achieving energy fluctuations for a constant temperature is to supplement the equations of motion with an equation of constraint [3.49]. Alternatively one can add to the forces in the equations of motion a force of constraint (*damped-force method*) [3.50-55]. It can be shown [3.49] that the damped-force method is a special case of the constraint method. Another possibility is that of immersing the system in a heat bath by introducing a stochastic force simulating collisions with virtual particles. In Sect. 4.2 we take up the idea of stochastic supplements to the equations of motion.

A natural choice for the constraint is to fix the kinetic energy to a given value during the course of a simulation. Such a constraint may be the *non-holonomic constraint* [3.49]

$$\Lambda = \frac{1}{2} \sum_i m v_i^2 = \text{const} \quad (3.44)$$

(*isokinetic MD*) or one may take the total kinetic energy proportional to time with a vanishing proportionality constant if the system has reached a constant temperature (*Gaussian isokinetic MD*) [3.56]

$$\frac{1}{2} \sum_i m v_i^2 = \alpha t. \quad (3.45)$$

Below, we shall adopt the isokinetic approach. Note that only the average temperature is fixed.

We have already encountered a method to constrain the kinetic energy to a given value. To equilibrate the system, energy was drained or added by an *ad hoc scaling* of the velocities [3.47, 57, 58]. After reaching the desired energy or temperature the system was left to itself. Algorithmically this is

cast into the following form, assuming a velocity form of the integration procedure (Algorithm A3 and Problem 3.4), i.e.,

```

do n = 1, max                                     time step loop
1. Compute the forces.
2. Compute  $\mathbf{r}^{n+1} = \mathbf{g}_1(\mathbf{r}^n, \mathbf{v}^n, \mathbf{F}^n)$ .
3. Compute  $\mathbf{v}^{n+1} = \mathbf{g}_2(\mathbf{v}^n, \mathbf{F}^n, (\mathbf{F}^{n+1}))$ .
4. Compute the kinetic energy.
5. Scale the velocities  $\mathbf{v}^{n+1} \leftarrow \mathbf{v}^{n+1}\beta$ .
end                                               time step loop

```

The functions $\mathbf{g}_1$ and $\mathbf{g}_2$ denote the recursion relations. Note that $\mathbf{g}_2$ can involve an additional dependence on the force at time $n+1$. In this case step 1 is introduced between 2 and 3. A bypass of step 5 is created after the equilibration phase. In the *ad hoc velocity scaling method* step 5 remains within the flow of the algorithm and scales the velocities at every time step.

What is the appropriate scaling factor β ? The system has $3N$ degrees of freedom. However, we require the system to have zero total linear momentum, so removing three degrees of freedom. The constraint of constant kinetic energy removes one more degree of freedom. Hence, the scaling factor is

$$\beta = \left[(3N - 4) k_B T_{\text{ref}} / \sum_i m v_i^2 \right]^{1/2} \quad (3.46)$$

so that after the scaling step we have

$$\frac{1}{2} \sum_i m v_i^2 = \frac{1}{2} (3N - 4) k_B T_{\text{ref}} .$$

Traditionally a weighting factor $3N$ has been used instead of $3N-4$. The reason is that there are several problems associated with the procedure, making it non-exact, although, as we will see later, it is exact in its differential form. To reveal why it is not exact, let us study the *ad hoc scaling* within the *leapfrog formulation* of the Verlet algorithm (Problem 3.4)

$$\begin{aligned}
v_i^{n-1/2} &= h^{-1}(\mathbf{r}_i^n - \mathbf{r}_i^{n-1}) , \\
v_i^{n+1/2} &= v_i^{n-1/2} \beta + m^{-1} h \mathbf{F}_i^n , \\
\mathbf{r}_i^{n+1} &= \mathbf{r}_i^n + v_i^{n+1/2} h , \\
v_i^n &= \frac{1}{2}(v_i^{n-1/2} + v_i^{n+1/2}) .
\end{aligned} \quad (3.47)$$

Assume that the scaling factor was computed from the previous half-step velocity

$$\beta = \left[3N k_B T_{\text{ref}} / \sum_i m (v_i^{n-1/2})^2 \right]^{1/2} . \quad (3.48)$$

It is not guaranteed that the kinetic energy computed with the time-step velocities is the same as for the previous half-time step. This is due to the time delay in the feedback loop controlling the temperature. The temperature will fluctuate to a certain degree and one finds [3.59]

$$\langle T^{n-1/2} \rangle \neq \langle T^n \rangle .$$

This is why we use T_{ref} instead of the actual temperature at which we want the system to be in the formula (3.46) for the scaling factor. Therefore it is understandable that it does not make too much of a difference if one uses $3N$ instead of $3N-4$. The discrepancy is adjustable through the parameter T_{ref} .

Algorithm A4. NVT Molecular Dynamics

1. Specify the initial positions r_i^1 .
2. Specify the initial velocities v_i^1 .
3. Compute the positions at time step $n+1$ as

$$r_i^{n+1} = r_i^n + h v_i^n + h^2 F_i^n / 2m.$$
4. Compute the velocities at time step $n+1$ as

$$v_i^{n+1} = v_i^n + h(F_i^{n+1} + F_i^n) / 2m.$$
5. Compute $\sum_i m(v_i^{n+1})^2$ and the scaling factor β .
6. Scale all velocities $v_i^{n+1} \leftarrow v_i^{n+1} \beta$. □

We shall now show how the method is derived [3.49]. We start with a Lagrangian formulation of the isolated unconstrained system

$$\mathcal{L} = \frac{1}{2} \sum_i m \dot{r}_i^2 - U(r) \quad (3.49)$$

with the Lagrangian equations of motion

$$\frac{\partial \mathcal{L}}{\partial r_i} - \frac{d}{dt} \frac{\partial \mathcal{L}}{\partial \dot{r}_i} = 0. \quad (3.50)$$

In this form the energy is a constant of motion. To introduce energy fluctuations we couple the system with an energy reservoir by introducing a generalized force, i.e.,

$$\frac{\partial \mathcal{L}}{\partial r_i} - \frac{d}{dt} \frac{\partial \mathcal{L}}{\partial \dot{r}_i} = - F(r_i, \dot{r}_i) \quad (3.51)$$

Two possibilities arise. We may consider generalized forces derived from generalized potentials or just take them as given. In general, for non-holonomic constraints a generalized potential cannot be constructed. However, the relative simplicity of the constraint here allows a construction. Let $V(r, \dot{r})$ denote the generalized potential, then

$$F(r_i, \dot{r}_r) = - \frac{\partial V}{\partial r_i} + \frac{d}{dt} \frac{\partial V}{\partial \dot{r}_i} , \quad (3.52)$$

yielding unconstrained equations of motion

$$\frac{\partial \mathcal{L}'}{\partial r_i} - \frac{d}{dt} \frac{\partial \mathcal{L}'}{\partial \dot{r}_i} = 0 , \quad (3.53)$$

with a new Lagrangian $\mathcal{L}' = \mathcal{L} - V$. To proceed we assume a simple form of V as a product of two functions

$$V = \xi(r, \dot{r}) \phi(r, \dot{r}) . \quad (3.54)$$

We imagine that ϕ represents the mechanism of energy transfer between the system and the reservoir. The function ξ ensures the fulfilment of the constraint. The detailed mechanism has still to be specified. With (3.54) the equations of motion are calculated as

$$\begin{aligned} m\dot{r}_i &= p_i + \xi \frac{\partial \phi}{\partial \dot{r}_i} + \phi \frac{\partial \xi}{\partial \dot{r}_i} , \\ \dot{p}_i &= - \frac{\partial U}{\partial r_i} - \xi \frac{\partial \phi}{\partial r_i} - \phi \frac{\partial \xi}{\partial r_i} , \end{aligned} \quad (3.55)$$

where $p_i = \partial \mathcal{L}' / \partial \dot{r}_i$. Let us now exploit the arbitrariness in the mechanism of energy transfer and assume that ϕ is a function of the velocities only and that formally ϕ is zero

$$\phi = \phi(\dot{r}) = 0 . \quad (3.56)$$

The equations of motion reduce to

$$\begin{aligned} m\dot{r}_i &= p_i + \xi \frac{\partial \phi}{\partial \dot{r}_i} , \\ \dot{p}_i &= - \frac{\partial U}{\partial r_i} . \end{aligned} \quad (3.57)$$

An obvious choice for ϕ , of course, is to take the constraint itself

$$\phi = \frac{1}{2} \sum_i m\dot{r}_i^2 - \Lambda = 0 \quad (3.58)$$

so that the equations of motion become

$$m\dot{r}_i = \frac{p_i}{m} \sqrt{\frac{2m\Lambda}{\sum_j p_j^2}}, \quad (3.59)$$

$$\dot{p}_i = -\frac{\partial U}{\partial r_i}.$$

We see that the introduction of energy fluctuations through a generalized potential with a specific choice of the detailed coupling leads to the velocity scaling mechanism. To constrain the kinetic energy a feedback loop is established. A problem arises when we discretize the differential equations. The discretization introduces a time delay in the feedback loop, leading to fluctuations in the average kinetic energy.

Interestingly, the second possibility leads to a scaling mechanism, too. If one assumes a generalized force of the type

$$F = \xi(r, \dot{r}) \phi(r, \dot{r}), \quad (3.60)$$

with the specific choice

$$F = \dot{r}_i^n, \quad (3.61)$$

one finds

$$m\dot{r}_i = p_i,$$

$$\dot{p}_i = -\frac{\partial U}{\partial r_i} + r_i^n \beta, \quad (3.62)$$

$$\beta = \dot{U} / \sum_i \dot{r}_i r_i^{n+1}.$$

For $n = 1$ the equations of motion are those obtained by *Hoover et al.* [3.51-55]. In this case the equations of motion conform with the *Gauss principle of least constraint* [3.59, 64]. Note that β does not reference the required temperature so that the initial conditions must be chosen in accordance with the constraint.

Example 3.2

In Example 3.1 (Sect.3.1.1) we studied a monatomic system consisting of 256 particles interacting with a Lennard-Jones potential. The simulation proceeded in such a way that energy was added or removed until a desired energy was achieved, corresponding to an average temperature. The energy remained constant during the rest of the simulation. To keep the temperature fixed and let the total internal energy fluctuate, the system must be brought in contact with a heat bath. One way of achieving this is to constrain the kinetic energy to a fixed value during the course of the simulation.

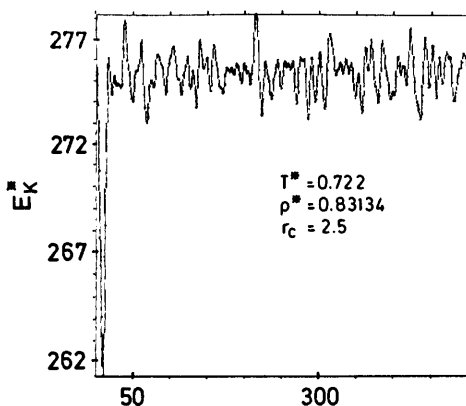


Fig.3.7. Shown is the evolution of the (reduced) kinetic energy computed after the scaling of the minus half step velocities in the leap frog formulation of the Verlet algorithm. Time is given in molecular dynamics steps

In the text we discussed two possible implementations. The first was to rescale the "minus half-step velocities" of the leapfrog algorithm at every time step. Figure 3.7 shows the kinetic energy calculated after the scaling and after the "plus half-step velocities" were computed. Evidently the kinetic energy is not stationary. There are over- and undershootings due to the time delay in the feedback loop. This is also the case if one applies the summed form of the Verlet algorithm. However, the fluctuations are somewhat less significant. □

It is also instructive to observe the different behaviour in the relaxation of the potential energy (Fig.3.8). To compare the results for the constant energy with the isokinetic simulations, the system was prepared with exactly the same initial conditions for the positions and the velocities. A summary of the results is given in Table 3.2. A glance at this table shows that there is no observable difference for the high- and low-temperature states. The average potential energies, computed during the second half of altogether 2000 steps, agree within the statistical uncertainty. Hence such thermodynamic variables are unaffected by the scaling.

It has been shown that the static properties computed along a trajectory indeed conform to the canonical ensemble averages [3.60-63]. Also the results from the above example bear out that static properties are invariant. The question neglected so far is how are the dynamic properties of the system influenced by any of the schemes? We may ask whether the transport properties are affected by the changes in the velocities imposed by the

Table 3.2. Results from isokinetic molecular dynamic simulations for the reduced potential energy. r_c gives the cut-off of the Lennard-Jones potential

$T^* = 2.53$ and $\rho^* = 0.636$		$T^* = 0.722$ and $\rho^* = 0.83134$	
r_c	U^*	r_c	U^*
2.5	-870.32 ± 26.54	2.5	-1423.01 ± 21.4
3.6	-922.12 ± 25.24	3.6	-1493.57 ± 23.43

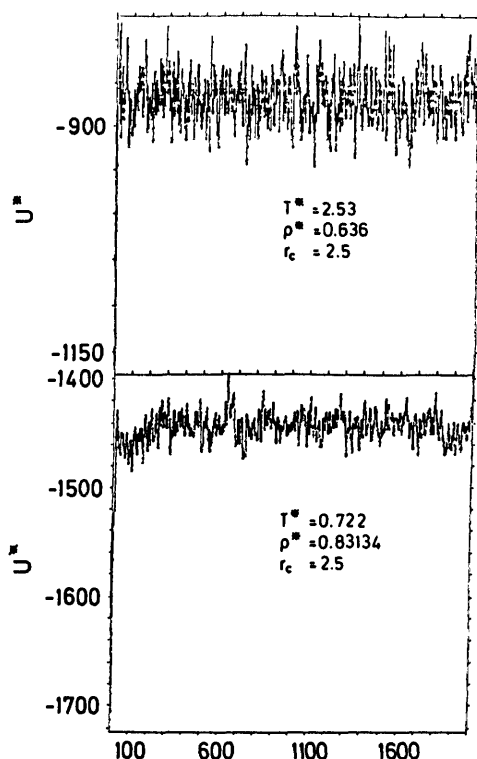


Fig.3.8. Reduced potential energy as a function of time (MD steps) observed in the isokinetic MD using the summed form algorithm

scaling. Clear-cut evidence from simulations does not exist yet. From an "experimental" point of view, possible effects due to the scaling are hard to disentangle from effects coming from the boundary condition imposed on the system. Furthermore, other finite-size effects are possible, not forgetting the potential cut-off. Also, analytically, no proof has been given that the algorithm becomes exact in the limit of infinite time.

3.1.3 Isothermal-Isobaric Ensemble Molecular Dynamics

Two algorithms are given for a simulation in which the temperature and the pressure are held constant.

A constant temperature and/or constant pressure molecular dynamics method is interesting not only from a theoretical point of view. The computation and the comparison of certain quantities with experimental observations sometimes require such an ensemble. One example is, of course, the specific heat C_p at constant pressure.

Let us begin with the constant pressure case. For the isolated N -particle system the energy E and the volume V are the independent variables. If we fix the pressure, then the volume, being the variable conjugate to the

pressure, must be allowed to fluctuate. The system is not isolated anymore but in contact with the exterior. Assume that the transfer between the system and the exterior is adiabatic. In this situation, having a constant particle number N and constant pressure P , the total internal energy is not conserved. The conserved quantity is the *enthalpy* H

$$H = E + P_E V . \quad (3.63)$$

Here, P_E is the externally applied pressure. In mechanical equilibrium the external pressure and the internal pressure are equal.

What we are dealing with is the isobaric-isoenthalpic ensemble (N, P, H) . As was the case for the canonical ensemble, we have to modify the equations of motion to allow for the constant pressure. Any modification has to be such that the average properties computed along the generated trajectory are those of the isobaric-isoenthalpic ensemble

$$\langle A \rangle_{NPH} = \lim_{t' \rightarrow \infty} \frac{1}{t' - t_0} \int_{t_0}^{t'} dt A(\mathbf{r}^N, \mathbf{v}^N; V(t)) . \quad (3.64)$$

We shall keep a cubic volume and maintain periodic boundary conditions. In principle, shape fluctuations could be allowed [3.24-27], which are important for crystal structures, but the formulae would become unduly complicated.

To make the volume fluctuations possible we introduce the volume V as a new dynamical variable. As such it is also assigned a mass M . To develop equations of motion for the particles and the volume we further take $P_E V$ as the potential energy corresponding to the new dynamic variable [3.59, 65, 66]. The Lagrangian now looks like

$$\mathcal{L}(\mathbf{r}, \dot{\mathbf{r}}, V, \dot{V}) = \frac{1}{2} \sum_i m \dot{\mathbf{r}}_i^2 - U(\mathbf{r}) + \frac{1}{2} M \dot{V}^2 + P_E V .$$

Of course, the variables $\mathbf{r}$ and V are not coupled. To proceed we appeal to intuition. If the system is subjected to pressure, the distances between the particles will change (Theorem 3.5). Conversely, if the distances change, the pressure changes. The crucial step is the replacement of the coordinates $\mathbf{r}_i$ of the particles by the scaled coordinates ρ_i , i.e.,

$$\rho_i = \mathbf{r}_i / V^{1/3} = \mathbf{r}_i / L . \quad (3.65)$$

Now all components of the position vectors of the particles are dimensionless numbers within the unit interval $[0, 1]$. With the transformation, the integrals of $\mathbf{r}_i$ over the fluctuating volume V become integrals of ρ_i over the unit cube. Having written down (3.65) we have made the implicit assumption that each spatial point responds in the same way. Due to this, there is no consistent physical interpretation of the approach.

The equation (3.65) couples the dynamical variables $\mathbf{r}$ to the volume. Taking the first time derivative we obtain

$$\dot{\mathbf{r}}_i = L\dot{\rho}_i + \rho\dot{\mathbf{L}}. \quad (3.66)$$

In equilibrium, the changes in the volume can be regarded as slow. Therefore we may assume

$$\mathbf{p}_i/m = L\dot{\rho}_i \quad (3.67)$$

as the momentum conjugate to ρ_i and the Lagrangian becomes

$$\mathcal{L}(\rho, \dot{\rho}, V, \dot{V}) = \frac{1}{2}L^2 \sum_i m\dot{\rho}_i^2 - U(L\rho) + \frac{1}{2}M\dot{V}^2 - P_E V.$$

Recall from Chap.2 that we anticipate possible effects on the intrinsic dynamics when we modify the equations. However, the static properties should not be affected. Concerning this point, note that the potential energy does not involve the new coordinates ρ but the true $\mathbf{r}$.

In a somewhat loose way the Hamiltonian of the system is formulated as [3.65, 66]

$$\mathcal{H} = \frac{L^2}{2} \sum_i m\dot{\rho}_i^2 + U(\rho L) + \frac{1}{2}M\dot{V}^2 + P_E V. \quad (3.68)$$

Here M is still a free parameter, about which we will have more to say later. Having set up the Hamiltonian the next task is to derive the equations of motion for the particles and the volume. These equations will now be coupled. In the Newtonian formulation they are

$$\frac{d^2 \rho_i}{dt^2} = \frac{F_i}{mL} - \frac{2}{3}\dot{\rho}_i \left(\frac{\dot{V}}{V} \right), \quad (3.69)$$

$$\frac{d^2 V}{dt^2} = \frac{P - P_E}{M},$$

with the pressure P computed from the virial

$$P = \frac{1}{3L} \left[\sum_i m\dot{\rho}_i^2 + \sum_{i < j} \mathbf{r}_{ij} F_{ij} \right]. \quad (3.70)$$

These equations yield a constant average pressure. We have two frames to consider. The first is the frame with the original coordinates. The calculation of the forces, the energy and other structural quantities must be car-

ried out in this frame. The second frame with the scaled coordinates is needed for the evolution of the system.

For (3.69, 70) we can immediately write down an algorithm. What is needed are only minor modifications of the summed form algorithm. There is, however, a problem due to the appearance of the first derivative of the position on the right-hand side of (3.69). Recall that the algorithm was developed for equations of the form

$$\frac{d^2 \mathbf{r}}{dt^2} = \mathbf{f}(\mathbf{r}) .$$

Assuming that the algorithm is still numerically stable with the inclusion of a first derivative, i.e. a velocity, on the right-hand side, we obtain for the positions and the volume at time $n+1$

$$\rho^{n+1} = \rho^n + h\dot{\rho}^n + \frac{1}{2} \frac{h^2 F^n}{mL^n} - \frac{1}{2} h^2 \rho^n \frac{\dot{V}^n}{V^n} , \quad (3.71)$$

$$V^{n+1} = V^n + h\dot{V}^n + \frac{1}{2M} h^2 (P^n - P_E) .$$

To compute the velocities and the volume velocity we take first the partial velocities

$$h\dot{\rho}'^{n+1} = h\dot{\rho}^n + \frac{1}{2} \frac{h^2 F^n}{mL^n} - \frac{1}{2} h^2 \dot{\rho}^n \frac{\dot{V}^n}{V^n} , \quad (3.72)$$

$$h\dot{V}'^{n+1} = h\dot{V}^n + \frac{1}{2M} h^2 (P^n - P_E) .$$

The next step is to compute $\frac{1}{2} h^2 F^{n+1}$ and $\Sigma r_{ij} F(r_{ij})$. At this stage another problem presents itself. To compute the pressure at the $(n+1)$ th step the velocities of the $(n+1)$ th step are required! To circumvent the computation of an extrapolation we simply take the partial velocities to estimate the kinetic energy. Using this approximation the velocities are

$$h\dot{V}^{n+1} = h\dot{V}'^{n+1} + \frac{1}{2M} h^2 (P^{n+1} - P_E) , \quad (3.73)$$

$$h\dot{\rho}^{n+1} = h\dot{\rho}'^{n+1} + \frac{1}{2} \frac{h^2 F^{n+1}}{mL^{n+1}} - \frac{1}{2} h^2 \dot{\rho}'^{n+1} \frac{\dot{V}^{n+1}}{V^{n+1}} .$$

Note that there is no rigorous proof for the validity of the procedure.

Let us formulate the algorithm developed as:

Algorithm A5. NPH Molecular Dynamics

1. Specify the initial positions and velocities.
2. Specify an initial volume V^0 consistent with the required density.
3. Specify an initial velocity for the volume, for example $\dot{V} = 0$.
4. Compute ρ^{n+1} and V^{n+1} according to (3.71).
5. Compute the partial velocities for the particles and the volume according to (3.72).
6. Compute the forces and the potential part of the virial.
7. Compute the pressure P^{n+1} using the partial velocities.
8. Compute the volume velocity.
9. Compute the particle velocities using the partial velocities.

We shall investigate the algorithm in the following example.

Example 3.3

As a system to test the Algorithm A5 we choose again argon with $N = 256$ particles and a potential cut-off at $r_c = 2.5$. The initial conditions for the positions and the velocities are identical to those in the previous examples. As the reference temperature we take $T^* = 2.53$ and the initial density $\rho^* = 0.636$. To equilibrate the system energy, i.e., to arrive at the reference temperature, all velocities are rescaled every 50th step.

We now have to consider the choice of the mass M . Notice from Fig. 3.9 that the initial pressure is negative. Hence the initial conditions are such that the system would like to contract. A negative pressure is not unphysical since the initial conditions, in general, do not correspond to equilibrium. At equilibrium the pressure has to be positive. On choosing a mass which is too small, the contraction results in a catastrophic overshooting of the volume. A similar observation was made by *Smith* [3.67]. In the particular examples depicted in Figs.3.9,10 the mass M^* is 0.01 (notice that the mass M is a

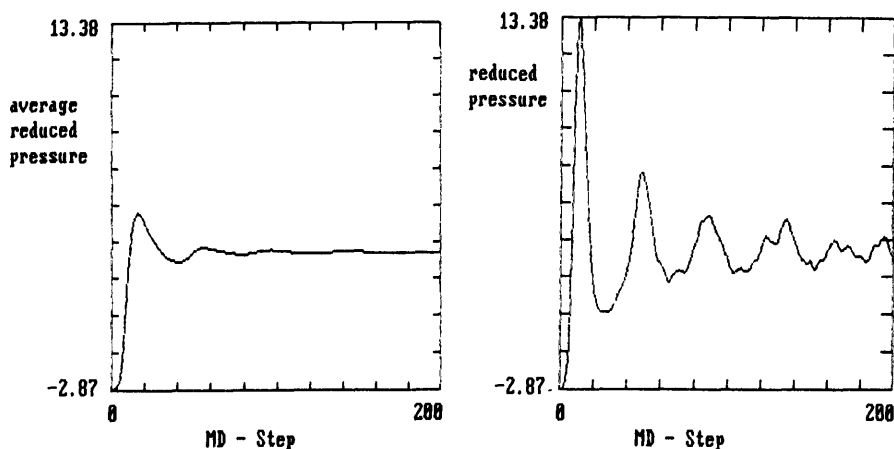


Fig.3.9. The right figure shows how the internal pressure relaxes towards the value given by the external pressure. In the left plot is shown the average of the pressure

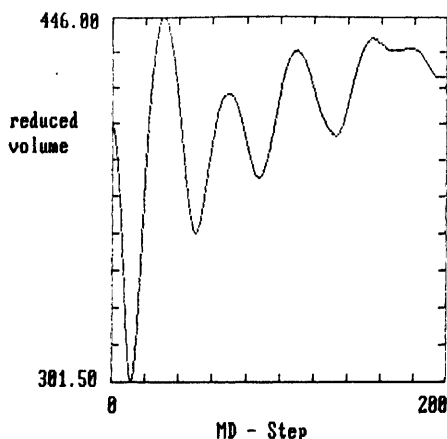


Fig.3.10. Shown is the evolution of the volume during the initial phase of a constant pressure MD simulation

reduced mass; $M^* = M\rho^4/m$). On the other hand, if the mass is too large the system develops long-wavelength fluctuations in the volume [3.66]. In the case studied here the system also shows fluctuations extending over many MD steps. This is, indeed, expected for any finite M [3.65]. The value of M determines the time scale for the volume fluctuations.

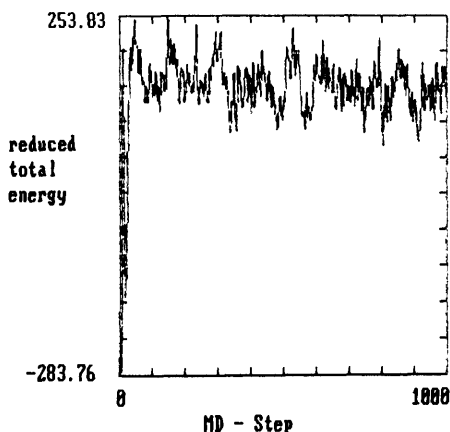
The relaxation behaviour of the pressure is interesting (Fig. 3.9). The initial large fluctuations decay very rapidly. Looking at the average reduced pressure, i.e., the average as a function of time, we see that the settling to a constant pressure sets in very early. However, there are still fluctuations. The magnitude of these depends on the chosen mass M^* [3.66]. $\square$

In the NPH molecular-dynamics algorithm there is one free parameter M . In the example we saw that its magnitude influences the relaxation to equilibrium. Not only are the pressure and the volume affected but also the kinetic energy [3.66] (note that in equilibrium the pressure relaxes more rapidly than the temperature [3.19]).

Unfortunately there is no criterion available for an appropriate choice. Indeed, it is difficult to develop such a criterion. As seen in the example, M must depend on the precise initial conditions. Furthermore, it is not yet established how far dynamical properties of the system, such as transport coefficients, are affected by the magnitude of M . Static properties are independent of M [3.65, 66].

To keep the impact as small as possible M has to be small. It is therefore desirable to have an algorithm which changes M from step to step. Initially M should be large, to compensate negative pressures, and gradually decrease as the system equilibrates. It follows that M should be coupled to the pressure difference.

Up to now we have considered an ensemble where the particle number, the pressure and the enthalpy are the independent thermodynamic variables. Such an isobaric-isoenthalpic ensemble is rather unusual and instead of a constant enthalpy we introduce now a constant temperature.



3.11. Total internal energy E^* as a function of time (MD-steps)

In Sect.3.1.2 we achieved a constant temperature during the MD simulation by rescaling all the velocities at every time step. This constrains the kinetic energy to a fixed value and gives a desired temperature (Algorithm A4). The same idea can be used also for the (N,P,T) ensemble algorithm. We combine the algorithm described above for constant pressure with velocity scaling.

Algorithm A6. NPT Molecular Dynamics

1-9. As in Algorithm A5.

10. Rescale all velocities as in step 8 of Algorithm A5.

Indeed, Algorithm A6 requires only a trivial change, since the only modification one has to make is to proceed with the equilibration phase, with the scaling being performed at each step, until the end of the simulation.

Example 3.4

The conditions in this example are exactly the same as in the preceding one. Instead of performing the scaling at every 50th step, the scaling is done every step.

In Fig.3.11 the total internal energy $E^* = E_k^* + U^*$ is shown. As in the example of Sect.3.1.2 the energy relaxes very quickly.

A quick relaxation is also seen in the pressure (Fig.3.12) and in the volume (Fig.3.13). □

Problems

3.1 The truncated octahedron boundary condition [3.68,69] is obtained by cutting off the corners of a cube of side $2A$ until half its volume

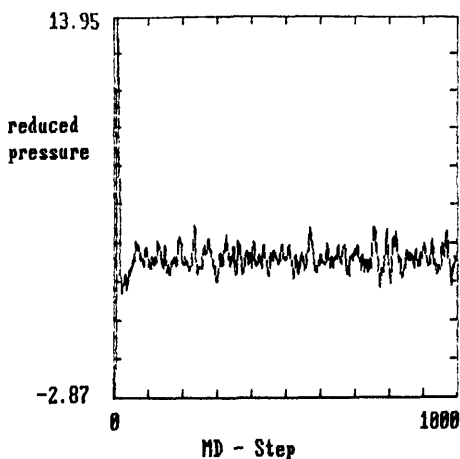


Fig.3.12. Evolution of the (reduced) pressure during the NPT molecular dynamics simulation

remains. What is the cut-off for the potential? Suppose we want the cut-off to be the same as for a box with side length $2L$, how must A be chosen? What is the volume? Devise a computational scheme for the truncated octahedron boundary condition.

- 3.2 Write a program for the spring problem using the **backward Euler algorithm** $u(t+h) = u(t) + hK(u(t+h), t+h)$.
- 3.3 Show that the modified equations (3.38) are mathematically equivalent to the Verlet algorithm.
- 3.4 There is still another variation of the Verlet algorithm with $O(h^2)$ error called the **leapfrog** formulation

$$\begin{aligned}
 v_i(t+\tfrac{1}{2}h) &= v_i(t-\tfrac{1}{2}h) + m^{-1}hF_i(t) , \\
 r_i(t+h) &= r_i(t) + v_i(t+\tfrac{1}{2}h)h , \\
 v_i(t-\tfrac{1}{2}h) &= h^{-1}[r_i(t) - r_i(t-h)] , \\
 v_i(t) &= \tfrac{1}{2}[v_i(t-\tfrac{1}{2}h) + v_i(t+\tfrac{1}{2}h)] .
 \end{aligned}$$

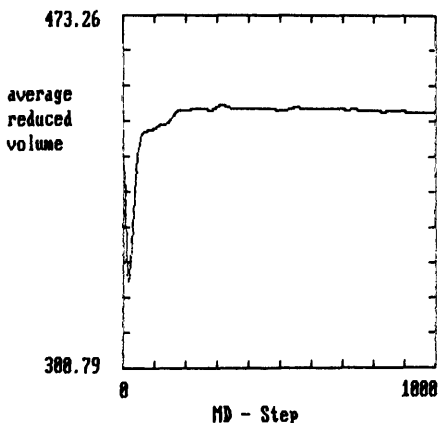
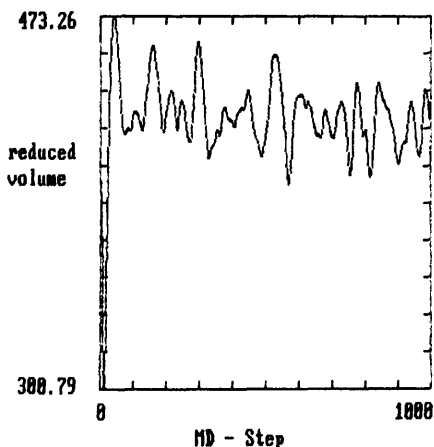


Fig.3.13. Reduced volume (*left*) and its subaverage (*right*) during the course of a simulation

Show the formal equivalence between the Verlet and the leapfrog formulation.

- 3.5 Given the values $\sigma = 0.3405$ nm, $\epsilon/k_B = 119.8$ K, $m = 6.63382 \cdot 10^{-26}$ kg for argon, a density $\rho^* = 0.83134$ and a cut-off $r_c = 2.5$, what is wrong with taking $N = 64$ particles?
- 3.6 Show that for a Lennard-Jones system the appropriate scalings of time and positions are $(m\sigma^2/48\epsilon)^{1/2}$ and σ . Further, show that the reduced temperature T^* is equal to $k_B T/\epsilon$. What do the reduced pressure and enthalpy look like?
- 3.7 The feasibility of the Verlet procedure to include only those particles in the calculations of the forces inside a ball of radius r_m relies on the relation

$$r_m - r_c < nvh,$$

where v is the average speed. Can you prove this?

- 3.8 Incorporate the nearest-neighbour table idea into the program for the Lennard-Jones system given in the appendix.
- 3.9 Start with a generalized force not derivable from a generalized potential and develop equations of motion yielding a constant kinetic energy.
- 3.10 **Block Distribution Function.** Even in an ensemble with a conserved number of particles it is possible to obtain quantities like the iso-thermal compressibility. The fluctation-dissipation theorem relates the density fluctuations to the iso-thermal compressibility. Suppose we cut up the system into cells. Each cell is of size b^d where $b = L/n$. During a simulation with a constant particle, constant volume and constant energy (or temperature) monitor the number of particles within the cells of size $b_1^d, b_2^d, \dots$ with $b_1 < b_2 \dots$. Compute the distribution $P(b, N)$. From the distribution one can obtain the compressibility as a function of the block size $\kappa(b)$. Extrapolate to the limit $\kappa(\infty)$ to get the thermodynamic limit. Perform such a simulation with a Lennard-Jones system. Can you determine the critical point of the system?
- 3.11 A problem of more a current research effort is, can you think of ways to parallelize the molecular dynamics simulation?
- 3.12 Can you think of another way to introduce periodic boundary conditions? (Hint: 4D, Quaternions).

4. Stochastic Methods

This chapter is concerned with methods which use stochastic elements to compute quantities of interest. These methods are not diametrically opposed to the deterministic ones. Brownian dynamics provides an example where the two methods are combined to form a hybrid technique. However, there are also inherently stochastic methods, such as the Monte-Carlo technique. An application of this simulation method was presented in the introductory chapter.

The stochastic methods are built on concepts developed in probability theory and statistical mechanics. They allow not only a treatment of problems apparently probabilistic in nature, like the random walk, but also problems which are on the face deterministic. The scope of applications is broad, making it a flexible and exciting tool in simulational physics.

One of the key elements in these types of simulations is the concept of the Markov process or Markov chain, which will be briefly introduced in the next section. For a more fundamental introduction the reader is referred to standard textbooks on probability theory [4.1] or introductory texts on the theory of Markov processes [4.2].

4.1 Preliminaries

Stochastic methods make use of the important concept of a *Markov process* or *Markov chain*, to be briefly reviewed in the following. In a sense, the Markov process is the probabilistic analogue to classical mechanics. The Markov process (chain) is characterized by a lack of memory, i.e., the statistical properties of the immediate future are uniquely determined by the present, regardless of the past.

An example may demonstrate what is meant by a Markov chain before we proceed more formally. Suppose that initially a particle is placed somewhere on a lattice, and this point serves as the origin. At each time step the particle hops to one of the nearest neighbours. For simplicity, assume that the lattice is two-dimensional. The essential feature is that at each time step the particle has the choice of hopping to *any* of the four nearest neighbours. The particle does not remember where it came from! In contrast, the particle may remember where it came from and avoid crossing its path. The former case is a *random walk*, while the latter is a *self-avoiding walk*. (For a detailed discussion of the random-walk problem, see [4.3,4] and references therein).

The random walk is a Markov chain. The outcome at each step is a state of the system, and we view the motion of the particle across the surface as a sequence of states. The transition from one state to another depends only on the preceding one, or the probability of the system being in state i depends only on the previous state $i-1$.

What we will be dealing with in the following sections are sequences of states, such as $\mathbf{x}_0, \dots, \mathbf{x}_n, \dots$, of a system analogous to those generated by the equations of motion; only here the states evolve probabilistically in time. Instead of treating time as a continuous variable, time is considered discrete and the above actually forms a chain. Each state $\mathbf{x}_i$ is the result of a trial, i.e., the random variable $\mathbf{X}_i$ has taken on the value $\mathbf{x}_i$, and it does so with an absolute probability a_i . Suppose the states $\mathbf{x}_0, \dots, \mathbf{x}_{n-1}$ are fixed at definite values. The probability that $\mathbf{x}_n$ occurs, given the fixed values, is called the *conditional probability* $P(\mathbf{x}_n | \mathbf{x}_{n-1}, \dots, \mathbf{x}_0)$.

Formally, a Markov chain is defined as follows.

Definition 4.1

The sequence $\mathbf{x}_0, \dots, \mathbf{x}_n, \dots$ is called a *Markov chain* if for any n we have

$$P(\mathbf{x}_n | \mathbf{x}_{n-1}, \dots, \mathbf{x}_0) = P(\mathbf{x}_n | \mathbf{x}_{n-1}) . \quad \square$$

The outcome of any trial depends on the preceding trial, and on it alone. By induction it is easy to show that the probability of the occurrence of a sequence $\mathbf{x}_0, \dots, \mathbf{x}_n$ factorizes as

$$\begin{aligned} P(\mathbf{x}_0, \dots, \mathbf{x}_n) &= P(\mathbf{x}_n | \mathbf{x}_{n-1}) \cdots P(\mathbf{x}_1 | \mathbf{x}_0) P(\mathbf{x}_0) \\ &= P(\mathbf{x}_n | \mathbf{x}_{n-1}) \cdots P(\mathbf{x}_1 | \mathbf{x}_0) a_0 . \end{aligned}$$

Due to this property, the conditional probabilities are called *one-step transition probabilities* or simply *transition probabilities*. To simplify the notation, let us abbreviate the transition probabilities as

$$P_{ij} = P(\mathbf{x}_j | \mathbf{x}_i) \quad (4.1)$$

or, in somewhat different form,

$$P_{ij} = P(\mathbf{x}_i \rightarrow \mathbf{x}_j) .$$

The property of a Markov chain of prime importance for applications in simulational physics is the existence of an invariant distribution of states. In the applications one usually starts out with an initial state $\mathbf{x}_0$ whose absolute probability a_0 is 1. Ultimately the states should be distributed according to a specified distribution. For example, fixing the temperature, one would like to generate states representing the thermal equilibrium

$$P(\mathbf{x}_i) \propto \exp[-\mathcal{H}(\mathbf{x}_i)/k_B T] .$$

To guarantee that, whatever the initial distribution, after a sufficiently long "time" (time is measured by the number of states generated) the distribution is approximately invariant, certain conditions must be placed on the transition probabilities. Before discussing these we give a formal definition of the invariant distribution.

Definition 4.2

A probability distribution (u_k) is called *invariant* or *stationary* for a given Markov chain if it satisfies

(i) for all k : $u_k \geq 0$;

(ii) $\sum_k u_k = 1$;

(iii) $u_j = \sum_i u_i p_{ij}$.

□

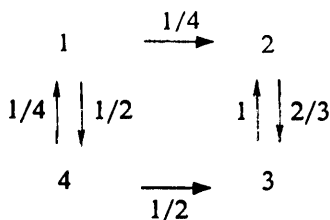
Considering the transition probabilities arranged as a matrix (*stochastic matrix*), the invariant distribution is a left eigenvector with eigenvalue 1.

If the states generated are states of equilibrium of the system under consideration we must have ergodicity. To give the notion of ergodicity a precise meaning in the context of Markov chains, some preparation is necessary. Let $p_{ij}^{(n)}$ denote the probability of a transition from the states x_i to x_j in exactly n steps, i.e., the n -step transition probability. A state x_j *can be reached* from x_i if there exists some $n > 0$ such that $p_{ij}^{(n)} > 0$. A chain is *irreducible* if, and only if, every other state can be reached from every state. This will clearly be required for the application. If the chain is reducible, the sequence of states will fall into classes with no transitions from one class into the other.

Consider a system with just four states with transition probabilities arranged into the stochastic matrix

$$\begin{bmatrix} 1/2 & 1/4 & 0 & 1/4 \\ 0 & 1/3 & 2/3 & 0 \\ 0 & 1 & 0 & 0 \\ 1/2 & 0 & 1/2 & 0 \end{bmatrix}$$

A transition from state number 1 into state number 4 occurs with a probability of 1/4 whereas a transition from state 4 into state 1 occurs with probability 1/2. The matrix is not irreducible! There is no path from state 2 to state 1 or 4. If the initial state is either 2 or 3 all subsequent states are either 2 or 3. The situation is more clearly displayed in a diagrammatic form where the probabilities of staying in a given state are omitted.



Once the system has reached the state 2 or 3 it is trapped.

A state x_i has a *period* $t > 1$ if $p_{ii}^{(n)} = 0$ unless $n = zt$ is a multiple of t , and t is the largest integer with this property. A state is *aperiodic* if no such $t > 1$ exists.

Let $f_{ij}^{(n)}$ denote the probability that in a process starting from x_i the first entry to x_j occurs at the n th step. Further, let

$$f_{ij}^{(0)} = 0 ,$$

$$f_{ij} = \sum_{n=1}^{\infty} f_{ij}^{(n)} ,$$

$$\mu_i = \sum_{n=1}^{\infty} n f_{ii}^{(n)} .$$

Then f_{ij} is the probability that starting from x_i the system will ever pass through x_j . In the case that $f_{ii} = 1$ the state x_i is called *persistent*, and μ_i is termed the *mean recurrence time*.

We are now able to formulate precisely what is meant by ergodic.

Definition 4.3

A state x_i is called *ergodic* if it is aperiodic and persistent with a finite mean recurrence time. A Markov chain with only ergodic elements is called ergodic. $\square$

Central to applications in simulational physics is the following theorem [4.1].

Theorem 4.4

An irreducible aperiodic chain possesses an invariant distribution if, and only if, it is ergodic. In this case $u_k > 0$ for all k and the absolute probabilities tend to u_k irrespective of the initial distribution. $\square$

Under the above conditions we are assured that eventually the states in the Markov chain are distributed according to some unique distribution; the initial distribution is irrelevant.

4.2 Brownian Dynamics

Methods to incorporate temperature in a simulation of a Hamiltonian system via stochastic forces are presented.

Before beginning a discussion of pure stochastic procedures we examine hybrid methods. Such algorithms involve a deterministic part consisting of the integration of equations of motion. The components added to the deterministic part are stochastic forces influencing the path of the system through phase space.

In the Molecular-Dynamics (MD) method, all degrees of freedom are explicitly taken into account, leading to classical equations of motion of a system of particles. A trajectory of the system through phase space is constructed by numerically integrating the equations of motion, starting with some initial conditions. Observables are then calculated along the trajectory. In *stochastic dynamics* computer-simulation methods, of which *Brownian dynamics* is a special case, some degrees of freedom are represented only through their stochastic influence on the others. Suppose a system of particles interacts with a viscous medium. Instead of specifying a detailed interaction of a particle with the particles of the viscous medium, we represent the medium as a stochastic force acting on the particle. The stochastic force reduces the dimensionality of the dynamics.

The Brownian-Dynamics (BD) simulation algorithms described in the following all concern the problem of generating paths in phase space yielding a constant temperature. The detailed interaction of the particles with the heat bath is neglected, and only taken into account by the stochastic force. We may define [4.5] the BD simulation method as follows.

Definition 4.5 Brownian Dynamics

The BD method computes phase-space trajectories of a collection of molecules that individually obey Langevin equations in a field of force. $\square$

As for the MD method, we can treat point particles and collections of particles with subunits. Here we focus on simple point particles, and leave algorithms for polymer simulations aside.

BD-simulation methods deal with systems described by stochastic differential equations. Part of the ground work has already been covered in the previous chapter, where algorithms were developed for problems of the form

$$\frac{du(t)}{dt} = K(u(t), t) ,$$

where K did not involve stochastic elements (types 1 and 2). Now we allow K to depend on a random function (type 3). Since we are not concerned

with the existence and the uniqueness of a solution, it is assumed that a unique solution exists.

We would like to couple a system of particles to a heat bath. The particles interact with each other via some deterministic force. Let us start with one free particle, for which the *Langevin equation* of motion is given by [4.6-8]

$$m \frac{dv}{dt} = R(t) - \beta v . \quad (4.2)$$

The right-hand side represents the coupling to the heat bath. The effect of the random force $R(t)$, on which we elaborate below, is to heat the particle. To balance overheating (on the average), the particle is subjected to friction. Formally, the solution to the Langevin equation can be written as

$$v(t) = v(0) \exp\left[-\frac{\beta}{m}t\right] + \frac{1}{m} \int_0^t \exp[-(t-\tau)\beta/m] R(\tau) d\tau . \quad (4.3)$$

The integral on the right-hand side is a *stochastic integral* and the solution $v(t)$ is a random variable. The stochastic properties of the solution depend significantly on the stochastic properties of the random force $R(t)$ and less on the initial conditions. To proceed further, we make use of the fact that integration and averaging commute [4.9]

$$\langle v(t) \rangle = \langle v(0) \rangle \exp(-\beta t/m) + \frac{1}{m} \int_0^t \exp[-(t-\tau)\beta/m] \langle R(\tau) \rangle d\tau . \quad (4.4)$$

Now we want the random force to disappear on average, i.e.,

$$\langle R(t) \rangle = 0 . \quad (4.5)$$

The average here is meant to be the average over the equilibrium ensemble. Furthermore, we require that at two different times $t = t'$ the random force is uncorrelated

$$\langle R(t)R(t') \rangle = q\delta(t-t') . \quad (4.6)$$

At this point we need to determine the strength of the noise q . This may be done by considering the *spectral densities*. Essentially, the spectral density of a random function $z(t)$ is the Fourier transform, which in turn is also a random variable. Assume that $z(t)$ is a stationary process over a time T , and define the spectral density as [4.8, 10]

$$G(f) = \lim_{T \rightarrow \infty} \frac{2}{T} |A(f)|^2 , \quad \text{where} \quad (4.7)$$

$$A(f) = \int_0^T z(t) \exp(-2\pi i f t) dt .$$

The spectral density $z(t)$ is related to the correlation function $\langle z(t)z(t+\tau) \rangle$ by the Wiener-Khintchine theorem. To connect up with the problem posed in (4.2) assume

$$\langle z(t) \rangle = 0 , \quad \langle z(t)z(t+\tau) \rangle = q\delta(\tau) .$$

Then one finds

$$G_z(f) = 2 \int_{-\infty}^{\infty} \exp(-2\pi i f \tau) q\delta(\tau) d\tau = 2q ,$$

i.e., a white spectrum. Consider the spectral density for the solution $v(t)$ to (4.3). Fourier transforming both sides of (4.3) and taking the modulus we find

$$G_v(f) = \frac{2q/m^2}{(\beta/m)^2 + (2\pi f)^2} . \quad (4.8)$$

Two consequences arise. The first is that the average square velocity is

$$\langle v^2 \rangle = \int_0^{\infty} G_v(f) df = \frac{q}{2\beta m} . \quad (4.9)$$

Since we want $\langle v^2 \rangle = k_B T/m$, it follows that

$$q = 2\beta k_B T . \quad (4.10)$$

The second consequence concerns the form of the *correlation function*

$$\rho(\tau) = \frac{\langle v(t)v(t+\tau) \rangle}{\langle v^2 \rangle} . \quad (4.11)$$

Using the Wiener-Khintchine theorem again it follows that

$$\rho(\tau) = e^{-\beta\tau/m} , \quad (4.12)$$

i.e., the correlation between the velocities decreases exponentially with the characteristic relaxation time m/β .

Before restating the problem, we have to specify the kind of distribution $R(t)$ should have. In order to have a Brownian motion it is important

[4.8] that $R(t)$ is Gaussian distributed. Then the problem boils down to finding the solution to the stochastic differential equation

$$\frac{dv}{dt} + \frac{\beta v}{m} = \frac{R(t)}{m}$$

with the supplementary conditions

$$\begin{aligned}\langle R(t) \rangle &= 0, \quad \langle R(t)R(0) \rangle = 2\beta k_B T \delta(t), \\ P(R) &= (2\pi \langle R^2 \rangle)^{-1/2} \exp(-R^2/2\langle R^2 \rangle).\end{aligned}\tag{4.13}$$

Since $R(t)$ is a Gaussian it follows that $v(t)$ is a Gaussian. Is $v(t)$ a Markov process [4.11]?

Theorem 4.6

A one-dimensional *Gaussian process* will be Markovian only when the correlation is

$\rho(r) = \exp(-ar)$, i.e., the spectrum must be of the form

$$(a^2 + b^2)^{-1}.$$

□

A similar theorem holds for n -dimensional Gaussian processes [4.12]. The stochastic collisions induce transitions such that the conditional probabilities factorize. Important for our application is that it can be shown [4.8] that $v(t)$ possesses an invariant distribution

$$P(v) = (2\pi m k_B T)^{-1/2} \exp(-v^2/2m k_B T),$$

i.e., a Maxwell distribution! The limiting distribution yields the correct distribution required for a constant-temperature algorithm.

Before developing a numerical algorithm, we need to consider the *correlation times* of the velocity and the random force, which are defined as

$$t_v = \frac{1}{\langle v^2 \rangle} \int_0^\infty \langle v(t)v(0) \rangle dt, \tag{4.14}$$

$$t_R = \frac{1}{\langle R^2 \rangle} \int_0^\infty \langle R(t)R(0) \rangle dt. \tag{4.15}$$

These correlation times obey the relation [4.5]

$$t_v t_R = \frac{\langle v^2 \rangle m^2}{\langle R^2 \rangle}. \tag{4.16}$$

In the particular case we are considering here, we have

$$t_v = m/\beta, \quad t_R = \beta k_B T \langle R^2 \rangle^{-1}. \quad (4.17)$$

It is reasonable to require that the correlation time for the random force is much smaller than the correlation time for the velocity

$$t_R \ll t_v. \quad (4.18)$$

Recall that a numerical algorithm to solve an equation of motion involves a finite step size in time h , i.e.,

$$x(t) \rightarrow x(t+h), \quad v(t) \rightarrow v(t+h).$$

Assume that during the time step h the particle experiences a constant random force and that the correlation time is h . Before each integration step $n \rightarrow n+1$ we choose a random force R_n from a Gaussian distribution with mean zero, and a variance $\langle R^2 \rangle$ according to (4.13). All the supplementary requirements are fulfilled. We still have to determine $\langle R^2 \rangle$, which can be done using (4.17) with the correlation time $t_R = h$

$$\langle R^2 \rangle = \beta k_B T / h. \quad (4.19)$$

Algorithm A7. Brownian Dynamics

- 1) Assign an initial position and velocity.
- 2) Draw a random number from a Gaussian distribution with mean zero and variance as in (4.13).
- 3) Integrate the velocity to obtain v^{n+1} .
- 4) Add the random component to the velocity.

The above algorithm can be generalized to a collection of particles including systematic forces [4.13-22]. Here we want to examine another method [4.23] of imposing a constant temperature on the system.

Another approach to taking into account the coupling of the system to a heat bath is to subject the particles to collisions with virtual particles [4.23]. Such collisions are imagined to affect only the momenta of the particles, hence they affect the kinetic energy and introduce fluctuations in the total energy. This idea is incorporated into the Hamiltonian equations of motion by adding a stochastic force to the momentum equation

$$\frac{dr_i}{dt} = \frac{\partial \mathcal{H}}{\partial p_i} = p_i, \quad \frac{dp_i}{dt} = \frac{-\partial \mathcal{H}}{\partial r_i} = F_i + R_i(t). \quad (4.20)$$

Each stochastic collision is assumed to be an instantaneous event affecting only one particle, and it is further assumed that the process is Poissonian. The time intervals at which a particle suffers a collision are distributed according to

$$P(t) = \nu e^{-\nu t}, \quad (4.21)$$

where ν is the mean rate of collisions. Notice that in this approach no friction force appears.

Algorithm A8. Brownian Dynamics

- 1) Choose time intervals according to (4.21).
- 2) Integrate the Hamilton equations of motion until the time for a stochastic collision.
- 3) If the particle to suffer a collision is i , choose a momentum at random from a Boltzmann distribution at temperature T .
- 4) Proceed with step 2.

It can be shown [4.23] that under certain conditions the above algorithm generates a Markov chain, and that the time average of any quantity A calculated along a generated trajectory is equal to the canonical ensemble average

$$\bar{A} = \langle A \rangle_{NVT}. \quad (4.22)$$

Example 4.1

The argon system, by now very familiar, will once again serve as an example. In the particular case considered here the parameters for the simulation were $T^* = 0.722$, $\rho^* = 0.83134$ with $N = 256$ particles. The cut-off of the potential was $r_c = 2.5$ with no smooth continuation to zero. The simulation had a duration of 1000 molecular dynamics steps. Every 20th step, new velocities were drawn from a Boltzmann distribution with the temperature $T^* = 0.722$. The method of generating such a distribution was the log-method described in Appendix A1.

The relaxations of the kinetic, potential and total energies are shown in Fig. 4.1. As was the case with the usual molecular dynamics (Example 3.2), there is a quickly achieved equilibration of the kinetic energy. During the observation period the potential energy has not equilibrated. correspondingly, the total internal energy has not equilibrated. $\square$

In the above approach to obtaining a constant temperature, the trajectories are not smooth. Each time the velocities are replaced, a discontinuity is introduced in the trajectory. Furthermore, the question arises as to how the dynamic properties are affected.

It must be pointed out that the requirement of a Gaussian distributed random force is not necessary. It depends on the notion of convergence one is applying [4.24, 25]. For all practical purposes one can use uniformly distributed random numbers.

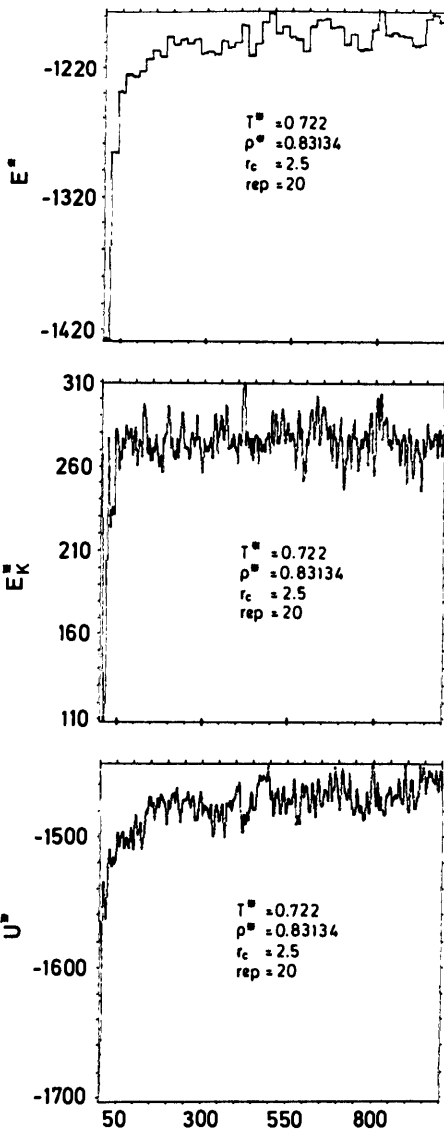


Fig.4.1. Shown are the energies obtained with the Brownian dynamics where the velocities were replaced by velocities drawn from a Boltzmann distribution. The total energy, kinetic and potential energy are given in reduced form as a function of the MD steps

4.3 Monte-Carlo Method

In this section we outline the principle behind the Monte-Carlo method as it is applied in statistical mechanics. In the subsections specific methods for different thermodynamic ensembles will be presented.

We met the *Monte-Carlo method* in the first example of a computer simulation method in this text. Our problem was to estimate the percolation threshold p_c . We did so by generating configurations by a random process. Each lattice was filled according to the outcome of a trial. We then checked for the particular realization whether a percolating cluster occurred. As a result we obtained curves $P_\infty(p, L)$, i.e., the probability for percolation as a function of the concentration of filled sites and the system size. The essential point of that approach is that the percolation threshold appears as the result of an averaging over many configurations. In other words, we sampled the space of all possible configurations and obtained the percolation probability as an expectation value. The computation of expectation values is the very heart of the Monte-Carlo method.

Before proceeding, we give a general definition of the Monte-Carlo method, applicable not only in the context of simulational physics but also in the context of numerical mathematics [4.26]

Definition 4.7. The Monte-Carlo Method

The Monte-Carlo (MC) method is defined by representing the solution of a problem as a parameter of a hypothetical population, and using a random sequence of numbers to construct a sample of the population, from which statistical estimates of the parameter can be obtained. $\square$

The definition shows that the scope of applications is enormous and fascinating. Many problems that at first glance do not seem to allow treatment with the MC method, can be transformed into stochastic ones. Here we concentrate on the application of the MC method to statistical-mechanics problems. For other applications we refer the reader to [4.27, 28] and references contained therein.

Up to now the approach to computing properties of physical systems has been to use the fundamental equations of motion to generate paths in phase space. The quantity of interest has then been evaluated along this path. That the quantity thus obtained is indeed equal to the ensemble average is ensured by the fundamental relation (2.3), i.e., that the trajectory averages are equal to ensemble averages. The MC method in simulational physics takes a different approach. It starts out with a description of the system in terms of a Hamiltonian, and an appropriate ensemble for the problem is selected. Then all observables are computable using the associated distribution function and the partition function. The idea is to *sample* the main contributions to get an estimate for the observable. The MC technique in statistical physics is centred around the computation of mathematical expectations.

Ultimately the goal is to compute quantities appearing as results of high-dimensional integrations. However, valuable insight is provided by first looking at the simpler problem of one-dimensional integration. The problem that we would like to solve is stated as follows: Given a function $f(x)$ ($a \leq x \leq b$), compute the integral

$$I = \int_a^b f(x) dx . \quad (4.23)$$

We can reformulate the ansatz in terms of an average, and thereby make the crucial step from a deterministic to a stochastic problem. We have according to the mean-value theorem of calculus

$$\langle f(x) \rangle = \frac{I}{b-a} . \quad (4.24)$$

The integral can be computed by choosing n points x_i randomly from the interval $[a, b]$ with a uniform distribution and forming the sample average of the heights of $f(x)$

$$I = \int_a^b f(x) dx \simeq \frac{b-a}{n} \sum_i^n f(x_i) . \quad (4.25)$$

Such an approach is an example of *straightforward sampling*. The question that arises immediately is: Does the method converge? The question should not be confused with the statement (4.24). This equation expresses a statement of convergence in the context of calculus, which is different from the convergence in the statistical context. Convergence, in the sense used in calculus, means that given an arbitrarily small number ϵ one can find an element such that the element is within the ϵ neighbourhood of the value of the integral. Convergence in the statistical sense cannot guarantee this, but only that a number can be found that is with a certain probability within the ϵ neighbourhood.

Theorem 4.8. Law of Large Numbers

Let $x_1, \dots, x_n$ be random variables selected according to a probability density function $\mu(x)$ with

$$\int_{-\infty}^{\infty} \mu(x) dx = 1 .$$

Assume that $I = \int_{-\infty}^{+\infty} f(x) \mu(x) dx$ exists. Then for every $\epsilon > 0$

$$\lim_{n \rightarrow \infty} P \left\{ I - \epsilon \leq \frac{1}{n} \sum_i^n f(x_i) \leq I + \epsilon \right\} = 1 . \quad \square$$

The theorem guarantees the convergence of the method. However, one does not want to generate a large number of samples of finite length; one wants to generate a single sample.

Theorem 4.9. Strong Law of Large Numbers

$$P\left\{\lim_{n \rightarrow \infty} \frac{1}{n} \sum_{i=1}^n f(x_i) = I\right\} = 1. \quad \square$$

The theorem states that for a sufficiently large sample one can come arbitrarily close to the desired value of the integral. The second question that must now be addressed is the estimation of the error involved if the sample is of length n . In passing, we note that the above is also true if random variables are correlated, as is the case for a Markov chain [4.29].

Theorem 4.10. Central Limit Theorem

$$\text{Let } \sigma^2 = E_{\mu}[(f(x)-I)^2] = \int_{-\infty}^{\infty} f^2(x)\mu(x)dx - I^2$$

be the variance of $f(x)$. Then

$$P\left\{\left|\frac{1}{n} \sum_{i=1}^n f(x_i) - I\right| \leq \frac{\lambda\sigma}{\sqrt{n}}\right\} = \sqrt{2\pi} \int_{-\lambda}^{+\lambda} \exp(-x^2/2)dx + O(1/\sqrt{n}). \quad \square$$

For a given confidence interval the error bound is proportional to σ and inversely proportional to the square root of n . The convergence, in the sense described above, is very slow. An increase of the sample size n by a factor of 100 results in a decrease of the uncertainty by a factor of only 10! For practical purposes the convergence of the straightforward sampling is too slow. However, there is another parameter available to reduce the uncertainty.

In the straightforward sampling, all points at which the function is evaluated are chosen uniformly. No reference is made to the nature of the function. If the function has a large variation, the uncertainty from the Monte-Carlo estimate will be large. Conversely, if the function is uniform then the estimate will be most accurate. Suppose the function $f(x)$ is peaked around the mean value. Little contribution to the average comes from the tails, so it would be more efficient to sample the function at points where the main contribution comes from. Suppose further that one can construct a function $p(x) > 0$ such that it mimics the behaviour of $f(x)$ but can be computed analytically

$$\int_a^b p(x) dx = 1, \quad p(x) > 0 \quad (4.26)$$

after proper normalization. It follows that

$$I = \int_a^b f(x) dx = \int_a^b \frac{f(x)}{p(x)} p(x) dx. \quad (4.27)$$

We may choose the points x_i according to the measure $p(x)dx$, instead of uniformly, and weight the function evaluated at x_i properly. The average of the function $f(x)$ is then

$$\langle f(x) \rangle \simeq \frac{1}{n} \sum_i^n \frac{f(x_i)}{p(x_i)}. \quad (4.28)$$

Computing the variance one obtains

$$\sigma^2 = \int_a^b \left(\frac{f(x)}{p(x)} \right)^2 p(x) dx - \left[\int_a^b \frac{f(x)}{p(x)} p(x) dx \right]^2. \quad (4.29)$$

Without loss of generality one can assume that $f(x) > 0$. The actual form of $p(x)$ determining the distribution of evaluation points is still at our disposal. To make the variance as small as possible, choose

$$p(x) \simeq f(x) / \int_a^b f(x) dx, \quad (4.30)$$

then the variance is practically zero. At this point we have arrived at the idea of *importance sampling*. Basically we have chosen a measure preferring points which give the dominant contributions to the integral. Points which lie in the tails occur less frequently. With importance sampling we have succeeded in reducing the statistical uncertainty without increasing the sample size. The problem now is that the function $p(x)$ requires prior knowledge of the integral. For the moment we leave this problem aside, and pause to state the kind of problem we would like to solve using the Monte-Carlo method.

The general problem in statistical mechanics which we want to address is as follows:

Let N be the number of particles. Associated with each particle i is a set of dynamical variables, s_i , representing the degrees of freedom. The set

$((s_1), \dots, (s_N))$ describes the phase space. Let $\mathbf{x}$ denote a point in the phase space Ω . The system is assumed to be governed by a Hamiltonian $\mathcal{H}(\mathbf{x})$ where the kinetic-energy term has been dropped. This can be done because the contribution of the kinetic-energy term allows an analytic treatment.

We want to compute the observable A of the system. Let $f(\cdot)$ be an appropriate ensemble; for example, $f(\cdot)$ might be the distribution function of the canonical ensemble, then A is computed as

$$\langle A \rangle = Z^{-1} \int_{\Omega} A(\mathbf{x}) f(\mathcal{H}(\mathbf{x})) d\mathbf{x} , \quad (4.31)$$

where

$$Z = \int_{\Omega} f(\mathcal{H}(\mathbf{x})) d\mathbf{x}$$

is the partition function. Note that in the description of the system we have dropped the kinetic-energy term. The Monte-Carlo method gives information on the configurational properties, in contrast to the molecular-dynamics method which yields the true dynamics. The molecular-dynamics method gives information about the time dependence and the magnitude of position and momentum variables. By choosing an appropriate ensemble, like the canonical ensemble, the MC method can evaluate observables at a fixed particle number, volume and temperature. The great advantage of the MC method is that many ensembles can be quite easily realized, and it is also possible to change ensembles during a simulation!

To compute the quantity A we are faced with the problem of carrying out the high-dimensional integral of (4.31). Only for a limited number of problems is one able to perform the integration analytically. For some types of problems the integration can be carried out using approximate schemes, like the steepest-descent method. With the Monte-Carlo method, to evaluate the integral, we do not need any approximation except that we consider the phase space to be discrete. Consequently, we frequently switch from integrals to sums.

To solve the problem we draw on the ideas developed earlier for the one-dimensional integration. Suppose the appropriate ensemble is the canonical one

$$f(\mathcal{H}(\mathbf{x})) \propto \exp[-\mathcal{H}(\mathbf{x})/k_B T] .$$

All states $\mathbf{x}$ corresponding to a large energy give small contributions to the integral. Only certain states give large contributions. We therefore expect the distribution to be sharply peaked around the average value of $\mathcal{H}(\mathbf{x})$. Now suppose we calculate the integral (4.31) by randomly selecting states $\mathbf{x}$ and summing up the contributions, completely analogously to (4.25). The more states we generate, the more accurate the estimate will become. However, since the phase space is high-dimensional we would need a tremen-

dous number of states, for most of which the contribution to the sum is negligible. To reduce the problem to a manageable level we make use of the idea of importance sampling. As was the case for one-dimensional integration, we do not take the phase points completely at random. Rather, we select them with a probability $P(\mathbf{x})$.

Let us develop the ideas more formally. The first step is to select states from the phase space at random. This gives an approximation to the integral (assuming n states were generated)

$$\langle A \rangle \simeq \sum_i^n A(\mathbf{x}_i) f(\mathcal{H}(\mathbf{x}_i)) / \sum_i^n f(\mathcal{H}(\mathbf{x}_i)) . \quad (4.32)$$

If we are to choose the states with probability $P(\mathbf{x})$, then (4.32) becomes

$$\langle A \rangle \simeq \frac{\sum_i^n A(\mathbf{x}_i) P^{-1}(\mathbf{x}_i) f(\mathcal{H}(\mathbf{x}_i))}{\sum_i^n P^{-1}(\mathbf{x}_i) f(\mathcal{H}(\mathbf{x}_i))} . \quad (4.33)$$

Choosing

$$P(\mathbf{x}) = Z^{-1} f(\mathcal{H}(\mathbf{x})) , \quad (4.34)$$

i.e., the probability is equal to the equilibrium distribution, the variance, which we wanted to reduce, is practically zero. At this point we are faced with the same difficulty as where we left the one-dimensional case. Let us accept the choice of (4.34). The computation of the quantity A then reduces to simple arithmetic averaging

$$\langle A \rangle \simeq \frac{1}{n} \sum_i^n A(\mathbf{x}_i) . \quad (4.35)$$

Using the idea of importance sampling we have considerably reduced the task of solving the statistical-mechanics problem (4.31) numerically. Note that no approximation was involved in arriving at (4.35). That the method converges is guaranteed by the central limit theorem.

To proceed further, an algorithm has to be devised that generates states distributed according to

$$P(\mathbf{x}) = Z^{-1} f(\mathcal{H}(\mathbf{x})) .$$

The specific choice of the variance reduction function means that one samples the thermodynamic equilibrium states of the system. However, their distribution is not a priori known. To circumvent the problem *Metropolis* et al. [4.30] put forward the idea of using a Markov chain such that starting from an initial state $\mathbf{x}_0$ further states are generated which are ultimately distributed according to $P(\mathbf{x})$. Subsequent states generated by the Markov chain are such that the successor lies close to the preceding one. It follows that there is a well-defined correlation between subsequent states. The Markov chain is the probabilistic analogue to the trajectory generated by the equations of motion in molecular dynamics.

What one has to specify are *transition probabilities* from one state $\mathbf{x}$ of the system to a state $\mathbf{x}'$ per unit time. In Monte-Carlo applications the transition probability is customarily denoted by $W(\mathbf{x}, \mathbf{x}')$. To ensure that the states are ultimately distributed according to $P(\mathbf{x})$, i.e., that they are thermodynamic equilibrium states, restrictions must be placed on $W(\mathbf{x}, \mathbf{x}')$ (Theorem 4.4).

Restrictions 4.11

- (i) For all complementary pairs $(S, \bar{S})$ of sets of phase points there exist $\mathbf{x} \in S$ and $\mathbf{x}' \in \bar{S}$ such that $W(\mathbf{x}, \mathbf{x}') \neq 0$.
- (ii) For all $\mathbf{x}, \mathbf{x}'$: $W(\mathbf{x}, \mathbf{x}') \geq 0$.

(iii) For all $\mathbf{x}$:
$$\sum_{\mathbf{x}'} W(\mathbf{x}, \mathbf{x}') = 1.$$

(iv) For all $\mathbf{x}$:
$$\sum_{\mathbf{x}'} W(\mathbf{x}', \mathbf{x}) P(\mathbf{x}') = P(\mathbf{x}).$$

□

Let us examine the restrictions. The first is the statement of connectivity or ergodicity, and the second that of positivity. The term ergodicity should not be confused with its meaning in physics, though they are closely connected. We will have more to say about this point later. The third restriction is conservation, i.e., the total probability that the system will come to some state $\mathbf{x}$ is unity. The fourth restriction says that the limiting distribution is the equilibrium distribution according to which we want the states to be distributed.

Assume that the transition probabilities were specified, and the states $\mathbf{x}_0, \mathbf{x}_1, \dots$ generated. The evolution of the probability $P(\mathbf{x}_i)$ with which the states are distributed may be described by the *Master equation* [4.31-33]

$$\frac{dP(\mathbf{x}, t)}{dt} = - \sum_{\mathbf{x}'} W(\mathbf{x}, \mathbf{x}') P(\mathbf{x}, t) + \sum_{\mathbf{x}'} W(\mathbf{x}', \mathbf{x}) P(\mathbf{x}', t). \quad (4.36)$$

Here, time is considered continuous, and we have used the abbreviation $P(\mathbf{x}_i) = P(\mathbf{x}, t)$. In addition, we changed from the finite difference

$\Delta P(\mathbf{x}_i)/\Delta t$ to the differential $dP(\mathbf{x}, t)/dt$. In the thermodynamic limit this becomes exact. Equation (4.36) is a rate equation. The first term describes the rate of all transitions out of the considered state, whereas the second term describes the rate of transitions into the considered state. Let us calculate the stationary solution to the master equation

$$\sum_{\mathbf{x}'} W(\mathbf{x}, \mathbf{x}') P(\mathbf{x}) = \sum_{\mathbf{x}'} W(\mathbf{x}', \mathbf{x}) P(\mathbf{x}') . \quad (4.37)$$

Invoking conservation (iii), it follows that

$$\sum_{\mathbf{x}'} W(\mathbf{x}', \mathbf{x}) P(\mathbf{x}) = P(\mathbf{x}) . \quad (4.38)$$

From (4.37) it is clear that a restriction stronger than (iv) can be imposed by requiring a *detailed balance* or *microscopic reversibility*

$$W(\mathbf{x}, \mathbf{x}') P(\mathbf{x}) = W(\mathbf{x}', \mathbf{x}) P(\mathbf{x}') . \quad (4.39)$$

Detailed balance is a sufficient but not necessary condition for the Markov chain to converge ultimately to the desired distribution [4.33–35]. There is considerable freedom since the transition probabilities are not uniquely specified.

We now exhibit one possible choice of transition probabilities [4.27, 30]. Let $\omega_{\mathbf{x}\mathbf{x}'}$ represent real positive numbers such that

$$\sum_{\mathbf{x}'} \omega_{\mathbf{x}\mathbf{x}'} = 1 \quad \text{and} \quad \omega_{\mathbf{x}\mathbf{x}'} = \omega_{\mathbf{x}'\mathbf{x}} . \quad (4.40)$$

The detailed balance condition involves only ratios, which suggests that one should define the transition probabilities using ratios

$$W(\mathbf{x}, \mathbf{x}') = \begin{cases} \omega_{\mathbf{x}, \mathbf{x}'} P(\mathbf{x}') / P(\mathbf{x}) & \text{if } \mathbf{x} \neq \mathbf{x}', P(\mathbf{x}') / P(\mathbf{x}) < 1 \\ \omega_{\mathbf{x}, \mathbf{x}'} & \text{if } \mathbf{x} \neq \mathbf{x}', P(\mathbf{x}') / P(\mathbf{x}) \geq 1 \\ \omega_{\mathbf{x}, \mathbf{x}'} + \sum_{\mathbf{x}} \omega_{\mathbf{x}, \mathbf{x}'} \{ [1 - P(\mathbf{x}')] / P(\mathbf{x}) \} & \text{if } \mathbf{x} = \mathbf{x}' . \end{cases} \quad (4.41)$$

It is easy to show (Problem 4.3) that this choice satisfies the restrictions (ii)–(iv) which must be placed on the transition probabilities. The actual selection for the real positive numbers has yet to be made and leaves freedom as to the convergence toward the equilibrium distribution.

That the transition probabilities depend only on the ratios of probabilities has an important consequence which is sometimes overlooked. Ultim-

ately the distribution of states must correspond to the equilibrium distribution

$$Z^{-1} f(\mathcal{H}(\mathbf{x})) .$$

Because $P(\mathbf{x}) = Z^{-1} f(\mathcal{H}(\mathbf{x}))$ it follows that the proportionality constant Z , which is the partition function, does not enter the transition probabilities. Indeed, this is what makes the approach feasible. But the price to be paid is that the partition function itself is not directly accessible in a simulation, which in turn leads to the consequence that, for example, the free energy

$$F = -k_B T \ln Z$$

or the entropy

$$S = (U-F)/T$$

cannot be computed directly. We return to this problem when discussing the canonical ensemble Monte-Carlo technique.

Algorithm A9. Monte-Carlo Method

- 1) Specify an initial point $\mathbf{x}_0$ in phase space.
- 2) Generate a new state $\mathbf{x}'$.
- 3) Compute the transition probability $W(\mathbf{x}, \mathbf{x}')$.
- 4) Generate a uniform random number $R \in [0, 1]$.
- 5) If the transition probability W is less than the random number R , then count the old state as a new state, and return to step 2.
- 6) Otherwise, accept the new state and return to step 2.

In the above algorithm, Step 1 is analogous to the initialization in the molecular and Brownian dynamics methods. The Markov chain loses its memory of the initial state so that the precise initial state is, to a large extent, irrelevant. However, care has to be exercised since the initial state might be in an irrelevant part of the phase space. In the second step, one chooses a new state or configuration randomly. For example, in a Monte Carlo simulation of a Lennard-Jones system one would select an atom at random and displace it randomly to another position within a certain radius. In Steps 3 to 5 a decision is made about whether to accept or reject the Monte-Carlo move. Why do we need the random number $R \in [0, 1]$? Because the probability $P(R < W(\mathbf{x}, \mathbf{x}'))$ is equal to $W(\mathbf{x}, \mathbf{x}')$, in agreement with (4.41). The above algorithm is the basis of any Monte-Carlo simulation, regardless of the ensemble.

What kind of averages are performed by a Monte Carlo simulation? At first glance one would say that the MC method performs an ensemble average. However, it has been recognized [4.36, 37] that actually a time average is performed in the course of a MC simulation, similar to the one in molecular dynamics; only here the trajectory runs through configurational space,

whereas the trajectory in molecular dynamics runs through the position-momentum space.

So far it has been tacitly assumed that the choice of transition probabilities satisfies the ergodicity restriction. The term ergodicity within the context of Markov chains refers to the statement that any state is accessible from any other state. More strongly expressed, any state must be accessible from any other state in a *finite* number of transitions. If this is not the case the states are split into ergodicity classes and there is no transition possible between the classes. This happens in a system consisting of hard spheres at high density [4.35, 38]. There may be no transition from a hexagonal close-packed state to states near the face-centred-cubic close-packing states. Even for Hamiltonians which are bounded, an effectively broken symmetry may occur, for example, in a system undergoing phase transitions [4.39, 40].

Often the ergodicity of the Markov chain is connected to the size [4.41], shape [4.42] and observation time [4.43] of the system under study. This practical non-ergodicity may not be related to true non-ergodicity. Clearly the shape and the boundary conditions influence the possible configurations. Certain lattice structures cannot be accommodated by a cubical box with periodic boundary conditions.

In Chap.3 we mentioned the dynamic interpretation of the Monte-Carlo process. The first step towards associating dynamics with the process was made by writing down the evolution of the probability distribution of the states as a master equation. We have

$$\langle A(t) \rangle = \sum_{\mathbf{x}} A(\mathbf{x}) P(\mathbf{x}, t) \quad (4.42)$$

as the definition of the time-dependent average of the observable A. The dynamic evolution of the observable A follows from this definition. It can be shown [4.37] that

$$\langle A(t) \rangle = \sum_{\mathbf{x}} A(\mathbf{x}(t)) P(\mathbf{x}, t_0), \quad (4.43)$$

i.e., one may average over the initial state $P(\mathbf{x}, t_0)$ while the state $\mathbf{x}(t)$, and hence the observable, develops in time. Although this leads to a dynamic evolution of the observable, the dynamics does not correspond to a dynamics in the sense of the one generated by the Newtonian equations of motion. This is seen by considering a Hamiltonian of a model that has no intrinsic dynamics whatsoever.

Though the MC method is exact in the sense described above, there are several practical limitations, imposed by the limited capacity of the computer. We want to mention them briefly at this point and elude to them later when we study specific examples. One of the limitations is, of course, the finite size of the system. Usually fairly large systems can be simulated (for example, 600^3 [4.44]). The number of "particles" is much larger than in

molecular or Brownian dynamics simulations, but still far away from the thermodynamic limit. To gain insight into the dependence of the result on the system size a finite-size analysis has to be made, which allows the extrapolation to $N \rightarrow \infty$. For some types of problems, e.g. the study of systems undergoing phase transitions, this can be done conveniently by a *finite-size scaling analysis* [4.45]. For other problems, ad hoc checks are necessary. Such an ad hoc check may be to simulate a very different system size and compare the results with the one with which the original simulations were carried out.

Suppose the Markov chain is started with a state x_0 . Two questions arise.

- 1) How many of the initial states n_0 must be discarded?
- 2) How long must the chain be in order to reach some given accuracy $|\langle A \rangle - A| < \epsilon$?

These questions have been posed analogously already within the context of molecular dynamics and were discussed in Chap.2. The initial state, in general, does not correspond to an equilibrium state. The system must be allowed to relax before meaningful averages can be taken.

Let us disregard the possibility that the system becomes trapped in a metastable state. Theorem 4.4 guarantees that ultimately the generated states will be arranged with a distribution corresponding to the equilibrium. However, a general statement about the convergence to equilibrium cannot be made [4.1]. Consider a system undergoing a phase transition. Far away from the transition point the system relaxes quite fast into equilibrium. Close to the transition point the relaxation slows down [4.46]. A model for such a system would reflect the same behaviour (see, for example, studies on the phenomenon of "critical slowing down" for Ising-systems modelling liquid-gas transitions [4.47, 48]). It follows that one has to identify the *relevant* time scales of the model to estimate the number of initial states to be discarded. This can be accomplished in some cases by considering the relaxation times of the associated master equation [4.33, 36, 49, 50]. Of course, the relaxation depends considerably on the conservation laws, i.e., on the ensemble in which the quantities are calculated. Since each problem has to be considered separately we shall not discuss the matter further and direct the reader to the literature [4.51, 52] where some specific models have been studied.

If a discussion of the relaxation times of the master equation is not possible, one can also heuristically determine the necessary portion of the chain to be discarded. One simply observes how certain quantities relax into equilibrium. Once one is reasonably sure that equilibrium has been reached, one can start averaging.

The second limitation is the finiteness of the Markov chain. In principle, the observation time must be infinite, which, of course, is not realizable. It is then necessary to check for effects of a finite observation time. This requires runs of different lengths to see whether the asymptotic regime of validity of the central-limit theorem has been reached. To esti-

mate the error we could use simple statistical analysis. However, the subsequent states generated by the Monte-Carlo process are correlated. A simple determination of the error using the standard-deviation method is not possible. We must allow for the statistical inefficiency.

4.3.1 Microcanonical Ensemble Monte-Carlo Method

This section introduces one possible algorithm to perform simulations at constant energy. The so-called Creutz method is explained and as an example the two-dimensional Ising model is treated using this algorithm.

The kind of systems we would like to study with the microcanonical Monte-Carlo technique are those described by a Hamiltonian $\mathcal{H}$. In the microcanonical molecular-dynamics method the system has state variables $(\mathbf{x}, \mathbf{p})$ representing the generalized coordinates $\mathbf{x}$ and the corresponding conjugate momenta $\mathbf{p}$. To perpetuate the system in phase space the equations of motion are set up and solved numerically. For the microcanonical Monte-Carlo simulation we drop the kinetic energy term from the Hamiltonian. To compute properties of the system we thus cannot use the equations of motion. The approach taken is to evaluate the properties using the partition function Z . The dynamics will not reflect the true intrinsic system dynamics but the dynamics generated by a Markov chain. The configurational properties are, however, the same as those obtained by the MD method.

For a conservative system as considered here with a fixed number of particles N in a given volume V , the microcanonical ensemble distribution is expressed by a delta function, so that the partition function is

$$Z = \int_{\Omega} \delta(\mathcal{H}(\mathbf{x}) - E) d\mathbf{x} , \quad (4.44)$$

where E is the fixed energy of the system. The only configurations counted are those where the Hamiltonian is constrained to E . Using the partition function, quantities are computed as follows. With any observable A is associated a function $A(\mathbf{x})$ which depends on the state of the system. The usual assumption is that the observable A is equal to the ensemble average

$$\langle A \rangle_{NVE} = \frac{1}{Z} \int_{\Omega} A(\mathbf{x}) \delta(\mathcal{H}(\mathbf{x}) - E) d\mathbf{x} . \quad (4.45)$$

In the microcanonical ensemble, all states have a priori equal weight, as expressed by the delta function in (4.45). The general idea of the Monte-Carlo method for computing the integral on the right-hand side is to sample the available phase space of the system and carry out a summation. Similar-

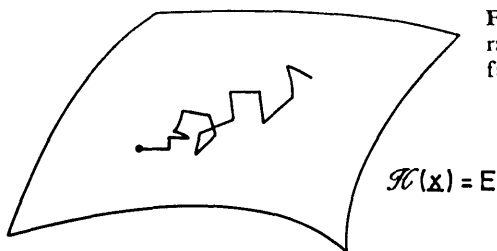


Fig.4.2. Schematic representation of a random walk on a constant energy surface in phase space

ly to the microcanonical MD technique, an algorithm must be constructed such that the system travels the constant energy surface in an ergodic manner. In the microcanonical MC method the system moves on the surface guided by a random walk (Fig.4.2) since all states have a priori equal weight. If the random walk is simple and not, for example, a self-avoiding random walk, where each state depends on the history, then a Markov chain is defined.

Suppose that somehow a state $\mathbf{x}$ is generated such that $\mathcal{H}(\mathbf{x}) = E$. Once on the surface a sampling algorithm has to produce further states on the surface. Assume that we relax the surface restriction a little and allow for ϵ variations in the region $E - \epsilon < \mathcal{H}(\mathbf{x}) < E + \epsilon$ away from the surface. We may do so by introducing an extra degree of freedom [4.53-55], called a *demon*, with energy E_D , into the partition function

$$Z = \sum_{\mathbf{x}} \sum_{E_D} \delta(\mathcal{H}(\mathbf{x}) + E_D - E). \quad (4.46)$$

The demon plays a role similar to the kinetic energy term in molecular dynamics. It produces changes in the configuration by travelling around the system and transferring energy. Thereby the demon creates a random walk of the system on the surface. We must, however, restrict the demon's energy, otherwise it will absorb all the energy! Such a restriction may, for example, limit the demon's energy to positive values. Algorithmically the outlined procedure looks as follows.

Algorithm A10. NVE Monte-Carlo

- 1) Construct a state such that $\mathcal{H}(\mathbf{x}) = E$.
- 2) Set the demon energy E_D (for example, $E_D = 0$).
- 3) Choose a part of the system.
- 4) Change the local state of the system so that $\mathbf{x} \rightarrow \mathbf{x}'$.
- 5) Calculate the energy change produced, i.e., $\Delta\mathcal{H} = \mathcal{H}(\mathbf{x}') - \mathcal{H}(\mathbf{x})$.
- 6) If the energy is lowered, accept the change, set $E_D \leftarrow E_D - \Delta\mathcal{H}$ and count $\mathbf{x}'$ as a new configuration. Return to step 3.
- 7) Otherwise, accept the change only if the demon carries enough energy, i.e., $\Delta\mathcal{H} - E_D > 0$. In this case $E_D \leftarrow E_D - \Delta\mathcal{H}$ and count $\mathbf{x}'$ as a new configuration.
- 8) Return to Step 3. □

The algorithm guarantees with Steps 6 and 7 that the system relaxes to thermal equilibrium. In addition, Step 7 also ensures the positivity of the domain's energy.

Conceptually we may view the demon as a thermometer. Indeed, the demon can take up or lose energy as it is successively brought in contact with parts of the system. Initially, the demon has an arbitrary distribution. The system acts as a reservoir and thermalizes the demon. Ultimately the energies become Boltzmann distributed [4.56], allowing the calculation of the temperature

$$P(E_D) \propto \exp(-E_D/k_B T) . \quad (4.47)$$

The Ising model serves as an example of the use of a Monte-Carlo simulation.

Example 4.2

The *Ising model* [4.57] is defined as follows. Let $G = L^d$ be a d -dimensional lattice. Associated with each lattice site i is a spin s_i which can take on the values $+1$ or -1 . The spins interact via an *exchange coupling* J . In addition, we allow for an external field H . The Hamiltonian reads

$$\mathcal{H} = -J \sum_{\langle i, j \rangle} s_i s_j + \mu H \sum_i s_i . \quad (4.48)$$

The first sum on the left-hand side of the equation runs over nearest neighbours only. The symbol μ denotes the magnetic moment of a spin. If the exchange constant J is positive, the Hamiltonian is a model for *ferromagnetism*, i.e., the spins tend to align parallel. For J negative the exchange is *antiferromagnetic* and the spins tend to align antiparallel. In what follows we assume a ferromagnetic interaction $J > 0$.

The Ising model exhibits a *phase transition* (see, for example, [4.39] by Stanley for an introduction to phase transitions). It has a *critical point* T_c where a second-order transition occurs. For temperatures T above T_c the *order parameter*, i.e., the magnetization m (number of "up" spins minus number of "down" spins divided by the total number of spins), is zero in zero magnetic field. For temperatures T below T_c there is a two-fold degenerate spontaneous magnetization. The phase diagram for the model is displayed schematically in Fig.4.3.

To calculate, for example, the magnetization of the three-dimensional model we can use the microcanonical Monte-Carlo method. The magnetization will be a function of the energy. However, with the distribution of the demon energy we also obtain the magnetization as a function of temperature. For simplicity, we set the applied field to zero.

Let E be the fixed energy and suppose that a spin configuration $s = (s_1, \dots, s_N)$ was constructed with the required energy. We set the demon energy to zero and let it travel through the lattice. At each site the demon attempts to flip the spin at that site. If the spin flip lowers the system energy,

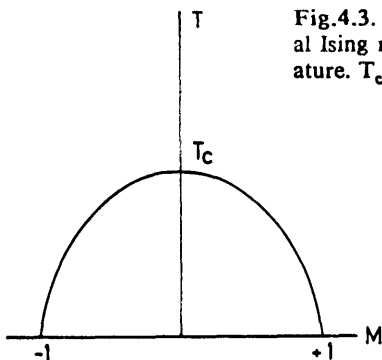


Fig.4.3. Schematic phase diagram of the three dimensional Ising model. M is the magnetization and T the temperature. T_c is the critical point

then the demon takes up the energy and flips the spin. On the other hand, if a flip does not lower the system energy the spin is only flipped if the demon carries enough energy. A spin is flipped if

$$E_D - \Delta \mathcal{H} > 0 \quad (4.49)$$

and the new demon energy is

$$E_D = E_D - \Delta \mathcal{H} . \quad (4.50)$$

After having visited all sites one "time unit" has elapsed and a new configuration is generated. In Monte-Carlo method language the time unit is called the *MC step per spin*. After the system has relaxed to thermal equilibrium, i.e., after n_0 Monte-Carlo Steps (MCS), the averaging is started. For example, we might be interested in the magnetization. Let n be the total number of MCS, then the approximation for the magnetization is

$$\bar{m} = \frac{1}{n-n_0} \sum_{i \geq n_0}^n m(s_i) , \quad (4.51)$$

where s_i is the i th generated spin configuration.

Since the demon energies ultimately become Boltzmann distributed, it is easy to show that

$$J/k_B T = 1/4 \ln(1 + 4J/\langle E_D \rangle) . \quad (4.52)$$

To carry out the simulation we use a simple cubic lattice of size 32^3 . Initially all spins are set "down". Then we select spins at random and turn them over until the desired energy is reached. From then on we proceed as developed above.

Figure 4.4 shows the resulting distribution of E_D at the fixed energy E after 3000 MCS and 6000 MCS. The exact value of the temperature is $T/T_c = 0.5911$, corresponding to E . The results from the simulations are

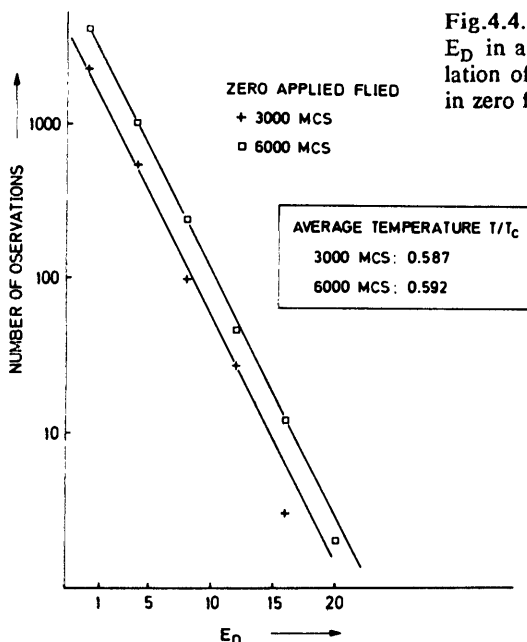


Fig.4.4. Distribution of the demon energy E_D in a microcanonical Monte-Carlo simulation of the three dimensional Ising model in zero field

$T/T_c = 0.587$, 3000 MCS , $T/T_c = 0.592$, 6000 MCS.

A fairly large number of Monte-Carlo steps are needed before the demon reflects the real temperature. This is to be expected since the relaxation into thermal equilibrium is governed by conservation laws. Due to the energy conservation a slow approach to equilibrium results for the demon representing the temperature. □

In the foregoing example no mention was made of the boundary conditions imposed on the system. How does a particle interact across the boundary? Several possible choices exist, which we group as

- 1) periodic boundary conditions,
- 2) free boundary conditions, and
- 3) non-standard boundary conditions.

In the third category we lump together boundary conditions which create effects that are not yet fully understood. An example falling into this class is the *self-consistent field* boundary condition [4.32, 33, 58]. Better understood in their behaviour [4.36] are the *periodic* and the *free boundary conditions*. The periodic boundary condition applies to a hypercubic system and was employed in the MD simulation. There this boundary condition was selected to eliminate surface effects to simulate the bulk behaviour. The same applies here because we are primarily concerned with the behaviour of the system in the thermodynamic limit. Let $L_1, \dots, L_d$ be the linear dimensions of the box. For any observable A we have

$$A(\mathbf{x}) = A(\mathbf{x} + \mathbf{L}_i), \quad i = 1, \dots, d \quad (5.53)$$

where

$$\mathbf{L}_i = (0, \dots, 0, L_i, 0, \dots, 0) .$$

The periodic boundary condition establishes translational invariance and eliminates surface effects to a large extent. Conceptually the system is infinite; however, it can still accommodate only finite lengths.

Some types of problems require mixed boundary conditions. Studies of wetting phenomena [4.59] furnish examples where both periodic and free boundaries are combined [4.60].

4.3.2 Canonical Ensemble Monte-Carlo Method

The Metropolis method for a constant temperature Monte-Carlo simulation is introduced. We treat various examples where this method is used to calculate thermodynamic quantities. Also a non-local simulation method is introduced.

In contrast to the microcanonical ensemble where all states have equal a priori weight, in the canonical ensemble some states are assigned different weights. A simple random walk through phase space is not applicable for the evaluation of observables in the (N, V, T) ensemble. In thermodynamic equilibrium some states occur more frequently. To generate a path such that the states occur with the correct probability, a Markov process has to be constructed, yielding a limit distribution corresponding to the equilibrium distribution of the canonical ensemble.

In the canonical ensemble the particle number N , the volume V , and the temperature T are fixed. In such a situation an observable A is computed as

$$\langle A \rangle = \frac{1}{Z} \int_{\Omega} A(\mathbf{x}) \exp[-\mathcal{H}(\mathbf{x})/k_B T] d\mathbf{x} \quad (4.54)$$

$$Z = \int_{\Omega} \exp[-\mathcal{H}(\mathbf{x})/k_B T] d\mathbf{x} .$$

To develop a heat-bath Monte-Carlo method we note that in equilibrium the distribution of states is

$$P(\mathbf{x}) = Z^{-1} \exp[-\mathcal{H}(\mathbf{x})/k_B T] . \quad (4.55)$$

If we impose the detailed balance condition in equilibrium we find

$$W(\mathbf{x}, \mathbf{x}')P(\mathbf{x}) = W(\mathbf{x}', \mathbf{x})P(\mathbf{x}')$$

or

$$W(\mathbf{x}, \mathbf{x}')/W(\mathbf{x}', \mathbf{x}) = P(\mathbf{x}')/P(\mathbf{x}) . \quad (4.56)$$

Due to the property of the exponential, the ratio of the transition probabilities depends only on the change in energy $\Delta\mathcal{H}$ on going from one state to another

$$W(\mathbf{x}, \mathbf{x}')/W(\mathbf{x}', \mathbf{x}) = \exp\{-[\mathcal{H}(\mathbf{x}') - \mathcal{H}(\mathbf{x})]/k_B T\} = \exp(-\Delta\mathcal{H}/k_B T) . \quad (4.57)$$

We may use the form (4.41) developed in Sect.4.3 to specify the transition probability for the Metropolis MC method

$$W(\mathbf{x}, \mathbf{x}') = \begin{cases} \omega_{\mathbf{x}\mathbf{x}'} , & \exp(-\Delta\mathcal{H}/k_B T) \text{ if } \Delta\mathcal{H} > 0 \\ \omega_{\mathbf{x}\mathbf{x}'} , & \text{otherwise} . \end{cases} \quad (4.58)$$

The numbers $\omega_{\mathbf{x}\mathbf{x}'}$ are still at our disposal. The only requirements they have to fulfil are those stated in (4.40). $W(\mathbf{x}, \mathbf{x}')$ is the transition probability per unit time and the ω 's determine the time scale.

Algorithm A11. Canonical Monte-Carlo Method

- 1) Specify an initial configuration.
- 2) Generate a new configuration $\mathbf{x}'$.
- 3) Compute the energy change $\Delta\mathcal{H}$.
- 4) If $\Delta\mathcal{H} < 0$, accept the new configuration and return to Step 2.
- 5) Compute $\exp(-\Delta\mathcal{H}/k_B T)$.
- 6) Generate a random number $R \in [0, 1]$.
- 7) If R is less than $\exp(-\Delta\mathcal{H}/k_B T)$, accept the new configuration and return to Step 2.
- 8) Otherwise, retain the old configuration as the new one and return to Step 2.

At this point we see more clearly the meaning of the choice of transition probabilities. The system is driven towards the minimum energy corresponding to the parameters (N, V, T) . Step 4 says that we always accept a new configuration having less energy than the previous one. Configurations which raise the energy are only accepted with a Boltzmann probability.

Example 4.3

To demonstrate an implementation of the canonical-ensemble Monte-Carlo method, we use again the Ising model already familiar to us from the previous section. The first step in constructing an algorithm for the simulation of the model is the specification of the transition probabilities from one state to another. The simplest and most convenient choice for the actual

simulation is a transition probability involving only a single spin; all other spins remain fixed. It should depend only on the momentary state of the nearest neighbours. After all spins have been given the possibility of a flip a new state is created. Symbolically, the *single-spin-flip* transition probability is written as

$$W_i(s_i) : (s_1, \dots, s_i, \dots, s_N) \rightarrow (s_1, \dots, -s_i, \dots, s_N) ,$$

where W_i is the probability per unit time that the i th spin changes from s_i to $-s_i$. With such a choice the model is called the *single-spin-flip Ising model* [4.61]. Note that in the single-spin-flip Ising model the numbers of up spins $N_\uparrow$ and down spins $N_\downarrow$ are not conserved, though the total number $N = N_\uparrow + N_\downarrow$ is fixed. It is, however, possible to conserve the order parameter [4.27]. Instead of flipping a spin, two nearest-neighbour spins are exchanged if they are of opposite sign. This is the Ising model with so-called *Kawasaki dynamics* [4.37]. In this particular example the volume is an irrelevant parameter. The volume and the number of particles enter only through their ratios, i.e., $(V/N, T)$ are the parameters.

To proceed we have to derive the actual form of the transition probability. Let $P(s)$ be the probability of the state s . In thermal equilibrium at the fixed temperature T and field K , the probability that the i th spin takes on the value s_i is proportional to the Boltzmann factor

$$P_{\text{eq}}(s_i) \propto \exp[-\mathcal{H}(s_i)/k_B T] . \quad (4.59)$$

The fixed spin variables are suppressed. We require that the detailed balance condition be fulfilled:

$$W_i(s_i)P_{\text{eq}}(s_i) = W_i(-s_i)P_{\text{eq}}(-s_i)$$

or

$$W_i(s_i)/W_i(-s_i) = P_{\text{eq}}(-s_i)/P_{\text{eq}}(s_i) . \quad (4.60)$$

With (4.48) it follows that

$$W_i(s_i)/W_i(-s_i) = \exp(-s_i E_i)/\exp(s_i E_i) , \quad (4.61)$$

where

$$E_i = J \sum_{\langle i, j \rangle} s_j .$$

The derived conditions (4.59–61) do not uniquely specify the transition probability W . We have a certain freedom to choose W to be numerically efficient. At least two choices of transition probabilities are consistent with (4.61):

The *Metropolis function* [4.30]

$$W_i(s_i) = \min\{\tau^{-1}, \tau^{-1} \exp(-\Delta\mathcal{H}/k_B T)\} \quad (4.62)$$

and the *Glauber function* [4.61]

$$W_i(s_i) = (1 - s_i \tanh E_i/k_B T)/2\tau, \quad (4.63)$$

where τ is an arbitrary factor determining the time scale. Usually τ is set to unity. To simulate the physical system, for which the Hamiltonian (4.48) is a model, more closely, we could consider the factor τ to depend on parameters like the temperature.

In Sect.4.3 we described a dynamic interpretation of the MC method. The question arising is how far dynamic properties such as dynamic correlation functions are influenced by the choice of the transition probabilities. Near thermal equilibrium this choice leads only to a renormalization of the time scale [4.31]. However, for states far from equilibrium, the choice greatly influences the relaxation towards equilibrium [4.62].

In what follows we choose the Metropolis function. Having specified the transition probabilities guaranteeing the relaxation into thermal equilibrium, the essential step in the development is done. Suppose an initial configuration is specified. First a lattice site has to be selected. This can be done either by going through the lattice in a typewriter fashion, or by selecting sites at random. Then W_i is computed. Next a random number is generated to be compared with the transition probability. If the probability of a transition is larger than the random number, the transition from s_i to $-s_i$ is accepted. Otherwise the spin remains in the state s_i . The algorithm proceeds by selecting a new site. After all sites have been visited once by the typewriter method, or N choices of sites in a random fashion have been made, a new state of the system is generated. This comprises one time unit, or one Monte-Carlo step. How far the Monte-Carlo time, which depends on τ , corresponds to time in a physical system is still an unresolved question [4.49, 50].

Algorithmically the Metropolis MC method looks as follows:

- 1) Specify an initial configuration.
- 2) Choose a lattice site i .
- 3) Compute W_i .
- 4) Generate a random number $R \in [0, 1]$.
- 5) If $W_i(s_i) > R$, then $s_i \rightarrow -s_i$.
- 6) Otherwise, proceed with Step 2 until N attempts have been made.

Figure 4.5 shows the results of Monte-Carlo simulations for the magnetization of the three-dimensional Ising model at various temperatures. The simulation had a duration of 1000 MCS. The first 500 steps were discarded and the magnetization averaged over the second 500 steps. The different symbols denote lattices of various sizes. To give a feeling for the computational needs, the inset shows the required execution time in seconds

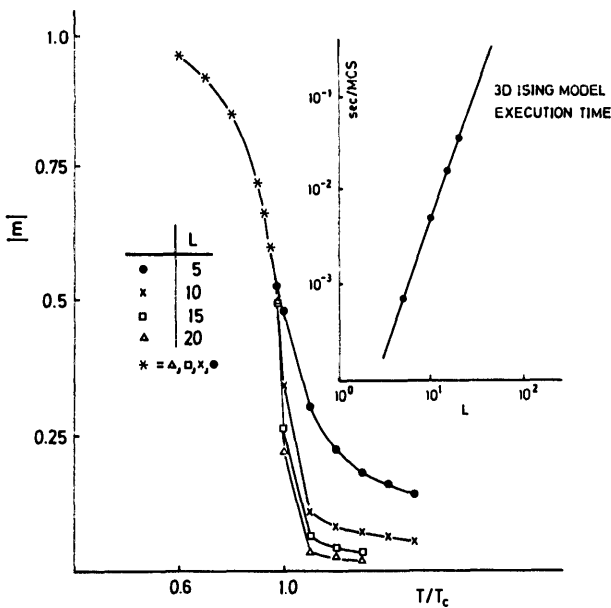


Fig.4.5. Magnetization for various temperatures and lattice sizes for the three dimensional Ising model with single spin flip. The inset shows the execution time requirements. The Monte-Carlo simulations proceeded for 1000 MCS and the averages were performed using the second 500 steps

for one Monte-Carlo step. The time increases proportional to the system size $N = L^3$. These execution times were obtained with the program PL4 listed in Appendix A2. That the execution time increases linearly with the system size is not true in general. Some algorithms, especially those for vector machines and parallel computers, perform in a different way (see references listed in conjunction with the discussion of the program PL4).

From the observed values it is apparent that the magnetization depends on the lattice size. The effect is most dramatic near the critical temperature. For low temperatures, i.e., T much smaller than T_c , the results are less sensitive to the lattice size. Indeed, the magnetization there converges to the true thermodynamic limit value rapidly. For high temperatures the magnetization is non-zero, though in the thermodynamic limit there is no spontaneous magnetization.

The behaviour of the magnetization is one typical example of finite-size effect occurring near second-order phase transitions [4.40, 63-66]. It can be understood by considering the correlation length. As the critical temperature is approached, the correlation length diverges, so that the finite system can accommodate only finite lengths. Hence, there will be rounding effects. In the case of first- and second-order phase transitions, the finite-size effects can be treated systematically [4.50]. Other situations require at least at the moment an ad hoc analysis.

Note that in Fig.4.5 the magnetization is plotted with absolute values. This is due to the two-fold degeneracy of the magnetization in the thermo-

dynamic limit. For each temperature below the critical temperature there is a spontaneous magnetization $+m(T)$ or $-m(T)$. For finite systems the delta functions are smeared out to two overlapping Gaussians, and the system has a finite probability for going from a positive to a negative magnetization. *It is therefore essential to accumulate the absolute values for the average.*

Here we come back again to the question of ergodicity. In the Ising model an effectively broken ergodicity occurs. For a temperature below the critical temperature, the system may have either a positive or negative magnetization. During the course of a simulation both orderings are explored in a finite system if the observation time is long enough. The free-energy barrier between the two orderings is of the order $N^{(d-1)/d}$ [4.42] and the relaxation time is roughly $\exp(aN^{(d-1)/d})$. Depending on the observation time and the size of the system, the states generated by the MC simulation may explore only one ordering. $\square$

There is a difficulty with the transition probability. Suppose $\mathcal{K} \gg k_B T$ or suppose $k_B T \simeq 0$. Due to the exponential function, Monte-Carlo moves in such a situation occur very infrequently. The acceptance probability is proportional to $\exp(-\Delta\mathcal{K}/k_B T)$! The motion through phase space is slow and an enormous number of states have to be generated in order for the system to reach equilibrium. If the system has continuous state variables, for example, in a simulation of the Lennard-Jones system, with MC methods, we can speed up the convergence. Let x_i denote the position of an atom. We generate a trial position x'_x by $x'_x = x_i + \delta$ where δ is a random number from the interval $[-\delta, +\delta]$. To raise the acceptance rate of the Monte Carlo moves we simply choose δ appropriately. However, there is a danger that the constraint introduces inaccuracies.

In the case where $k_B T \simeq 0$ we have to resort to other methods to speed up convergence [4.36, 65, 67, 68]. In particular, we could develop an algorithm where only successful moves are made (cf. the discussion on the Monte-Carlo realization of the Master equation in Sect.4.3). The time intervals in such a method are then not equidistant.

In the general discussion of the Monte-Carlo technique we mentioned that the partition function itself is not directly accessible in a simulation. However, methods exist [4.69-81] which circumvent the problem. One way, of course, is to integrate the results on thermodynamic variables related to the free energy by derivatives [4.70-73]. Let us take as an example the Ising problem where the relevant thermodynamic variables, i.e., the internal energy U and the magnetization, are related to the free energy and the entropy by

$$U = - T^2 \frac{\partial(F/T)}{\partial T} \Big|_H \quad \text{and} \quad M = - \frac{\partial F}{\partial H} \Big|_T .$$

Integrating these relations we get

$$S(T, H) = S(T', H) + U/T - k_B T \int_{1/k_B T'}^{1/k_B T} U d(1/k_B T) ,$$

$$F(T, H) = F(T, H') - \int_{H'}^H M dH .$$

Two problems arise for this approach. The main difficulty is that the reference entropy must be known. Only in some cases is an exact result available for the entropy. Second, the energy has to be computed along the path of integration, which can be quite a formidable task. Of course, one should not neglect the possible finite-size effects in the energy, though usually the energy is fairly insensitive to finite-size effects.

A quite interesting method, proposed by *Ma* [4.74], does not make use of integration but tries to estimate the entropy directly. Recall that the entropy is defined by

$$S = - k_B \sum_i p_i \ln(p_i) , \quad (4.64)$$

where p_i is the probability that the state x_i occurs. In principle, we could simply count the number of occurrences of the states and divide by the total number of observations. But, if the number of states is large, as it usually is (2^N for the Ising model with $N = 100^3$, for example), the frequency will be quite small. To raise the frequency we perform some kind of coarse-graining. Suppose the states are grouped into classes C_i . Each time a state in class i occurs we count this as one occurrence of class C_i . Let n_i be the frequency. Then $P_i = n_i/n$ (n is the total number of observations) is the probability of finding a state belonging to the class i . Furthermore, let R_i denote the probability of finding a state along the trajectory, i.e., along the Markov chain, that falls into the class i . Then

$$S = - k_B \sum_i P_i \ln(R_i) . \quad (4.65)$$

The crucial point in the approach is the separation into classes and the distribution of states inside a class. Assume the states were uniformly distributed in each class. If there are g_i states in class i we have

$$R_i = P_i/g_i .$$

For most problems it is not at all obvious how to classify the states.

Example 4.4

Up to now we have discussed examples with a discrete local state. In the Ising model the local state, i.e., the spin orientation s_i can be either $+1$ or -1 . What we want to study in this example is a model with the Hamiltonian

$$\mathcal{H}(c) = \sum_i \left[\frac{r}{2} c_i^2 + \frac{u}{4} c_i^4 \right] + \frac{C}{2} \sum_{\langle i, j \rangle} (c_i - c_j)^2, \quad (4.66)$$

where r, u, C are constants, and the local state variable c_i may assume values between $-\infty$ and $+\infty$. This Hamiltonian is related to the coarse-grained Landau-Ginzburg-Wilson free-energy functional of Ising models [4.82, 83]. We shall not be concerned with the precise relation [4.84]. We just mention that the parameters and the c_i 's are the result of a coarse-graining procedure involving blocks of spins. Here we want to develop a Monte-Carlo algorithm to simulate the model given by the above Hamiltonian.

The first step we shall carry out is to scale the Hamiltonian to reduce the number of parameters. For this we consider the mean-field approximation of the solution to the model. In the mean-field approximation possible spatial fluctuations of the order parameter are neglected. Hence, the second sum on the right-hand side of (4.66) can be ignored and the partition function is

$$Z = \text{Tr}_{\{c_i\}} \exp \left[- \sum_i \left[\frac{r}{2} c_i^2 + \frac{u}{4} c_i^4 \right] \right]. \quad (4.67)$$

If r is less than zero we can work out the free energy and find the order parameter

$$c_{\text{MF}} = \pm \sqrt{-r/u}. \quad (4.68)$$

If r is larger or equal to zero, the order parameter is identical to zero.

Next we need to know the susceptibility χ and the correlation length ξ

$$\begin{aligned} \chi(q) &= \chi_{\text{MF}} (1 + q^2 \xi^2)^{-1}, \\ \chi_{\text{MF}} &= (-2r)^{-1}, \\ \xi_{\text{MF}} &= \sqrt{C/(-2r)}, \end{aligned} \quad (4.69)$$

where q is the wave vector, and the above equations were obtained in linear response. Having derived the order parameter we normalize c_i with the mean-field order parameter $m_i = c_i / c_{\text{MF}}$ to find

$$\mathcal{H} = \frac{r^2}{u} \left[\sum_i \left[-\frac{m_i^2}{2} + \frac{m_i^4}{4} \right] + \xi_{\text{MF}}^2 \sum_{\langle i, j \rangle} (m_i - m_j)^2 \right]. \quad (4.70)$$

Essentially, we are left with two parameters to vary. However, we would like to have the Hamiltonian in a slightly different form. Let us evaluate the square in the second term on the right-hand side of (4.66) and rearrange the terms yielding

$$\mathcal{H} = \sum_i \left[\frac{r+2dC}{2} c_i^2 + \frac{u}{4} c_i^4 \right] - C \sum_{\langle i, j \rangle} c_i c_j, \quad (4.71)$$

where d is the dimension of the lattice. Recall that with each lattice site a local site variable c_i is associated, and that there are $2d$ nearest neighbours. Performing again a normalization

$$m_i = c_i [-(r + 2dC)/u]^{-1/2} \quad (4.72)$$

we find

$$\mathcal{H} = \alpha \sum_i (-\frac{1}{2} m_i^2 + \frac{1}{4} m_i^4) - \beta \sum_{\langle i, j \rangle} m_i m_j. \quad (4.73)$$

Notice the resemblance of the Hamiltonian to the Ising Hamiltonian in the previous examples. The recognition of the formal resemblance is the major step for the development of the algorithm. Why do we not set up the algorithm directly? In simulating directly a model such as (4.66) one encounters the difficulty that the local variable m_i is not bound. One may replace the interval $(-\infty, \infty)$ by $[-a, a]$. However, serious inaccuracies result from such a choice due to the truncation. Instead of truncating the interval we choose m_i 's with the single-site probability

$$P_i \propto \exp[-\alpha(-\frac{1}{2} m_i^2 + \frac{1}{4} m_i^4)] \quad (4.74)$$

and allow all values of m_i within the possible numerical accuracy. Figure 4.6 shows the distribution for two parameter values as obtained during a simulation.

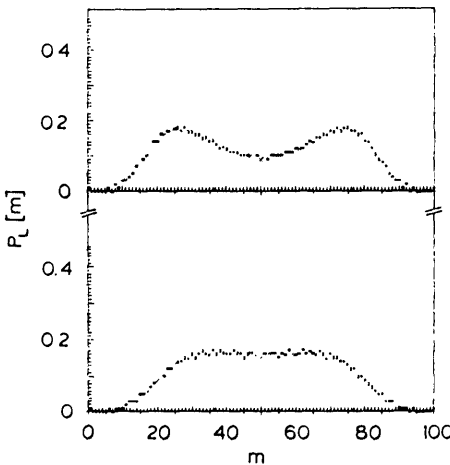


Fig.4.6. Probability distribution (4.74) of the local variable m_i as obtained by sampling

Let us split the Hamiltonian into two parts [4.85, 86]

$$\mathcal{H} = \mathcal{H}_1 + \mathcal{H}_2 ,$$

where

$$\mathcal{H}_1 = -\beta \sum_{\langle i, j \rangle} m_i m_j , \quad \mathcal{H}_2 = \alpha \sum_i (-\frac{1}{2} m_i^2 + \frac{1}{4} m_i^4) . \quad (4.75)$$

What have we gained? Let us write the expectation for an observable as

$$\begin{aligned} \langle A \rangle &= \frac{1}{Z} \int_{\Omega} A(m) \exp[-\mathcal{H}(m)] dm \\ &= \frac{1}{Z} \int_{\Omega} A(m) \exp[-\mathcal{H}_1(m) - \mathcal{H}_2(m)] dm . \end{aligned} \quad (4.76)$$

Because the Hamiltonian is split into two parts we may introduce a new measure (recall the procedure to introduce to the Monte-Carlo technique to reduce the variance)

$$d\lambda(m) = Z^{-1} \exp[-\mathcal{H}_2(m)] dm \quad (4.77)$$

and obtain

$$\langle A \rangle = \frac{1}{Z'} \int_{\Omega} A(m) \exp[-\mathcal{H}_1(m)] d\lambda(m) . \quad (4.78)$$

With this we have succeeded in reducing (4.66) to the problem of calculating the expectation of A within the Ising model with continuous spin variables. Instead of just flipping a spin we must choose a trial m_i distributed according to the measure $d\lambda$. Clearly $d\lambda$ can vary between zero and one; thus, a homogeneously distributed set of points in the phase space is mapped on the interval $[0, 1]$ with a density governed by the factor $\exp[-\mathcal{H}_2(m)]$ by means of the cumulative distribution

$$C(y) = \int_{-\infty}^y d\lambda(m) / \int_{-\infty}^{+\infty} d\lambda(m) . \quad (4.79)$$

The trial m_i is obtained by generating first a random number $r \in [0, 1]$ and then calculating the inverse of $C(r)$. The Monte-Carlo algorithm using the above looks like

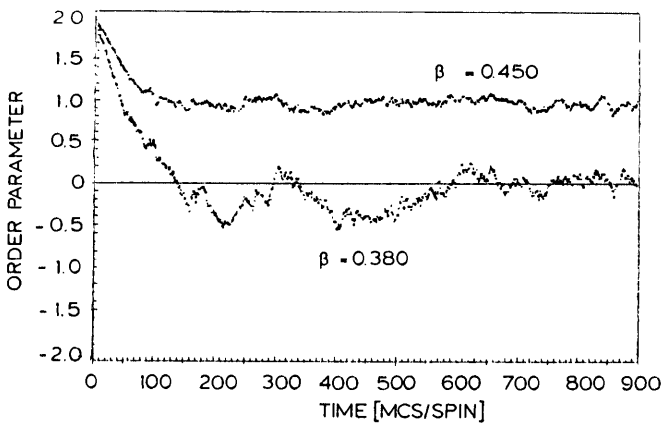


Fig.4.7. Shown is the order parameter relaxation for two values of the parameter β

- 1) Select a site i of the lattice.
- 2) Generate a random number $r \in [0, 1]$.
- 3) Invert $C(r)$ to get a trial value for m_i .
- 4) Compute the change $\Delta\mathcal{H}_1$ in the "Ising part" of the Hamiltonian
- 5) Generate a random number $R \in [0, 1]$.
- 6) If R is less than $\exp(-\Delta\mathcal{H}_1)$ accept the trial value m_i .
- 7) Otherwise, reject the trial m_i and the old state becomes the new state.
- 8) Go to Step 1.

Of course, there is no need to invert the functional for each trial. One may store a convenient number of $C^{-1}(r)$ in a table and interpolate for r values not stored.

The relaxation of the system from an initial state has already been mentioned several times. Figure 4.7 displays how the order parameter relaxes into equilibrium for two values of the parameter β with fixed α . We notice that the relaxation for $\beta = 0.45$ proceeds faster than for $\beta = 0.28$. Accordingly, different portions of the initial chain have to be discarded.

The results for the order parameter as a function of β are shown in Fig.4.8. As for the Ising model with a discrete local state, we observe a pronounced finite-size dependence. Below the critical point where the correlation length is small, the finite-size effects start to disappear. The data can, however, be collapsed to a single curve, as shown in Fig.4.9. The values for the magnetization above and below near the critical point scale. This is an example of finite-size scaling.

Finite-size effects are also dramatic in the susceptibility (Fig.4.10)

$$\chi_L \propto \langle m^2 \rangle - \langle m \rangle^2. \quad (4.80)$$

At the critical point the susceptibility diverges in the thermodynamic limit. Due to the finite size of the system, the divergence is rounded. In addition, the finite size leads to a shift in the critical temperature. $\square$

Fig.4.8. Finite size dependence of the order parameter

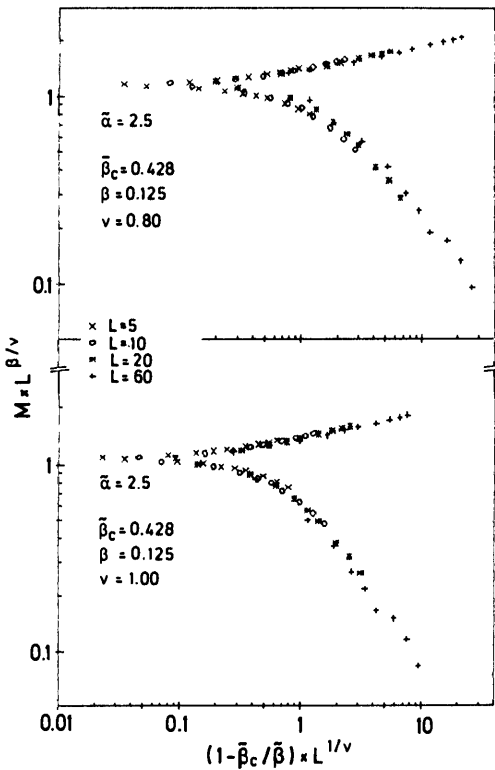
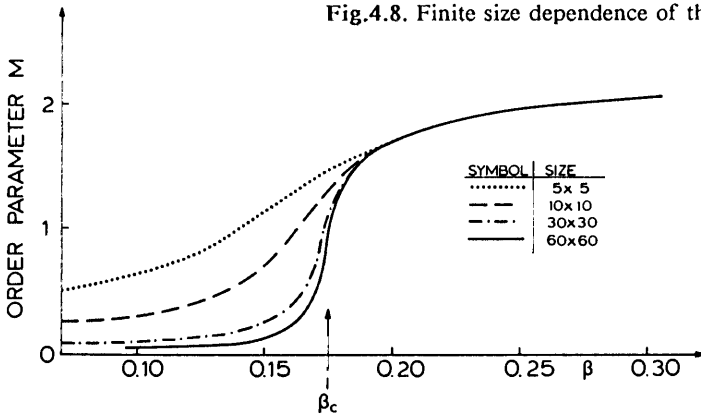
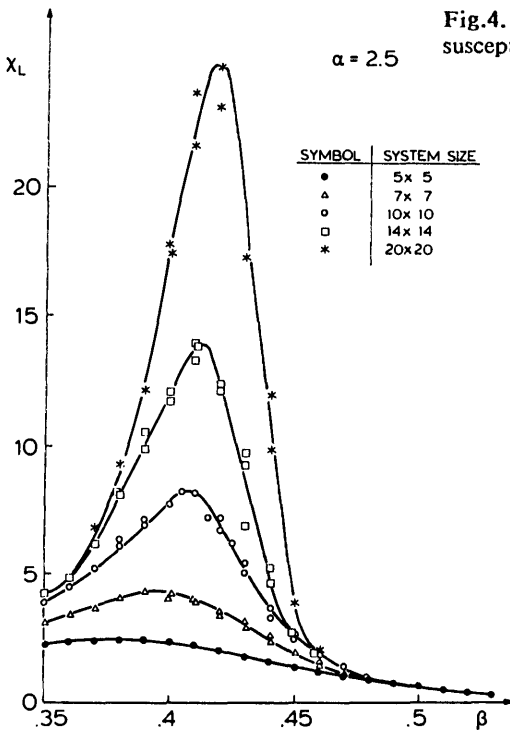


Fig.4.9. Finite size scaling plot of the order parameter

So far we have only encountered Monte-Carlo simulations on lattices with simple local objects, or off-lattice simulations where the particles could be moved locally. There exists the possibility of introducing changes in the configuration on a more global level.

One of the problems, especially close to second-order phase transitions, is the critical slowing down. The system there behaves in a very correlated fashion. Local changes, as they are produced, for example with the Metrop-

Fig.4.10. Finite size dependence of the susceptibility



olis importance sampling, cannot propel the system fast enough through phase space. The result is a very slow relaxation into equilibrium and the continued large correlation between successive configurations.

In the following example we want to examine a reformulation of the Ising model which will allow us to introduce larger changes to the configurations. This in turn leads to a reduction in the critical slowing down [4.87-89].

Example 4.5

The system for which we formulate the algorithm is the Ising model with the Hamiltonian

$$\mathcal{H}_{\text{Ising}} = -J \sum_{\langle i, j \rangle} s_i s_j \quad (4.81)$$

which we have met before several times. This also allows an immediate comparison. In principle, the algorithm can also be formulated for the Potts model.

To understand the reasoning and the algorithm it is perhaps best to first give a quick run through the main ideas and then go into a little bit more detail.

The main idea was put forward by *Fortuin and Kastelyn* [4.90]. They proposed, and succeeded in showing, that the Ising model Hamiltonian could be mapped onto the percolation problem, which we encountered at the very beginning of this text. The mapping gives a new partition function

$$Z = \sum_n B(\beta, n) 2^{c(n)}, \quad (4.82)$$

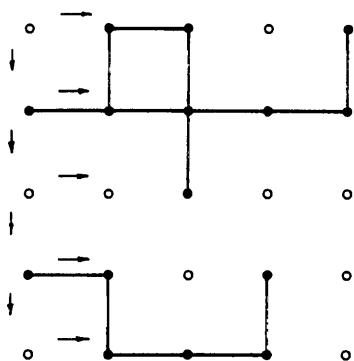
i.e., a combinatorial factor and contributions from the two possible cluster orientations. Instead of single spins we now have to talk about patches, or clusters of spins. Each cluster is independent of the other clusters.

We see now the advantage of such a reformulation. Instead of turning over single spins, we are able to turn over entire clusters of spins. This brings about large changes from one configuration to the other.

To perform a Monte-Carlo simulation using this idea *Swendsen and Wang* [4.87] designed an algorithm to produce the clusters and to go from one configuration to another. A configuration in the Swendsen-Wang method consists of an assignment of spin orientations to the lattice sites and an assignment of bonds between parallel spins. Consider such a configuration of a lattice with spin up and spin down. On top we have the bonds which are always broken between spins of opposite direction. Between spins of the same direction a bond can be present. An example is depicted in Fig.4.11. A cluster is defined as follows. Two up spins belong to the same cluster if they are nearest neighbours and if there is a bond between them.

Once all clusters of up spins and all clusters of down spins have been identified we can proceed to generate a new configuration. The first step consists in choosing a new orientation for each cluster. In the model without a magnetic field, the new orientation for each cluster is chosen at random, i.e., with a probability $\frac{1}{2}$ the orientation is reversed. After this reorientation all bonds are deleted so that only the spin configuration remains. Now the process of a bond assignment and new cluster orientation is repeated.

We shall now derive the probability with which we must assign a bond between parallel spins [4.89]. Let us derive this for the Ising model with a magnetic field



$$\mathcal{H}_{\text{Ising}} = -J \sum_{\langle i, j \rangle} s_i s_j + \mu H \sum_i s_i . \quad (4.83)$$

Let $P(\mathbf{s}) = Z^{-1} \exp(-\beta \mathcal{H})$ be the probability for the occurrence of the configuration $\mathbf{s}$. We shall define a new Hamiltonian

$$\tilde{\mathcal{H}}_{\text{Ising}} = J \left[N_B - \sum_{\langle i, j \rangle} s_i s_j \right] + H \left[N_s + \sum_i s_i \right] . \quad (4.84)$$

This Hamiltonian is zero for the ground state at zero temperature. Here N_B and N_s are the number of bonds on the lattice and the number of sites, respectively.

Denote by $N_p(\mathbf{s})$ the set of all bonds that lie between two parallel spins and $c(\Gamma)$ the set of all closed bonds [$c(\Gamma) \subset N_p(\mathbf{s})$]. Define p to be the probability of the presence of a bond and q the absence of a bond. Then we have for the probability of getting a bond configuration Γ

$$P(\Gamma) = \sum_{\mathbf{s}} P(\mathbf{s}) P(\Gamma|\mathbf{s}) \quad (4.85)$$

where the conditional probability that a bond configuration Γ is generated from a spin configuration $\mathbf{s}$ is

$$P(\Gamma|\mathbf{s}) = \delta_{\Gamma, \mathbf{s}} p^{c(\Gamma)} q^{N_p(\mathbf{s}) - c(\Gamma)} \quad (4.86)$$

Suppose we have a spin configuration. Choose the following spin orientations for the clusters λ

- -1 with probability $s = \{1 + \exp[2\beta H N(\lambda)]\}^{-1}$
- +1 with probability $t = 1 - s$

where $N(\lambda)$ is the number of sites associated with the cluster λ . Thus, the probability of generating a new spin configuration $\mathbf{s}'$ is given by

$$P(\mathbf{s}') = \sum_{\Gamma} P(\Gamma) P(\mathbf{s}'|\Gamma) \quad (4.87)$$

where $P(\mathbf{s}'|\Gamma)$ is the conditional probability that the spin configuration $\mathbf{s}'$ is generated from Γ , i.e.,

$$P(\mathbf{s}'|\Gamma) = \delta_{\Gamma, \mathbf{s}'} s^{\tau^-(\Gamma)} t^{\tau^+(\Gamma)} \quad (4.88)$$

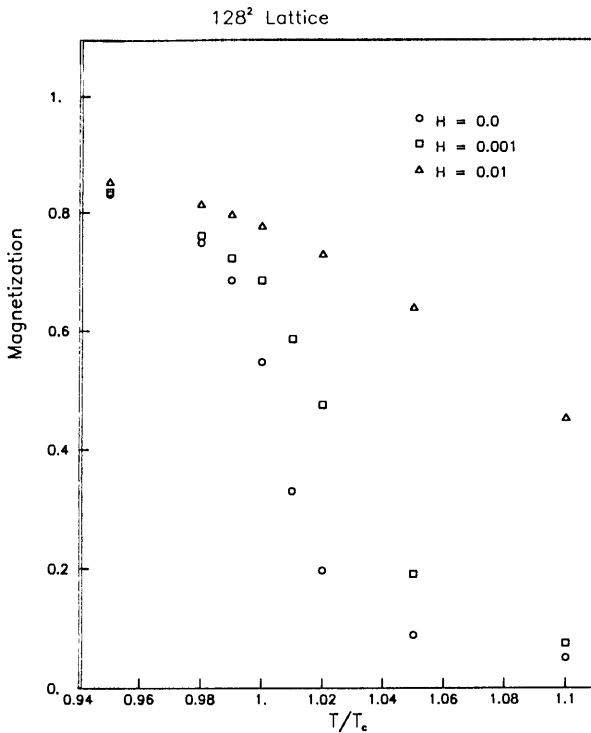


Fig.4.12. Monte-Carlo results using the Swendsen-Wang algorithm for the Ising model with a magnetic field. The system size is 128²

with $\gamma^-(\Gamma)$ being the number of clusters of spin -1 and $\gamma^+(\Gamma)$ the number of clusters of spin +1. The total number of clusters is given by $\gamma(\Gamma) = \gamma^-(\Gamma) + \gamma^+(\Gamma)$

We can now work out $P(\Gamma)$ and find that

$$q = \exp(-2\beta J) . \quad (4.89)$$

Hence bonds are present on the lattice with $p = 1 - \exp(-2\beta J)$ and clusters are reoriented with probability s .

Finally the partition function for the normalized Hamiltonian is

$$\tilde{Z} = \sum_{\Gamma} p^{c(\Gamma)} q^{N_B - c(\Gamma)} \prod_{\lambda}^{\gamma(\Gamma)} [1 + \exp(2\beta H N(\lambda))] . \quad (4.90)$$

Figure 4.12 gives the result of a Monte-Carlo simulation of a 128² lattice with different applied magnetic fields. At this point we must exercise some caution. There is no a priori argument guaranteeing us that the finite size effects are the same as those for the Metropolis or Glauber Ising model. Indeed, if we analyse the configurations using the percolation probability as

the order parameter, which in the thermodynamic limit is the same as the magnetization, we find that for temperatures above the critical point the size effects are different [4.91]. $\square$

Beside the above example, other proposals for speeding up the relaxation and reducing the correlation between configurations have been made. Most prominent are the Fourier acceleration [4.92] and the hybrid Monte-Carlo method [4.93].

4.3.3 Isothermal-Isobaric Ensemble Monte-Carlo Method

One possible method for dealing with an ensemble of constant pressure and constant temperature is given. The emphasis is put on the pressure part of the algorithm.

It must be emphasized that the algorithm which is presented here and the one presented in the next section on the grand ensemble MC method are by no means standard algorithms. Nor are they without problems. There are other possible methods and the reader is urged to try the ones presented here and compare them to the new ones and others in the present literature.

In the isothermal-isobaric ensemble observables are calculated as

$$\langle A \rangle = \frac{1}{Z} \int_0^\infty A Z_N(V, T) \exp(-PV/k_B T) dV, \quad (4.91)$$

$$Z = \int_0^\infty Z_N(V, T) \exp(-PV/k_B T) dV,$$

where Z_N is the partition function of the canonical ensemble with N particles so that

$$\langle A \rangle = \frac{1}{Z} \int_0^\infty \int_\Omega A(\mathbf{x}) \exp\{-[PV + \mathcal{H}(\mathbf{x})]/k_B T\} d\mathbf{x} dV. \quad (4.92)$$

The parameters at our disposal in a simulation are the particle number N , the pressure P and the temperature T . We shall restrict the exposition to a cubic volume with periodic boundary conditions, though, in principle, algorithms can be constructed allowing shape fluctuations.

Again we shall find it convenient to introduce scaled variables as in (3.75), namely

$$\rho_i = r_i/L. \quad (4.93)$$

Let us rewrite (4.82) in terms of the scaled variables

$$\langle A \rangle = \frac{1}{Z'} \int_0^\infty \int_\omega V^N \exp(-PV/k_B T) \exp[-\mathcal{H}(\mathbf{L}, \mathbf{p})/k_B T] A(\mathbf{L}, \mathbf{p}) d\mathbf{p} dV, \quad (4.94)$$

$$Z' = \int_0^\infty \int_\omega V^N \exp(-PV/k_B T) \exp[-\mathcal{H}(\mathbf{L}, \mathbf{p})/k_B T] d\mathbf{p} dV,$$

where ω is the unit cube. Let V_0 be a reference volume. Define a reduced volume and pressure

$$\tau = V/V_0, \quad \psi = PV_0/Nk_B T. \quad (4.95)$$

Then (4.94) becomes

$$\langle A \rangle = \frac{1}{Z''} V_0^{N+1} \int_0^\infty \int_\omega \tau^N \exp(-\psi N \tau) \exp[-\mathcal{H}(\mathbf{L}, \mathbf{p})/k_B T] A(\mathbf{L}, \mathbf{p}) d\mathbf{p} d\tau, \quad (4.96)$$

$$Z'' = V_0^{N+1} \int_0^\infty \int_\omega \tau^N \exp(-\psi N \tau) \exp[-\mathcal{H}(\mathbf{L}, \mathbf{p})/k_B T] d\mathbf{p} d\tau.$$

A straightforward approach to the design of an algorithm [4.38] is to define the function

$$\mathcal{H}' = N\psi\tau - N \ln \tau + \mathcal{H}(\mathbf{L}, \mathbf{p})/k_B T. \quad (4.97)$$

In addition to the coordinate variables, one new variable has appeared, corresponding to the volume. Apart from the new variable, everything can proceed as in the canonical ensemble Monte-Carlo method.

Algorithm A12. NPT Monte-Carlo Method

- 1) Specify an initial volume and coordinates.
- 2) Generate randomly a new volume τ' .
- 3) Generate a new set of coordinates $\mathbf{p}'$ consistent with the trial volume τ' .
- 4) Compute the change $\Delta \mathcal{H}'$.
- 5) If the change $\Delta \mathcal{H}'$ is less than zero, accept the new configuration and return to Step 2.
- 6) Compute $\exp(-\Delta \mathcal{H}')$.
- 7) Generate a random number $R \in [0, 1]$.
- 8) If R is less than $\exp(-\Delta \mathcal{H}')$, accept the new configuration and return to Step 2.
- 9) Otherwise, the old configuration is also the new one. Return to Step 2.

4.3.4 Grand Ensemble Monte-Carlo Method

A possible method for the Monte-Carlo simulation of the grand ensemble is given.

Finally we discuss the grand canonical ensemble MC method. The parameters to be specified are the chemical potential μ , the volume V and the temperature T . The grand ensemble MC method is of considerable interest since it allows fluctuations of the concentration. The major problem in constructing an algorithm for this ensemble is that the number of particles is not constant! Consider a volume V occupied initially by N particles. As time progresses, the number of particles inside the volume constantly changes. Particles have to be removed from or added to the volume. Which particles do we have to remove? Where should we put a particle? Before answering these questions we state how observables A are calculated in the grand canonical ensemble:

$$\langle A \rangle = \frac{1}{Z} \sum_N \left(\frac{a^N}{N!} \right) \int_{\Omega} A(\mathbf{x}^N) \exp[-U(\mathbf{x}^N)/k_B T] d\mathbf{x}^N, \quad (4.98)$$

$$Z = \sum_N (a^N/N!) \int_{\Omega} \exp[-U(\mathbf{x}^N)/k_B T] d\mathbf{x}^N,$$

where a is defined as

$$a = \lambda (h^2/2\pi m k_B T)^{-3/2}, \quad (4.99)$$

h being Planck's constant and the prefactor resulting from the momentum-space integration. The *absolute activity* is given by

$$\lambda = \exp(\mu/k_B T). \quad (4.100)$$

We notice again that the structure is similar to the canonical ensemble. The feature to be added is to let the number of particles fluctuate in the computational volume. A possible grand ensemble Monte-Carlo method involves three basic steps:

- 1) configuration changes in the coordinates of the particles;
- 2) creation of particles; and
- 3) destruction of particles.

We impose the constraint that in equilibrium an equal number of move, destruction and creation steps should be attempted. It is obvious

from (4.98) that the configurational changes need not be considered again. The procedure for this part is exactly the same as in the canonical-ensemble method. Consideration need only be given to the creation and destruction of a particle [4.94-97]. Suppose we add a particle at a random place to the volume V containing N particles. The probability of having $N + 1$ particles is

$$P(\mathbf{x}^{N+1}) = [a^{N+1}/(N+1)!]\exp[-U(\mathbf{x}^{N+1})/k_B T]/Z . \quad (4.101)$$

Similarly, if the volume contains N particles and we randomly destroy one, then the probability that the system is in a state with $N-1$ particles is

$$P(\mathbf{x}^{N-1}) = [a^{N-1}/(N-1)!]\exp[-U(\mathbf{x}^{N-1})/k_B T]/Z . \quad (4.102)$$

In the spirit of Sect.4.3 we can write

$$W(\mathbf{x}^N, \mathbf{x}^{N+1}) = \min\{1, P(\mathbf{x}^{N+1})/P(\mathbf{x}^N)\} , \quad (4.103)$$

$$W(\mathbf{x}^N, \mathbf{x}^{N-1}) = \min\{1, P(\mathbf{x}^{N-1})/P(\mathbf{x}^N)\} , \quad (4.104)$$

for the transition probabilities of the creation and destruction, respectively. With these transition probabilities we can write down an algorithm for the grand ensemble MC method.

Algorithm A13. Grand Ensemble Monte-Carlo Method I

- 1) Assign an initial configuration with N particles inside the volume V .
- 2) Select randomly with equal probability one of the procedures MOVE, CREATE or DESTROY.
- 3) **MOVE**
 - 3.1 Select a particle inside the volume and displace it randomly.
 - 3.2 Compute the change in energy $\Delta U = U(\mathbf{x}') - U(\mathbf{x})$.
 - 3.3 If ΔU is negative, accept the configuration and return to Step 2.
 - 3.4 Compute $\exp(-\Delta U/k_B T)$.
 - 3.5 Generate a random number $R \in [0, 1]$.
 - 3.6 If R is less than $\exp(-\Delta U/k_B T)$, accept the configuration and return to Step 2.
 - 3.7 Otherwise, the old configuration is also the new one. Return to Step 2.
- 4) **CREATE**
 - 4.1 Choose randomly coordinates inside the volume for a new particle.
 - 4.2 Compute the energy change $\Delta U = U(\mathbf{x}^{N+1}) - U(\mathbf{x}^N)$.

- 4.3 Compute $[a/(N+1)]\exp(-\Delta U/k_B T)$.
- 4.4 If this number is greater than unity, accept the new configuration and return to Step 2.
- 4.5 Generate a random number $R \in [0, 1]$.
- 4.6 If R is less than $[a/(N+1)]\exp(-\Delta U/k_B T)$, accept the creation and return to Step 2.
- 4.7 Otherwise, reject the creation of a particle and return to Step 2.

5) DESTROY

- 5.1 Select randomly a particle out of the N particles in the volume.
- 5.2 Compute the energy change $\Delta U = U(\mathbf{x}^{N-1}) - U(\mathbf{x}^N)$.
- 5.3 Compute $(a/N)\exp(-\Delta U/k_B T)$.
- 5.4 If this number is greater than unity, accept the new configuration and return to Step 2.
- 5.5 Generate a random number $R \in [0, 1]$.
- 5.6 If R is less than $(a/N)\exp(-\Delta U/k_B T)$, accept the destruction and remove the particle from the volume.
- 5.7 Otherwise, reject the destruction and return to Step 2.

The convergence of the grand ensemble MC algorithm depends on the allowed changes in the coordinates. If the allowed changes are too small, many configurations are needed for the system to reach equilibrium, i.e., convergence is slow. On the other hand, larger changes in the coordinates may result in a high rejection rate and slow down the convergence.

For a given volume there is a natural cut-off of particles which can fit into V . Let M be the number of particles which can be packed into the volume V . Another possible way of introducing the fluctuations in the particle number necessary for a constant chemical potential is always to have M particles present in the computational volume [4.98-101]. Of the M particles, N are real and the other $M-N$ are virtual or ghost particles. If during the course of the Monte-Carlo process a particle is added, one of the virtual particles is made real. If a particle is removed it is made virtual.

To set down the idea quantitatively, we rewrite (4.98) such that the system contains M particles, of which N are real and $M-N$ are ghost particles [4.101]. For this purpose we rewrite the integral as

$$\begin{aligned} & \int_{V^N} A(\mathbf{x}^N) \exp[-U(\mathbf{x}^N, N)/k_B T] d\mathbf{x}^N \\ &= V^{N-M} \int_{V^M} A(\mathbf{x}^M) \exp[-U(\mathbf{x}^M, N)/k_B T] d\mathbf{x}^M \end{aligned} \quad (4.105)$$

and an observable A is calculated as

$$\langle A \rangle = \frac{1}{Z} \sum_N \frac{1}{N!} (aV)^N \int_{V^M} A(\mathbf{x}^M) \exp[-U(\mathbf{x}^M, N)/k_B T] d\mathbf{x}^M, \quad (4.106)$$

$$Z = \sum_N \frac{1}{N!} (aV)^N \int_{V^M} \exp[-U(\mathbf{x}^M, N)/k_B T] d\mathbf{x}^M.$$

Having reformulated the evaluation of observables, the algorithm to solve (4.106) numerically proceeds as outlined in Sect.4.3, with the exception of the creation and annihilation of particles.

Algorithm A14. Grand Ensemble Monte-Carlo Method II

- 1) Assign coordinates to M particles in a volume V .
- 2) Select N particles as real.
- 3) Select a particle.
- 4) Generate new coordinates for the selected particle.
- 5) Compute the change in energy $\Delta U = U(\mathbf{x}') - U(\mathbf{x})$.
- 6) If ΔU is negative, accept the configuration and proceed with Step 11.
- 7) Compute $\exp(-\Delta U/k_B T)$.
- 8) Generate a random number $R \in [0, 1]$.
- 9) If R is less than $\exp(-\Delta U/k_B T)$, accept the configuration and proceed with Step 11.
- 10) Otherwise, reject the new configuration and keep the old configuration.
- 11) Select a particle to be deleted from the set of real particles.
- 12) Compute the change in energy and $(N/aV)\exp(-\Delta U/k_B T)$.
- 13) Generate a random number $R \in [0, 1]$.
- 14) If R is less than $(N/aV)\exp(-\Delta U/k_B T)$ put the particle into the set of ghost particles and proceed with Step 16.
- 15) Otherwise, leave the particle real.
- 16) Select a particle to be made real from the set of ghost particles.
- 17) Compute the change in energy and $[aV/(N+1)]\exp(-\Delta U/k_B T)$.
- 18) Generate a random number $R \in [0, 1]$.
- 19) If R is less than $[aV/(N+1)]\exp(-\Delta U/k_B T)$ add the particle to the set of real particles and proceed with Step 4.
- 20) Otherwise, reject the creation and proceed with Step 4.

One advantage of the outlined algorithm is the reduction of the number of unsuccessful attempts to add a particle. Indeed, especially at high densities, there is little room if we try to add a particle at a random coordinate. The probability of successfully adding a particle diminishes exponentially fast with increasing density. A disadvantage seems to be that the algorithm needs a fair amount of computer time. This is due to the fact that one

always has to take into account the interaction between all M particles and not just the interaction between the N particles.

We mentioned in the preceding subsection that the algorithms presented here and previously are by no means the last word! For the grand ensemble MC method an interesting suggestion has recently been made [4.102, 103] which differs from the method here. There are also ways to circumvent the problem altogether (cf. Problems). In any case, one should exercise some caution when dealing with these kinds of ensembles.

Problems

- 4.1 Devise an algorithm to compute the mean square displacement

$$\langle x(t)^2 \rangle = \frac{1}{3N} \sum_j \langle [x_j(t) - x_j(0)]^2 \rangle$$

for N particles in a volume with periodic boundary conditions.

- 4.2 Write an algorithm for a system of N uncoupled Brownian particles.
4.3 Show that the symmetrical choice of transition probabilities

$$W(\mathbf{x}, \mathbf{x}') = \begin{cases} 0 & \text{if } \mathbf{x} = \mathbf{x}' \\ \omega_{\mathbf{x}\mathbf{x}'} P(\mathbf{x}') / (P(\mathbf{x}') + P(\mathbf{x})) & \text{otherwise} \end{cases}$$

satisfies the Restrictions 4.11ii-iv.

- 4.4 Show that for the Ising model the temperature can be computed from the demon energy E_D as

$$1/k_B T = (1/4J) \ln(1 + 4J/\langle E_D \rangle).$$

Show also that if the magnetic field is non-zero then

$$1/k_B T = (1/2h) \ln \left[1 + \frac{2h}{\langle E_D \rangle} \right].$$

- 4.5 Show that the Metropolis function and the Glauber function are mathematically equivalent.
4.6 Show that the Metropolis and the Glauber functions are limiting cases of a transition probability with one more parameter z (taking $\tau' = 2\tau$)

$$W_z(s_i) = W_{\text{Glauber}} \left[1 - \sinh^2 \left[\frac{E_i}{k_B T} \right] \left(\frac{\tanh(E_i/zk_B T)}{\tanh(E_i/k_B T)} - 1 \right) \right].$$

- 4.7 The Ising model may also be simulated with a conserved order parameter. This is the so-called **Kawasaki dynamics** [4.37]. Instead of flip-

ping a spin, two unequal nearest neighbours are selected and exchanged if a comparison of the drawn random number and the transition probability permits this. Modify the program PL4 in Appendix A2 for the conserved order-parameter case.

- 4.8 Adapt the program given in Appendix A2 for the three-dimensional single-spin-flip Ising model to two dimensions. [The exchange coupling in two dimensions is $J/k_B T_c = \frac{1}{2} \ln(1+\sqrt{2})$.]
- 4.9 Finite-size effects are an important consideration in simulations. For many types of problems large systems are required. Invent an algorithm for the single-spin-flip Ising model which minimizes the required storage for a simple square lattice.
- 4.10 Rewrite the grand ensemble Monte-Carlo method with symmetrical transition probabilities.
- 4.11 **Self-Avoiding Random Walk.** The self-avoiding random walk is a random walk where the walker does not cross its own path. At each step the walker checks if the neighbouring sites have been visited before. Of course, the walker is not allowed to retrace its steps. Quite often the walker encounters a situation where all the sites in the immediate neighbourhood have been visited before. The walk then terminates. Write a program which shows on a screen how a self-avoiding random walk proceeds across a two-dimensional grid.
- 4.12 An obvious extension to the Creutz algorithm is to introduce more than one demon, i.e., allow more degrees of freedom. Can you use this to vectorize or parallelize the Creutz algorithm?
- 4.13 In the limit of one demon per lattice site the Creutz algorithm crosses over to the usual Metropolis Monte-Carlo algorithm. What are the pitfalls?
- 4.14 **Q2R Ising Model.** Beside the Creutz idea of performing simulations at constant energy, one can do simulations without introducing an extra degree of freedom. Take the two-dimensional Ising model. Each spin is surrounded by four nearest neighbours. Suppose the sum of the nearest neighbour spins of spin i is zero, $0 = \sum_{nn(ij)} s_j$. In this case the energy is unaffected by a reversal of the central spin i . Starting from a configuration at the desired energy sweep through the lattice and reverse all spins where the energy is left invariant. Consider the ergodicity of the algorithm. How must one perform the sweeps? Is the algorithm ergodic at all?
- 4.15 Show that for the Q2R one is able to obtain the temperature, i.e., $\exp(-\beta J)$ by sampling.
- 4.16 Can you design an algorithm where several spins are coded into the same computer word, the decision and updating are done using logical operators? [4.104].
- 4.17 **Cellular Automata.** The above exercise concerns a special cellular automaton. Consider a lattice. At each lattice site there is an automaton with a given set of states $S = \{s_1, \dots, s_n\}$. The states are changed by a set of rules $R = \{r_1, \dots, r_m\}$. The rules usually depend on the states of the neighbouring automata.

- 4.18 **Ergodicity of Cellular Automata.** Cellular automata can be updated synchronously and asynchronously. Consider the ergodicity and develop a criterion [4.105].
- 4.19 **Kauffman Model.** There is an interesting class of cellular automata which is intended to simulate some biological features. The Kauffman model [4.106] is a random Boolean cellular automaton. The states of one cell can be either one or zero. In two dimensions there are four nearest neighbour cells.
- 4.20 **Helical Boundary Conditions.** Consider a simple two-dimensional lattice $L(i, j)$ with i and j ranging from 0 to $n-1$. For helical boundary conditions we make the identification

$$\begin{aligned} L(i, n) &= L(i+1, 0) \\ L(n, j) &= L(0, j+1) . \end{aligned}$$

Are the finite size effects influenced by the choice of boundary conditions? Can you give analytical arguments? Perform simulations with free, periodic, and helical boundary conditions for the Ising model and compare the results for the order parameter and the susceptibility. Which of your algorithms is the fastest?

- 4.21 Program the Swendsen-Wang algorithm for the two-dimensional Ising model (you will need a cluster identification algorithm). The magnetization and the susceptibility can be obtained from the cluster size distribution. Are the size effects the same as for the usual Metropolis algorithm [4.91]?
- 4.22 Derive along the lines given in the Example 4.5 the bond probability p for the Ising model without a magnetic field. Show that the probabilities derived for the case with a magnetic field reduce to the one in zero field.
- 4.23 **Block Distribution Function.** There exists another way of introducing fluctuations in the number of particles to calculate such quantities as the isothermal compressibility [4.107]. Imagine the computational box partitioned into small boxes. Let the linear box size be S , i.e., $n = L/S$ is the number of boxes into which the computational volume has been split. In each of the boxes of sides of length S we find in general a different number of particles. For a fixed overall particle number and fixed block size we can construct the probability function $P_s(N)$ giving the probability of finding N particles inside the box of volume S^d . How do you compute the isothermal compressibility from the probability function $P_s(N)$?

4.24 Heat-Bath Algorithm. Once again consider the Ising model. The heat bath algorithm to simulate the model consists of selecting the new spin orientation independent of the old one by setting

$$s_i = \text{sign}\{p_i - r\}$$

where r is a random number between 0 and 1 and

$$p_i = \frac{e^{-\beta \Sigma_j s_j}}{1 + e^{-\beta \Sigma_j s_j}}.$$

Is there a difference between the Glauber probabilities and the heat bath probabilities?

Appendix

A1. Random Number Generators

Some computer simulation methods such as Brownian dynamics or Monte-Carlo methods rely, due to their stochastic nature, on random numbers. The quality of the results produced by such simulations depends on the quality of the random numbers. An easy demonstration is the following "experiment". Suppose we have to visit the sites of a simple cubic lattice L^3 at random. The three coordinates are obtained from three successively generated random numbers $r_1, r_2, r_3 \in (0, 1)$ as

$$\begin{aligned} ix &= r_1 * L + 1, \\ iy &= r_2 * L + 1, \\ iz &= r_3 * L + 1, \end{aligned}$$

where ix, iy, iz are integer variables, implying a conversion of the real right-hand sides to integers, i.e., removal of the fractional part. If there are no correlations between successively generated random numbers all sites will eventually be visited. However, only certain hyperplanes are visited if correlations exist. This was most impressively demonstrated first by *Lach* [A1.1] in 1962.

Another manifestation of the influence of the quality of the random numbers on simulation results are the findings of the Santa Barbara group [A1.2]. In their Monte-Carlo results (obtained with a special purpose computer [A1.3]) a peculiar behaviour for a certain system size was found. Results from an independent group [A1.4] did not show the observed peculiarity and it was concluded that the Random Number Generator (RNG) had caused the phenomenon.

For application in simulational physics the generator must have several features:

- 1) good statistical properties,
- 2) efficiency,
- 3) long period, and
- 4) reproducibility.

The most important desired properties are, of course, the statistical properties. Unfortunately there is no generator available without fault, but the list of known properties is growing. This reduces the risk of applying a particular bad one in a simulation.

Computational efficiency is extremely important. Now simulation programs require huge numbers of random numbers. To consume 10^{10} numbers is not outrageous anymore, so the computation of a single random number must be very fast. But the storage requirement is also important. The generator should be very fast in time and economical in space.

Actually it is not random numbers that are generated but sequences of *pseudo-random numbers*. Almost all generators create the pseudo-random numbers by recursion relations using the modulo function. Hence, the sequence repeats itself after a certain number of steps. The period must be such that it allows for at least the required number of random numbers. However, it is dangerous to exhaust or even come near the cycle. Coming close to the cycle produces fallacious results, as seen for example by *Margolina et al.* [A1.5].

There exist many ways of generating random numbers on a computer. A host of methods can be found in the book by *Ahrens and Dieter* [A1.6] (also *Knuth* [A1.7]). In the following we focus first on creating *uniformly distributed numbers*.

The most popular and prevalent generators in use today are *linear congruential generators*, or *modulo generators* for short. They produce in a deterministic way from an initial integer, a sequence of integers that appears to be random. The advantage, of course, is their reproducibility. This may seem a paradox, but consider that an algorithm must be tested. If the generator were to produce different sequences for each test run, the effect of changes in the algorithm would be difficult to assess.

Although the modulo generator produces integer sequences, these can always be normalized to the unit interval $[0, 1]$. One simply divides the generated number by the largest possible. The integer sequence is, however, to be preferred for reasons of computational efficiency. In many cases a simple transformation changes an algorithm requiring random numbers from the interval $[0, 1]$ into one using integers.

Algorithm A1.1. Modulo Generator (Lehmer, 1948)

Let m, a, c, x_0 be integers with $m > x_0$, $m > a$ and $x_0 > 0$, $a > 0$, $c > 0$. The pseudo-random number x_i of the sequence (x_i) is obtained from its predecessor x_{i-1} by

$$x_i = ax_{i-1} + c \pmod{m} . \quad (\text{A1.1})$$

With appropriate choices for the integers m, a, c, x_0 , Algorithm A1.1 produces a sequence of integers which appear random [A1.8]. Clearly, the sequence has a repeating *cycle* or *period* not larger than m . As explained, the period should be as large as possible. The maximum period is achieved with $c \neq 0$. Such a generator is called *mixed*. But, mixed generators are known to give poor results [A1.9], and we concentrate on *multiplicative* generators where $c = 0$. This has the additional advantage of a reduction in computational overhead. By varying the *seed* x_0 , different sequences of the basic sequence are realized, as needed for independent runs.

To guarantee a maximum period for the multiplicative modulo generator, the modulus m and the multiplier have to be chosen carefully. Under the conditions of the following theorem a single periodic sequence of length $m-1$ is obtained, hence only one less than is possible with the mixed generator [A.10].

Theorem A1. (Carmichael, 1910)

The multiplicative modulo generator has the period $m-1$ if the following conditions hold:

- (i) m is prime;
- (ii) a is a positive primitive root of m , i.e.,
 $a^m = 1 \pmod{m}$ and for all $n < m$ $a^n \neq 1 \pmod{m}$.

□

Condition (i) is necessary because if d is a divisor of m and if x_i is a multiple of d then all successors are multiples of d . The condition (i) cannot always be met, but see below.

The introduced generator is easy to implement on a computer. Using, in its straightforward form, the intrinsic modulo function, the algorithm reads

P1: $ix = \text{MOD}(a * ix, m)$

The MOD function has the advantage of transportability of the program from one machine to another. Its disadvantage is the computational inefficiency. Many operations are carried out, among them the most time-consuming, division. A more efficient and convenient implementation is as follows. Suppose the computer word is composed of n bits. One bit is used to indicate the sign. Then $2^{n-1}-1$ is the largest integer. Suppose further that $2^{n-1}-1$ is a prime. For example, let $n = 32$; $2^{31}-1 = 2147483647$ is prime (Mersenne number)! Choose $m = 2^{n-1}-1$ and an appropriate multiplier a . A random number is obtained from its predecessor by the following FORTRAN statements

P2: $ix = ix * a$
 if $(ix .lt. 0)$ $ix = ix + m + 1$

The first statement is equivalent to the modulo operation and the second reverses a possible overflow. The procedure is only feasible if the run-time system of the computer does not detect or ignores overflows.

If a is a primitive root of $m = 2^{31}-1$ the cycle is $2^{31}-2$. This may seem large but consider the following. Suppose each site of a lattice of size 300^3 (a lattice of size 600^3 has been simulated [A1.11]) is visited at random. Hence $3 \cdot 300^3$ random numbers are drawn. If we repeat this only 25 times then almost all numbers of the cycle are exhausted!

That $2^{31}-1$ is a Mersenne number is somewhat fortunate for 32 bit machines. A list of prime factorization [A1.7] shows that for all n ($32 < n < 65$) 2^n-1 is not prime. The theorem of Carmichael is not applicable for ma-

chines utilizing more than 32 bits to represent integers. In such a case the period attainable is less than the maximum. On some machines integers are represented by 49 bits. A popular choice in this situation is $m = 2^{48}$, $a = 11^{13}$ [A1.12]. We shall not embark on the choice for machines with a word size of 64 bits since below we introduce a generator especially suited to 64 bit vector machines.

In practice, the modulo generator must be "warmed up". Before starting to use the sequence, an initial portion must be discarded. Yet another problem is the seed. It has been recognized that there are bad choices for the seed. A safe way to avoid a bad choice is to use the last pseudo-random number from a previous run as a new seed. This also decreases the likelihood that overlapping portions of the basic sequence will be generated in successive runs. Care must be taken that the cycle length is not exhausted.

We return to the appropriate choice of the multiplier. It can be shown [A1.13] that the correlation between successive numbers will be approximately the reciprocal of the multiplying factor. In addition, the factor should be considerable less than the square root of the modulus [A1.14]. Using the modulus $2^{31}-1$ the most popular multipliers are $a = 65539$ or 16807 . However, $a = 65539$ is a bad choice [A1.15]. It does not meet the square root criterion and indeed shows triplet correlations. The choice $a = 16807$ appears to be better. It meets almost all criteria, but in general modulo generators have a built-in *d-space non-uniformity*. If successive d-tuples from a multiplicative congruential generator are taken as coordinates in a d-dimensional space, as is often done in applications, all points lie on a certain finite number of hyperplanes [A1.16,17]. The number of hyperplanes is always not greater than a certain function of d and the bit length of the integer arithmetic, implying that some bits are highly correlated.

Another method has been proposed by Tausworth [A1.18]. Its advantages are its long period, computational efficiency and inherent parallelism, making it especially suitable for use on vector machines. In addition, the algorithm produces not only a pseudo-random sequence of integers but also pseudo-random bit sequences. Similar to the modulo random number generator, the algorithm computes the new random number from its predecessors.

Algorithm A1.2. Feedback Shift Register RNG (Tausworth, 1965)

Let c_i ($i = 1, n$) be a set of integers having values 0 or 1; c_n is required to have the value one. The sequence of pseudo-random numbers $x = (x_k)$ is generated by the linear recursion relation

$$x_k = c_1 x_{k-1} + c_2 x_{k-2} + \dots + c_n x_{k-n} \pmod{2}. \quad (\text{A1.2})$$

The algorithm is a generalization of the modulo generator A1.1. For fixed c_i the new value x_k is determined by the predecessors $x_{k-1}, x_{k-2}, \dots, x_{k-n}$ and only those. Each such n-tuple has a unique successor. Like the modulo generator, the above linear recurring sequence has a period after which an n-tuple repeats. The period cannot be greater than $2^n - 1$. For

the period to be exactly equal to $2^n - 1$ it is necessary and sufficient that

$$f(x) = 1 + c_1 + c_2 x^2 + \dots + x^n$$

is primitive over $GF(2)$. The most exhaustive list of primitive polynomials over $GF(2)$ has been compiled for trinomials

$$f(x) = x^p + x^q + 1, \quad (A1.3)$$

where p is a Mersenne exponent ($2^p - 1$ is prime) and $p > q > 0$ [A1.19]. But also primitive trinomials are known where p is not a Mersenne number. In applications, such a trinomial (250, 108) has found widespread use [A1.20]. On vector machines, however, larger p values are more efficient [A1.21], for example (532, 37). Notice that the period is independent of the word size of the host machine.

We shall now outline an algorithm for the feedback shift register generator, as proposed by *Lewis* and *Payne* [A1.22], with the modifications of *Kirkpatrick* and *Stoll* [A1.20]. Addition in the Galois field $GF(2)$ is equivalent to performing an EXCLUSIVE-OR (we denote the EXCLUSIVE-OR operator by $\oplus$). For trinomials $x^p + x^q + 1$ the linear recursion (A1.2) reduces to

$$x_k = x_{k-q} \oplus x_{k-p}. \quad (A1.4)$$

Let (x_i) be the sequence of n -bit integers, which we imagine as n columns of random bits. For trinomials only one EXCLUSIVE-OR operation is needed with some address manipulations to generate a new number from the predecessors. To start the generator, a p -tuple $(x_1, \dots, x_p)$ is necessary. Such a p -tuple can be obtained by generating p pseudo-random numbers with the modulo generator. These numbers must be linearly independent for a cycle length of $2^p - 1$. To ensure the linear independence the following is used. Let s be the number of bits in the mantissa. Choose s distinct numbers from p initial random numbers. Imagine them arranged in an $s \times s$ matrix. Set the diagonal elements to "1" and the lower triangle to "0" [A1.20].

An adaption of the algorithm for use on CRAY vector machines is shown in the program listing L1. Calling the subroutine INR250 creates the starting values. Subroutine R250 is the actual generator. It returns the random numbers $x_i \in (0, 1)$.

Program Listing L1. Adaptation of the algorithm A1.2 proposed by *Tausworth* to generate pseudo-random numbers. The program is due to *Groote* [A1.21] and tailored for use on CRAY vector machines.

```
C      +----- USE OPT=BTREG IN THE CFT COMMAND (CFT 13 AND HIGHER) -----
C      : FILL THE 250 STARTING VALUES INTO M (KIRKPATRICK-STOLL-GREENWOOD).
C      : CRAY STANDARD RANDOM NUMBER GENERATOR IS USED TO CREATE 250 NUMBERS.
C      : NUMBERS ARE MODIFIED TO ASSURE 47 OF THEM TO BE LINEAR INDEPENDENT.
C      : THE LAST BIT IS SET TO 0 (BECAUSE R250 CAN BE FASTER WITH THIS).
C      +-----
```

C + -----

```

C      : THE M'S ARE EVEN INTEGER RANDOM NUMBERS 0<=M<2**48
C      : IFL IS ADDED TO CREATE A NOT NORMALIZED FLOATING POINT NUMBER
C      : 0<X<1 SHIFTR IS APPLIED ONLY TO HAVE IFL*M(K,1) TYPE BOOLEAN.
C      : IT WILL CHAIN. EP IS DUMMY ADDED TO NORMALIZE THE CONSTRUCTED
C      : REAL NUMBER.
C      +-----+
C      M(K,1)=XOR(M(K,1),M(K+103,1))
C      I=I+1
10    X(I)=SHIFTR(IFL*M(K,1),0) + EP
C      +-----+
C      DO 20 K=K,MIN(L,250)
C      M(K-147,2)=XOR(M(K-147,2),M(K-147,1))
C      I=I+1
20    X(I)=SHIFTR(IFL*M(K-147,2),0) + EP
C      +-----+
C      IF (K.EQ.251) THEN
C      K=1
C      GOTO 5
C      ENDIF
C      +-----+
C      : END ROTATIONAL LOOP
C      +-----+
C      KK=K
C      END

```

The d-space non-uniformity is reduced for feedback shift register generators [A1.6,22]. It is even claimed to be uniform in d-space [A1.18]. But bad choices of tuples (p,q) exist [A1.23]. As a criterion, q should be small and not near (p-1)/2.

Sometimes computer simulations require more involved distributions than the uniform one. In Sect.4.2 we encountered the problem of drawing random variates from a normal or Gaussian distribution. Sampling such a distribution is computationally rather inefficient compared to the algorithms described above. But in the applications described in this book, the generation of a Gaussian distribution is not time critical. For example, in molecular dynamics one only needs to generate 3N numbers once and in Brownian dynamics $n \cdot 3N$, where n is the number of replacements of the velocities. Compared with the force evaluations this constitutes a marginal part of the execution time required for a simulation.

Generally algorithms generating non-uniform variates do so by converting uniform variates. In its most straightforward form a normal deviate x with mean $\langle x \rangle$ and standard deviation σ is produced as follows [A1.24,25].

Algorithm A1.3

Let n be an integer, determined by the needed accuracy. Then

- 1) sum n uniform random numbers from the interval (-1,1),
- 2) $x = \langle x \rangle + \sigma * \text{sum} * \text{SQRT}(3.0/n)$.

For some purposes the simple method will be sufficient, but if good accuracy is needed the above algorithm should be avoided. More efficient and accurate is the idea of *von Neumann* [A1.26] with the modification of *Forsythe* [A1.27]

Algorithm A1.4. (von Neumann, 1963; Forsythe, 1972)

Let $G(x)$ be a function on the interval $[a, b]$ with $0 < G(x) < 1$ and $f(x)$ the probability distribution

$$f(x) = \alpha \exp[-G(x)] ,$$

where α is a constant.

- 1) Generate r from uniform-(0,1).
- 2) Set $x = a + (b-a) * r$.
- 3) Calculate $t = G(x)$.
- 4) Generate $r_1, r_2, \dots, r_k$ from uniform-(0,1) where k is determined from the condition $t > r_1 > r_2 > \dots > r_{k-1} < r_k$. If $t < r_1$, then $k = 1$.
- 5) If k is even, reject x and go to 1; otherwise, x is a sample.

Again, the method converts uniform random numbers into non-uniform ones. Figure A1.1 shows the distribution resulting from the function $G(x) = x^2$ and $-1 < x < 1$. The top part of Fig.A1.1 displays the result from 10 000 drawings and the bottom part for 100 000. We shall not dwell further on this algorithm, and direct the reader's attention to the book by *Ahrens* and *Dieter* [A1.6]. A fast adaptation of the algorithm can be found in [A1.28].

An interesting method of generating normal variates, which was actually used in the examples, is the *polar method* [A1.29]. It has the advantage that two independent, normally distributed variates are produced with practically no additional cost in computer time.

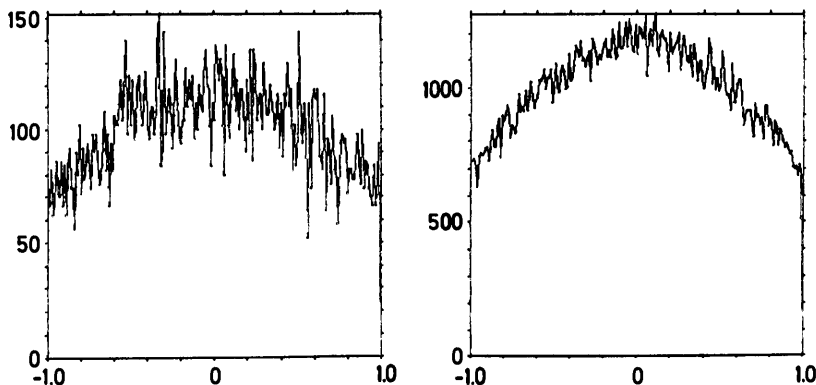


Fig.A1.1. Distribution generated by the method of v. Neumann. The left figure shows the distribution after 10^4 drawings and the right after 10^5

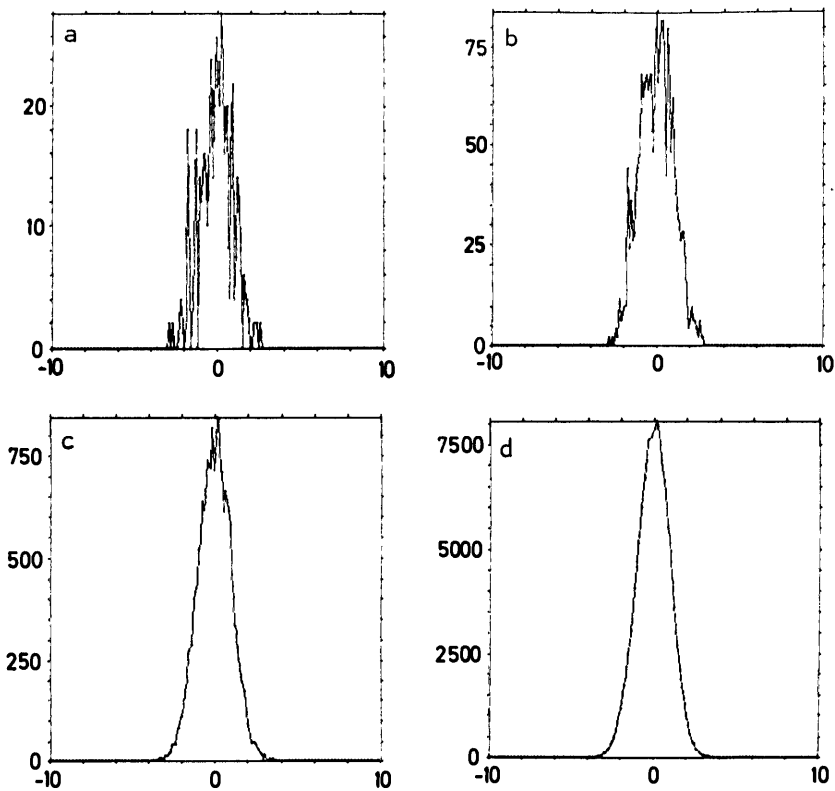


Fig.A1.2a-d. Resulting distributions using the polar method. The figures show the result after 256, 10^3 , 10^4 and 10^5 drawings (a-d)

Algorithm A1.5. (Polar Method)

- 1) Generate two independent random variables, U_1, U_2 from the interval $(0, 1)$.
- 2) Set $V_1 = 2U_1 - 1$, $V_2 = 2U_2 - 1$.
- 3) Compute $S = V_1^2 + V_2^2$.
- 4) If $S \geq 1$, return to Step 1.
- 5) Otherwise, set

$$X_1 = V_1 * \text{SQRT}\{(-2\text{LNS})/S\},$$

$$X_2 = V_2 * \text{SQRT}\{(-2\text{LNS})/S\}.$$

The results using the polar method are shown in Fig.A1.2. Even for a small number of samples ($N = 256$) the mean is very close to zero (0.01) and the standard deviation close to unity (0.91).

Algorithms A1.4 and A1.5 generate normally distributed numbers with a mean zero and a unit standard deviation. Sometimes a non-zero mean and a non-unit standard deviation are required. This is achieved by the following transformation. If X is distributed with mean zero and standard deviation one then the transformation

$$Y = \mu + \sigma X$$

yields a normally distributed variable with mean μ and standard deviation σ .

A2. Program Listings

Here we summarize the source listings of programs used in the examples in various chapters. It is hoped that they provide a first step toward developing one's own programs and a stimulus for own ideas in a rapidly expanding and exciting new field. Emphasis was not always put on the most efficient algorithm and code. Rather, some programs were deliberately kept simple so as to facilitate an understanding of the basic ideas presented in the text. The programs are hence not exact copies of those actually used. The reader interested in learning about simulation should try to run some of the supplied programs to get acquainted with the workings of the algorithms and to develop a feeling for the requirements of a simulation.

Each program is headed by a brief description of the algorithm used in the code. Included in the listing are additional comments and explanations of the parameters. Some programs may require adaptation for different host machines.

Program Listing PL1. Microcanonical Molecular-Dynamics Program

```

C-----
C
C      M O L E C U L A R   D Y N A M I C S
C
C      MICROCANONICAL ENSEMBLE
C
C      APPLICATION TO ARGON. THE LENNARD-JONES POTENTIAL IS TRUNCATED
C      AT RCOFF AND NOT SMOOTHLY CONTINUED TO ZERO. INITIALLY THE
C      NPART PARTICLES ARE PLACED ON AN FCC LATTICE. THE VELOCITIES
C      ARE DRAWN FROM A BOLTZMANN DISTRIBUTION WITH TEMPERATURE TREF.
C
C      INPUT PARAMETERS ARE AS FOLLOWS
C
C      NPART      NUMBER OF PARTICLES (MUST BE A MULTIPLE OF 4)
C      SIDE      SIDE LENGTH OF THE CUBICAL BOX IN SIGMA UNITS
C      TREF      REDUCED TEMPERATURE
C      RCOFF     CUTOFF OF THE POTENTIAL IN SIGMA UNITS
C      H         BASIC TIME STEP
C      IREP      VELOCITIES SCALING EVERY IREP'TH TIME STEP
C      ISTOP     STOP OF SCALING OF THE VELOCITIES AT ISTOP
C      TIMEMX    NUMBER OF INTEGRATION STEPS
C      ISEED     SEED FOR THE RANDOM NUMBER GENERATOR
C
C      VERSION 1.0 AS OF AUGUST 1985
C

```

```

C          DIETER W. HEERMANN
C
C-----
C
C      REAL      X(1:786),VH(1:786),F(1:786)
C
C
C      REAL      FY(1:256),FZ(1:256),Y(1:256),Z(1:256)
C
C      REAL      H,HSQ,HSQ2,TREF
C      REAL      KX,KY,KZ
C
C      INTEGER   CLOCK,TIMEMX
C
C      EQUIVALENCE (FY,F(257)),(FZ,F(513)),(Y,X(257)),(Z,X(513))
C
C-----
C
C      DEFINITION OF THE SIMULATION PARAMETERS
C
C-----
C
C          NPART   = 256
C          DEN      = 0.83134
C          SIDE     = 6.75284
C          TREF     = 0.722
C          RCOFF    = 2.5
C          H        = 0.064
C          IREP     = 50
C          ISTOP    = 500
C          TIMEMX   = 3
C          ISEED    = 4711
C
C-----
C
C      SET THE OTHER PARAMETERS
C
C-----
C
C      WRITE(6,*) 'MOLECULAR DYNAMICS SIMULATION PROGRAM'
C      WRITE(6,*) '-----'
C      WRITE(6,*)
C      WRITE(6,*) 'NUMBER OF PARTICLES IS      ',NPART
C      WRITE(6,*) 'SIDE LENGTH OF THE BOX IS ',SIDE
C      WRITE(6,*) 'CUT OFF IS                  ',RCOFF
C      WRITE(6,*) 'REDUCED TEMPERATURE IS     ',TREF
C      WRITE(6,*) 'BASIC TIME STEP IS         ',H
C      WRITE(6,*)
C
C

```

```

      A      = SIDE / 4.0
      SIDEH  = SIDE * 0.5
      HSQ    = H * H
      HSQ2   = HSQ * 0.5
      NPARTM = NPART - 1
      RCOFFS = RCOFF * RCOFF
      TSCALE = 16.0 / (1.0 * NPART - 1.0)
      VAVER  = 1.13 * SQRT( TREF / 24.0 )

C
      N3     = 3 * NPART
      IOF1   = NPART
      IOF2   = 2 * NPART

C
      CALL RANSET( ISEED )

C
C
C-----
C
C      THIS PART OF THE PROGRAM PREPARES THE INITIAL CONFIGURATION
C
C-----
C
C      SET UP FCC LATTICE FOR THE ATOMS INSIDE THE BOX
C      =====
C
      IJK = 0
      DO 10 LG=0,1
      DO 10 I=0,3
      DO 10 J=0,3
      DO 10 K=0,3
      IJK = IJK + 1
      X(IJK) = I * A + LG * A * 0.5
      Y(IJK) = J * A + LG * A * 0.5
      Z(IJK) = K * A
10  CONTINUE
      DO 15 LG=1,2
      DO 15 I=0,3
      DO 15 J=0,3
      DO 15 K=0,3
      IJK = IJK + 1
      X(IJK) = I * A + (2-LG) * A * 0.5
      Y(IJK) = J * A + (LG-1) * A * 0.5
      Z(IJK) = K * A + A * 0.5
15  CONTINUE

C
C      ASSIGN VELOCITIES DISTRIBUTED NORMALLY
C      =====
C
      CALL MXWELL(VH,N3,H,TREF)

C

```

```

      DO 50 I=1,N3
        F(I) = 0.0
50    CONTINUE
C
C-----
C
C    START OF THE ACTUAL MOLECULAR DYNAMICS PROGRAM.
C
C    THE EQUATIONS OF MOTION ARE INTEGRATED USING THE 'SUMMED FORM'
C    (G. DAHLQUIST AND A. BJOERK, NUMERICAL METHODS, PRENTICE HALL
C    (1974)). EVERY IREP'TH STEP THE VELOCITIES ARE RESCALED SO AS
C    TO GIVE THE SPECIFIED TEMPERATURE TREF.
C
C    VERSION 1.0  AUGUST 1985
C    DIETER W. HEERMANN
C-----
C
      DO 200 CLOCK=1,TIMEMX
C
C          ===ADVANCE POSITIONS ONE BASIC TIME STEP===
      DO 210 I=1,N3
        X(I) = X(I) + VH(I) + F(I)
210    CONTINUE
C
C          ===APPLY PERIODIC BOUNDARY CONDITIONS===
      DO 215 I=1,N3
        IF ( X(I) .LT. 0 )    X(I) = X(I) + SIDE
        IF ( X(I) .GT. SIDE ) X(I) = X(I) - SIDE
215    CONTINUE
C
C          ===COMPUTE THE PARTIAL VELOCITIES===
      DO 220 I=1,N3
        VH(I) = VH(I) + F(I)
220    CONTINUE
C
C          =====
C          =
C          =      THIS PART COMPUTES THE FORCES ON THE PARTICLES      =
C          =
C          =====
C
      VIR = 0.0
      EPOT = 0.0
      DO 225 I=1,N3
        F(I) = 0.0
225    CONTINUE
C
C
      DO 270 I=1,NPART

```

```

      XI = X(I)
      YI = Y(I)
      ZI = Z(I)
      DO 270 J=I+1,NPART
        XX = XI - X(J)
        YY = YI - Y(J)
        ZZ = ZI - Z(J)

C
        IF ( XX .LT. -SIDEH ) XX = XX + SIDE
        IF ( XX .GT.  SIDEH ) XX = XX - SIDE

C
        IF ( YY .LT. -SIDEH ) YY = YY + SIDE
        IF ( YY .GT.  SIDEH ) YY = YY - SIDE

C
        IF ( ZZ .LT. -SIDEH ) ZZ = ZZ + SIDE
        IF ( ZZ .GT.  SIDEH ) ZZ = ZZ - SIDE
        RD = XX * XX + YY * YY + ZZ * ZZ
        IF ( RD .GT. RCOFFS ) GOTO 270

C
        EPOT = EPOT + RD ** (-6.0) - RD ** (-3.0)
        R148 = RD ** (-7.0) - 0.5 * RD ** (-4.0)
        VIR = VIR - RD * R148
        KX = XX * R148
        F(I) = F(I) + KX
        F(J) = F(J) - KX
        KY = YY * R148
        FY(I) = FY(I) + KY
        FY(J) = FY(J) - KY
        KZ = ZZ * R148
        FZ(I) = FZ(I) + KZ
        FZ(J) = FZ(J) - KZ

270      CONTINUE
      DO 275 I=1,N3
        F(I) = F(I) * HSQ2

275      CONTINUE

C
C
C      =====
C      =
C      =          END OF THE FORCE CALCULATION
C      =
C      =====
C
C      ---COMPUTE THE VELOCITIES---
      DO 300 I=1,N3
        VH(I) = VH(I) + F(I)

300      CONTINUE

C
C      ---COMPUTE THE KINETIC ENERGY---
      EKin = 0.0
      DO 305 I=1,N3

```

```

      EKin = EKin + Vh(I) * Vh(I)
305  CONTINUE
      EKin = EKin / Hsq
C
C      ---COMPUTE THE AVERAGE VELOCITY---
      VEL = 0.0
      COUNT = 0.0
      DO 306 I=1,NPART
        VX = Vh(I) * Vh(I)
        VY = Vh(I+IOF1) * Vh(I+IOF1)
        VZ = Vh(I+IOF2) * Vh(I+IOF2)
        SQ = SQRT( VX + VY + VZ )
        SQT = SQ / H
        IF ( SQT .GT. VAVR ) COUNT = COUNT + 1
        VEL = VEL + SQ
306  CONTINUE
      VEL = VEL / H
C
      IF ( CLOCK .LT. ISTOP ) THEN
C      ---NORMALIZE THE VELOCITIES TO OBTAIN THE---
C      ---SPECIFIED REFERENCE TEMPERATURE ---
        IF ( (CLOCK/IREP)*IREP .EQ. CLOCK) THEN
C          WRITE(6,*) 'VELOCITY ADJUSTMENT'
          TS = TSCALE * EKin
C          WRITE(6,*) 'TEMPERATURE BEFORE SCALING IS ',TS
          SC = TREF / TS
          SC = SQRT( SC )
C          WRITE(6,*) 'SCALE FACTOR IS ',SC
          DO 310 I=1,N3
            Vh(I) = Vh(I) * SC
310        CONTINUE
          EKin = TREF / TSCALE
        END IF
      END IF
C
C      =====
C      =
C      =          COMPUTE VARIOUS QUANTITIES          =
C      =
C      =====
C
C
C
C      IF ( (CLOCK/2)*2 .EQ. CLOCK ) THEN
        EK = 24.0 * EKin
        EPOT = 4.0 * EPOT
        ETOT = EK + EPOT
        TEMP = TSCALE * EKin
        PRES = DEN * 16.0 * ( EKin - VIR ) / NPART
        VEL = VEL / NPART
        RP = (COUNT / 256.0 ) * 100.0

```

```

      WRITE(6,6000) CLOCK,EK,EPOT,ETOT,TEMP,PRES,VEL,RP
C      END IF
C
C      200 CONTINUE
C      6000 FORMAT(1I6,7F15.6)
C
C      STOP
C      END
C
C-----
C
C      M X W E L L  RETURNS UPON CALLING A MAXWELL DISTRIBUTED DEVIATES
C      FOR THE SPECIFIED TEMPERATURE TREF. ALL DEVIATES ARE SCALED BY
C      A FACTOR.
C
C      CALLING PARAMETERS ARE AS FOLLOWS
C
C      VH      VECTOR OF LENGTH NPART ( MUST BE A MULTIPLE OF 2)
C      N3      VECTOR LENGTH
C      H      SCALING FACTOR WITH WHICH ALL ELEMENTS OF VH ARE
C      MULTIPLIED.
C      TREF    TEMPERATURE
C
C      VERSION 1.0 AS OF AUGUST 1985
C      DIETER W. HEERMANN
C
C-----
C
C      SUBROUTINE MXWELL(VH,N3,H,TREF)
C
C      REAL VH(1:N3)
C
C      REAL U1,U2,V1,V2,S,R
C
C      NPART   = N3 / 3
C      IOF1    = NPART
C      IOF2    = 2 * NPART
C      TSCALE  = 16.0 / (1.0 * NPART - 1.0)
C
C      DO 10  I=1,N3,2
C
C      1      U1 = RANF()
C      U2 = RANF()
C
C      V1 = 2.0 * U1 - 1.0
C      V2 = 2.0 * U2 - 1.0
C      S  = V1 * V1 + V2 * V2
C
C      IF ( S .GE. 1.0 ) GOTO 1
C      R      = -2.0 * ALOG(S) / S

```

```

      VH(I)   = V1 * SQRT( R )
      VH(I+1) = V2 * SQRT( R )
10  CONTINUE
C
      EKin = 0.0
      SP = 0.0
      DO 20 I=1,NPART
        SP = SP + VH(I)
20  CONTINUE
      SP = SP / NPART
      DO 21 I=1,NPART
        VH(I) = VH(I) - SP
        EKin = EKin + VH(I) * VH(I)
21  CONTINUE
      WRITE(6,*) 'TOTAL LINEAR MOMENTUM IN X DIRECTION IS ',SP
      SP = 0.0
      DO 22 I=IOF1 + 1,IOF2
        SP = SP + VH(I)
22  CONTINUE
      SP = SP / NPART
      DO 23 I=IOF1 + 1,IOF2
        VH(I) = VH(I) - SP
        EKin = EKin + VH(I) * VH(I)
23  CONTINUE
      WRITE(6,*) 'TOTAL LINEAR MOMENTUM IN Y DIRECTION IS ',SP
      SP = 0.0
      DO 24 I=IOF2 + 1,N3
        SP = SP + VH(I)
24  CONTINUE
      SP = SP / NPART
      DO 25 I=IOF2 + 1,N3
        VH(I) = VH(I) - SP
        EKin = EKin + VH(I) * VH(I)
25  CONTINUE
      WRITE(6,*) 'TOTAL LINEAR MOMENTUM IN Z DIRECTION IS ',SP
      WRITE(6,*) 'VELOCITY ADJUSTMENT'
      TS = TSCALE * EKin
      WRITE(6,*) 'TEMPERATURE BEFORE SCALING IS ',TS
      SC = TREF / TS
      SC = SQRT( SC )
      WRITE(6,*) 'SCALE FACTOR IS ',SC
      SC = SC * H
      DO 30 I=1,N3
        VH(I) = VH(I) * SC
30  CONTINUE
      END

```

The program simulates a Lennard-Jones system of N particles in a MD cell of volume $V = \sigma L^3$ at a fixed energy. All parameters are assumed to be in a scaled form. To advance the positions and to compute the velocities the

program uses the summed velocity form of the Verlet Algorithm A3. Initially the particles are placed on a face-centred-cubic lattice. This requires that the number of particles is a multiple of 4. The velocities are chosen from a Gaussian distribution. To bring the energy down to the desired value, or equivalently, to a mean temperature, the velocities are scaled periodically until the energy is within an acceptance interval. The system is not tampered with after the equilibration phase. The program can also be used for isokinetic molecular dynamics simulations by setting the parameter IREP equal to unity and ISTP larger than the number of MD steps.

Those interested in finding references for FORTRAN programs for molecular dynamics simulations should consult [A2.2] and references therein.

Program Listing PL2. Brownian Dynamics Program for Constant Temperature

```

C-----
C
C      B R O W N I A N   D Y N A M I C S
C
C      APPLICATION TO ARGON. THE LENNARD-JONES POTENTIAL IS TRUNCATED
C      AT RCOFF AND NOT SMOOTHLY CONTINUED TO ZERO. INITIALLY THE
C      NPART PARTICLES ARE PLACED ON AN FCC LATTICE. THE VELOCITIES
C      ARE DRAWN FROM A BOLTZMANN DISTRIBUTION WITH TEMPERATURE TREF.
C
C      INPUT PARAMETERS ARE AS FOLLOWS
C
C      NPART      NUMBER OF PARTICLES (MUST BE A MULTIPLE OF 4)
C      SIDE      SIDE LENGTH OF THE CUBICAL BOX IN SIGMA UNITS
C      TREF      REDUCED TEMPERATURE
C      DEN      REDUCED DENSITY
C      RCOFF     CUTOFF OF THE POTENTIAL IN SIGMA UNITS
C      H        BASIC TIME STEP
C      IREP      REPLACEMENT OF THE VELOCITIES EVERY IREP'TH
C               TIME STEP
C      TIMEMX    NUMBER OF INTEGRATION STEPS
C      ISEED     SEED FOR THE RANDOM NUMBER GENERATOR
C
C      VERSION 1.0 AS OF AUGUST 1985
C
C      DIETER W. HEERMANN
C-----
C
C      REAL      X(1:786),VH(1:786),F(1:786)
C
C      REAL      FY(1:256),FZ(1:256),Y(1:256),Z(1:256)
C
C      REAL      H,HSQ,HSQ2,TREF

```

```

      REAL      KX,KY,KZ
C
      INTEGER   CLOCK,TIMEMX
C
      EQUIVALENCE (FY,F(257)),(FZ,F(513)),(Y,X(257)),(Z,X(513))
C
C-----
C
      DEFINITION OF THE SIMULATION PARAMETERS
C
C-----
C
      NPART      = 256
      SIDE       = 6.75284
      TREF       = 0.722
      DEN        = 0.83134
      RCOFF      = 2.5
      H          = 0.064
      IREP       = 10
      TIMEMX     = 3
      ISEED      = 4711
C
C-----
C
      SET THE OTHER PARAMETERS
C
C-----
C
      WRITE(6,*) 'BROWNIAN DYNAMICS SIMULATION PROGRAM'
      WRITE(6,*) '-----'
      WRITE(6,*)
      WRITE(6,*) 'NUMBER OF PARTICLES IS      ',NPART
      WRITE(6,*) 'SIDE LENGTH OF THE BOX IS ',SIDE
      WRITE(6,*) 'CUT OFF IS                  ',RCOFF
      WRITE(6,*) 'REDUCED TEMPERATURE IS      ',TREF
      WRITE(6,*) 'BASIC TIME STEP IS         ',H
      WRITE(6,*)
C
      A          = SIDE / 4.0
      SIDEH      = SIDE * 0.5
      HSQ        = H * H
      HSQ2       = HSQ * 0.5
      NPARTM     = NPART - 1
      RCOFFS     = RCOFF * RCOFF
      TSCALE     = 16.0 / NPARTM
      VAVER      = 1.13 * SQRT( TREF / 24.0 )
C
      IOF1       = NPART
      IOF2       = 2 * NPART
      N3         = 3 * NPART

```

```

C
C      CALL RANSET( ISEED )
C
C
C-----
C
C      THIS PART OF THE PROGRAM PREPARES THE INITIAL CONFIGURATION
C
C-----
C
C      SET UP FCC LATTICE FOR THE ATOMS INSIDE THE BOX
C      =====
C
C      IJK = 0
C      DO 10 LG=0,1
C      DO 10 I=0,3
C      DO 10 J=0,3
C      DO 10 K=0,3
C      IJK = IJK + 1
C      X(IJK)      ) = I * A + LG * A * 0.5
C      Y(IJK      ) = J * A + LG * A * 0.5
C      Z(IJK      ) = K * A
10  CONTINUE
C      DO 15 LG=1,2
C      DO 15 I=0,3
C      DO 15 J=0,3
C      DO 15 K=0,3
C      IJK = IJK + 1
C      X(IJK      ) = I * A + (2-LG) * A * 0.5
C      Y(IJK      ) = J * A + (LG-1) * A * 0.5
C      Z(IJK      ) = K * A + A * 0.5
15  CONTINUE
C
C      ASSIGN BOLTZMANN DISTRIBUTED VELOCITIES AND CLEAR THE FORCES
C      =====
C
C      CALL MXWELL(VH,N3,H,TREF)
C
C      DO 20 I=1,N3
C      F(I) = 0.0
20  CONTINUE
C
C-----
C
C      START OF THE ACTUAL BROWNIAN DYNAMICS PROGRAM.
C
C      THE EQUATIONS OF MOTION ARE INTEGRATED USING THE 'SUMMED FORM'
C      (G.DAHLQUIST AND A. BJOERK, NUMERICAL METHODS, PRENTICE HALL
C      1974). THE STOCHASTIC PART IS AS FOLLOWS: AT REGULAR TIME
C      INTERVALS THE VELOCITIES ARE REPLACED BY VELOCITIES DRAWN FROM

```

```

C      A BOLTZMANN DISTRIBUTION AT THE SPECIFIED TEMPERATURE.
C      (H.C. ANDERSEN, J.CHEM. PHYS. 72, 2384 (1980) AND W.C. SWOPE,
C      H.C. ANDERSEN, P.H. BERENS AND K.R. WILSON, J. CHEM. PHYS. 76
C      637 (1982).
C
C      VERSION 1.0 AUGUST 1985
C      DIETER W. HEERMANN
C
C-----
C
C      DO 200 CLOCK=1,TIMEMX
C
C      ===ADVANCE POSITIONS ONE BASIC TIME STEP===
C      DO 210 I=1,N3
C          X(I) = X(I) + VH(I) + F(I)
210      CONTINUE
C
C      ===APPLY PERIODIC BOUNDARY CONDITIONS===
C      DO 215 I=1,N3
C          IF ( X(I) .LT. 0 ) X(I) = X(I) + SIDE
C          IF ( X(I) .GT. SIDE ) X(I) = X(I) - SIDE
215      CONTINUE
C
C      ===COMPUTE THE PARTIAL VELOCITIES===
C      DO 220 I=1,N3
C          VH(I) = VH(I) + F(I)
220      CONTINUE
C
C      =====
C      =
C      = THIS PART COMPUTES THE FORCES ON THE PARTICLES. =
C      =
C      =====
C
C      VIR = 0.0
C      EPOT = 0.0
C      DO 225 I=1,N3
C          F(I) = 0.0
225      CONTINUE
C
C
C      DO 270 I=1,NPART
C          XI = X(I)
C          YI = Y(I)
C          ZI = Z(I)
C      DO 270 J=I+1,NPART
C          XX = XI - X(J)
C          YY = YI - Y(J)
C          ZZ = ZI - Z(J)

```

```

C
      IF ( XX .LT. -SIDEH ) XX = XX + SIDE
      IF ( XX .GT. SIDEH ) XX = XX - SIDE
C
      IF ( YY .LT. -SIDEH ) YY = YY + SIDE
      IF ( YY .GT. SIDEH ) YY = YY - SIDE
C
      IF ( ZZ .LT. -SIDEH ) ZZ = ZZ + SIDE
      IF ( ZZ .GT. SIDEH ) ZZ = ZZ - SIDE
      RD = XX * XX + YY * YY + ZZ * ZZ
      IF ( RD .GT. RCOFFS ) GOTO 270
C
      EPOT = EPOT + RD ** (-6.0) - RD ** (-3.0)
      R148 = RD ** (-7.0) - 0.5 * RD ** (-4.0)
      VIR = VIR - RD * R148
      KX = XX * R148
      F(I) = F(I) + KX
      F(J) = F(J) - KX
      KY = YY * R148
      FY(I) = FY(I) + KY
      FY(J) = FY(J) - KY
      KZ = ZZ * R148
      FZ(I) = FZ(I) + KZ
      FZ(J) = FZ(J) - KZ
270      CONTINUE
      DO 275 I=1,N3
      F(I) = F(I) * HSQ2
275      CONTINUE
C
C      =====
C      =
C      =
C      =
C      =
C      =====
C
C      ===COMPUTE THE VELOCITIES===
      DO 300 I=1,N3
      VH(I) = VH(I) + F(I)
300      CONTINUE
C
C      ===COMPUTE THE KINETIC ENERGY===
      EKIN = 0.0
      DO 305 I=1,N3
      EKIN = EKIN + VH(I) * VH(I)
305      CONTINUE
      EKIN = EKIN / HSQ
C
C      ===COMPUTE THE AVERAGE VELOCITY===
      VEL = 0.0
      COUNT = 0.0

```

```

DO 306 I=1,NPART
  VX = VH(I) * VH(I)
  VY = VH(I+IOF1) * VH(I+IOF1)
  VZ = VH(I+IOF2) * VH(I+IOF2)
  SQ = SQRT( VX + VY + VZ )
  SQT = SQ / H
  IF ( SQT .GT. VAVER ) COUNT = COUNT + 1
  VEL = VEL + SQ
306 CONTINUE
  VEL = VEL / H

C
  IF ( (CLOCK/IREP)*IREP .EQ. CLOCK) THEN
C
C      ---REPLACE THE VELOCITIES---
      WRITE(6,*) 'VELOCITY REPLACEMENT'
      CALL MXWELL(VH,N3,H,TREF)
      EKin = TREF / TSCALE
  END IF

C
C
C      =====
C      =
C      =          COMPUTE VARIOUS QUANTITIES          =
C      =
C      =====

  EK   = 24.0 * EKin
  EPOT = 4.0 * EPOT
  ETOT = EK + EPOT
  TEMP = TSCALE * EKin
  PRES = DEN * 16.0 * ( EKin - VIR ) / NPART
  VEL  = VEL / NPART
  RP   = (COUNT / 256.0 ) * 100.0
  WRITE(6,6000) CLOCK,EK,EPOT,ETOT,TEMP,PRES,VEL,RP
6000 FORMAT(1I6,7F15.6)

C
200 CONTINUE

C
C
  STOP
  END

C
C-----
C
C  M X W E L L  RETURNS UPON CALLING A MAXWELL DISTRIBUTED DEVIATES
C  FOR THE SPECIFIED TEMPERATURE TREF. ALL DEVIATES ARE SCALED BY
C  A FACTOR.
C
C  CALLING PARAMETERS ARE AS FOLLOWS
C

```

```

C      VH      VECTOR OF LENGTH NPART ( MUST BE A MULTIPLE OF 2)
C      N3      VECTOR LENGTH
C      H      SCALING FACTOR WITH WHICH ALL ELEMENTS OF VH ARE
C              MULTIPLIED.
C      TREF    TEMPERATURE
C
C      VERSION 1.0 AS OF AUGUST 1985
C      DIETER W. HEERMANN
C

```

```

C-----

```

```

C
      SUBROUTINE MXWELL(VH,N3,H,TREF)
C
      REAL VH(1:N3)
C
      REAL U1,U2,V1,V2,S,R
C
      NPART   = N3 / 3
      IOF1    = NPART
      IOF2    = 2 * NPART
      TSCALE  = 16.0 / (1.0 * NPART - 1.0)
C
      DO 10 I=1,N3,2
C
      1      U1 = RANF()
          U2 = RANF()
C
          V1 = 2.0 * U1 - 1.0
          V2 = 2.0 * U2 - 1.0
          S  = V1 * V1 + V2 * V2
C
          IF ( S .GE. 1.0 ) GOTO 1
          R   = -2.0 * ALOG(S) / S
          VH(I) = V1 * SQRT( R )
          VH(I+1) = V2 * SQRT( R )
10      CONTINUE
C
      EKin = 0.0
      SP = 0.0
      DO 20 I=1,NPART
          SP = SP + VH(I)
20      CONTINUE
      SP = SP / NPART
      DO 21 I=1,NPART
          VH(I) = VH(I) - SP
          EKin = EKin + VH(I) * VH(I)
21      CONTINUE
      SP = 0.0
      DO 22 I=IOF1 + 1,IOF2
          SP = SP + VH(I)

```

```

22  CONTINUE
    SP = SP / NPART
    DO 23 I=IOF1 + 1,IOF2
        VH(I) = VH(I) - SP
        EKIN = EKIN + VH(I) * VH(I)
23  CONTINUE
    SP = 0.0
    DO 24 I=IOF2 + 1,N3
        SP = SP + VH(I)
24  CONTINUE
    SP = SP / NPART
    DO 25 I=IOF2 + 1,N3
        VH(I) = VH(I) - SP
        EKIN = EKIN + VH(I) * VH(I)
25  CONTINUE
    TS = TSCALE * EKIN
    SC = TREF / TS
    SC = SQRT( SC )
    SC = SC * H
    DO 30 I=1,N3
        VH(I) = VH(I) * SC
30  CONTINUE
    END

```

The program simulates a Lennard-Jones system with N particles in an MD cell of volume $V = \sigma L^3$ at constant temperature. Initially the particles are placed on a face centred cubic lattice. The velocities are chosen from a Gaussian distribution. The Andersen algorithm is used to achieve the constant temperature. At specified intervals all velocities are replaced.

Program Listing PL3. Microcanonical Monte-Carlo Method

```

C-----DWHH-----
C
C      SIUMULATION OF THE MICRO-CANONICAL ENSEMBLE
C      ON THE COEXISTENCE CURVE AND IN THE METASTABLE
C      REGION FOR THE NEAREST-NEIGHBOUR ISING MODEL.
C
C      FIELD VERSION
C
C      DIETER W. HEERMANN
C
C      AUGUST 1983
C-----
C
C      DIMENSION ISS(12,12,12),IM(12),IP(12)
C      DIMENSION IDIST(2000)
C

```

```

      REAL      DEMON,H
      REAL      ENERGY,ET
      REAL      RLCUBE

C
      REAL      MODM2,PB,RAN
      REAL      DEMAV,MAGAV

C
C-----
C
C      SIMULATION PARAMETERS
C
C-----
C
      H      = 0.0
      L      = 12
      MCSMAX = 100
      M      = - L*L*L / 2
      ISEED   = 4711
      PB      = 0.0155
      IFLAG   = 2
      RLCUBE  = L*L*L

C
C-----INITIALIZE
C
      DO 1      I=1,L
         IM(I) = I-1
         IP(I) = I+1
1      CONTINUE
      IM(1) = L
      IP(L) = 1

C
      DO 2      I=1,1000
         IDIST(I) = 0
2      CONTINUE

C
      DO 5      I=1,L
         DO 5   J=1,L
            DO 5 K=1,L
               ISS(I,J,K) = -13
5      CONTINUE
         ic = 0
      DO 10 I=1,L
         DO 10 J=1,L
            DO 10 K=1,L
               RAN = RANF(ISEED)
               IF ( RAN .GT. PB ) GOTO 10
               M = M + 1
               ISS(I,J,K)      = ISS(I,J,K) + 14
               ISS(IM(I),J,K)   = ISS(IM(I),J,K) + 2

```

```

ISS(IP(I),J,K)      = ISS(IP(I),J,K) + 2
ISS(I,IM(J),K)      = ISS(I,IM(J),K) + 2
ISS(I,IP(J),K)      = ISS(I,IP(J),K) + 2
ISS(I,J,IM(K))      = ISS(I,J,IM(K)) + 2
ISS(I,J,IP(K))      = ISS(I,J,IP(K)) + 2
10  CONTINUE
C
ENERGY = 0.0
DO 20 I=1,L
DO 20 J=1,L
DO 20 K=1,L
ICI    = ISS(I,J,K)
IVORZ  = ISIGN(1,ICI)
ICIA   = ICI * IVORZ
ENERGY = ENERGY + ICIA - 7
20  CONTINUE
ENERGY = - ENERGY* 2.0 *3.0/8.0 - H * 2.0 * M
ENERGY = ENERGY / 32768.0
H = H * 4.0 / 3.0
WRITE(*,6000) PB,ENERGY,M
IF ( IFLAG .EQ. 1 ) STOP 1
C
C-----
C
C  MONTE CARLO
C
C-----
C
DEMAV = 0.0
MAGAV = 0.0
DEMON = 0.0
FLDEM = 0.0
C
DO 200 MCS=1,MCSMAX
DO 100 IZ=1,L
IMZ = IM(IZ)
IPZ = IP(IZ)
DO 100 IY=1,L
IMY = IM(IY)
IPY = IP(IY)
DO 100 IX=1,L
C
ICI    = ISS(IX,IY,IZ)
IVORZ  = ISIGN(1,ICI)
IEN    = ICI * IVORZ - 7
C
IF ( DEMON - IEN - H * IVORZ .LT. 0 ) GOTO 100
DEMON = DEMON - IEN - H * IVORZ
C-----FLIP SPIN

```

```

M = M - IVORZ
ISS(IX,IY,IZ) = ICI - IVORZ * 14
ICH = - 2 * IVORZ
ISS(IM(IX),IY,IZ) = ISS(IM(IX),IY,IZ) + ICH
ISS(IP(IX),IY,IZ) = ISS(IP(IX),IY,IZ) + ICH
ISS(IX,IMY,IZ) = ISS(IX,IMY,IZ) + ICH
ISS(IX,IPY,IZ) = ISS(IX,IPY,IZ) + ICH
ISS(IX,IY,IMZ) = ISS(IX,IY,IMZ) + ICH
ISS(IX,IY,IPZ) = ISS(IX,IY,IPZ) + ICH
100 CONTINUE
C
C IPTR = 10 * DEMON + 1
C IDIST( IPTR ) = IDIST( IPTR ) + 1
DEMAV = DEMAV + DEMON
MAGAV = MAGAV + M
FLDEM = FLDEM + DEMON * DEMON
C
200 CONTINUE
DEMAV = DEMAV / MCSMAX
MAGAV = MAGAV / MCSMAX
WRITE(*,6200) DEMAV,MAGAV
C
FLUCT = (FLDEM - DEMAV*DEMAV/MCSMAX) / MCSMAX
WRITE(*,6400) FLUCT
C DO 900 J=1,991,10
C WRITE(*,6500) (IDIST(J-1+I),I=1,10)
C900 CONTINUE
C
C-----F O R M A T S-----
C
6000 FORMAT(1H ,1E20.6,2X,1E20.6,2X,1I10)
6100 FORMAT(1H ,1I10,3X,1E20.6,3X,1I10)
6200 FORMAT(1HO,'DEMON AV = ',1E20.6,3X,'MAG AV = ',1E20.6)
6300 FORMAT(1HO,1I10,1X,1E20.6,1X,1E20.6,1X,1E20.6,1X,1I10)
6400 FORMAT(1HO,'DEMON FLUCTUATION = ',1E20.6)
6500 FORMAT(1HO,10(2X,1I10))
STOP
END

```

To simulate the behaviour of the three-dimensional Ising model in the microcanonical approach, the program uses the Creutz method (Algorithm A10). In this particular program the code is partly optimized as described below.

The most time-consuming step in a Monte-Carlo algorithm is the computation of the energy change. The transition probabilities need not be computed, they can be stored in a look-up table for easy access. To reduce the number of operations we make the following observation. A spin can take on only the values +1 and -1. The sum over the nearest neighbours of a given spin must be

$$\sum_{\langle i,j \rangle} s_i s_j \in \{-6, -4, -2, 0, 2, 4, 6\} .$$

If 7 is added to the sum, then only integers from 1 to 13 appear. These integers can be used as entries into an array. The extra addition is not necessary for FORTRAN compilers allowing negative indices for arrays. Instead of computing the energy change for a given spin flip, we store the energy instead of just the spin orientation. With this, the decision for or against a spin flip is made without computation, hence drastically reducing the number of operations in the innermost loop. If the decision for a spin flip is positive, an update must be performed on the central spin and on the neighbouring six spins only.

Program Listing PL4. Canonical Monte-Carlo Method

```

C-----
C
C      PROGRAM TO SIMULATE THE THREE DIMENSIONAL ISING MODEL
C      WITH THE GLAUBER DYNAMICS
C
C      REQUIRED PARAMETERS FOR A SIMULATION ARE:
C      -----
C      TEMP   :  TEMPERATURE IN UNITS OF THE CRITICAL TEMPERATURE
C      H      :  APPLIED MAGNETIC FIELD
C      L      :  LINEAR SYSTEM SIZE
C      ISEED   :  SEED FOR THE MODULO GENERATOR.
C      MCSMAX :  MAXIMUM NUMBER OF MONTE CARLO STEPS
C      ISTART  :  DISCARD THE FIRST 'ISTART' CONFIGURATIONS
C
C      REMARK:   FOR PRECISION RESULTS THE RANDOM NUMBER GENERATOR
C      -----
C      R250 SHOULD BE USED.
C-----
C
C      DIMENSION ISS(10,10,10),IM(10),IP(10)
C      DIMENSION IEX(14)
C
C      REAL      IEX
C      REAL      MAG,FLMAG
C
C      SET THE SIMULATION PARAMETERS
C      -----
C
C      WRITE(*,*) 'GIVE TEMPERATURE = '
C      READ(*,*) TEMP
C      WRITE(*,*) 'GIVE MAGNETIC FIELD = '
C      READ(*,*) H
C      L      = 10

```

```

        WRITE(*,*) 'GIVE ISEED'
        READ(*,*) ISEED
        MCSMAX = 1000
        ISTART = 0

C
C
        T      = TEMP / 0.221673
        WRITE(6,*) 'ISING 3D   LINEAR SIZE = ',L
        WRITE(6,*) 'TEMPERATURE IS           ',TEMP
        WRITE(6,*) 'FIELD IS                 ',H

C
C      SET UP THE RANDOM NUMBER GENERATOR
C      -----
C
        ISEED = ISEED * 4 + 1

C
C      INITIALIZE THE OTHER VARIABLES
C      -----
C
        DO 1 I=1,L
            IM(I) = I-1
            IP(I) = I+1
1      CONTINUE
        IM(1) = L
        IP(L) = 1
        RLCUBE = L*L*L

C
        M      = - L*L*L / 2
        COUNT = 0.0
        MAG    = 0.0
        FLMAG  = 0.0

C
C      INITIALIZE THE TRANSITION PROBABILITIES
C      -----
C
        DO 4 I=1,13,2
            IEX(I ) = 1.0
            IEX(I+1) = 1.0
            EX = EXP((I-7.0)*2.0/T - H)
            IF (EX .GT. 1 ) IEX(I) = 1.0/EX
            EX = EXP((I-7.0)*2.0/T + H)
            IF (EX .GT. 1 ) IEX(I+1) = 1.0/EX
4      CONTINUE

C
C      INITIALIZE THE FIRST CONFIGURATION AS ALL SPINS DOWN
C      -----
C
        DO 5 I=1,L
            DO 5 J=1,L
                DO 5 K=1,L

```

```

      ISS(I,J,K) = -13
5    CONTINUE
C
C-----
C
C    MONTE CARLO PART
C-----
C
      DO 200 MCS=1,MCSMAX
      DO 100 IZ=1,L
        IMZ = IM(IZ)
        IPZ = IP(IZ)
        DO 100 IY=1,L
          IMY = IM(IY)
          IPY = IP(IY)
          DO 100 IX=1,L

            CCI = ISS(IX,IY,IZ)
            IVORZ = ISIGN(1,CCI)
            IEN = CCI * IVORZ

            IF ( IEN .LT. 7 ) GOTO 8
            RANF = RANF(ISEED)
            IF ( RANF .GT. IEX(IEN) ) GOTO 100
8          CONTINUE
C-----FLIP SPIN
            M = M - IVORZ
            ISS(IX,IY,IZ) = CCI - IVORZ * 15
            ICH = - 2 * IVORZ
            ISS(IM(IX),IY,IZ) = ISS(IM(IX),IY,IZ) + ICH
            ISS(IP(IX),IY,IZ) = ISS(IP(IX),IY,IZ) + ICH
            ISS(IX,IMY,IZ) = ISS(IX,IMY,IZ) + ICH
            ISS(IX,IPY,IZ) = ISS(IX,IPY,IZ) + ICH
            ISS(IX,IY,IMZ) = ISS(IX,IY,IMZ) + ICH
            ISS(IX,IY,IPZ) = ISS(IX,IY,IPZ) + ICH
100        CONTINUE
C
            IF ( MCS .LE. ISTART ) GOTO 200
            COUNT = COUNT + 1.0
            MAG = MAG + ABS(2.0 * M )
            RMAG = (2.0*M) / RLCUBE
            WRITE(*,*) MCS,RMAG
200        CONTINUE
C
            RMAG = MAG / RLCUBE
            RMAG = RMAG / COUNT
            WRITE(6,7000) TEMP,COUNT,RMAG
C
7000  FORMAT(1H0,'TEMP = ',1E20.10,2X,'AV OVER ',1E20.10,2X,

```

```

+      * MAG = ',1E20.10,2X)
      STOP
      END

```

The program is an adaptation of the algorithm described in Sect.4.3.3. Its purpose is to simulate the two-dimensional Ising model with single spin flip. The ideas for the code are the same as described in the Program Listing PL3. For other ideas for optimizing an Ising program, the reader is directed to [A2.2, 3] describing the multi spin coding method. Those interested in methods for vector machines should consult the papers by *Kalle* and *Winkelmann* [A2.4], *Williams* and *Kalos* [A2.5], and [A2.6-8] for other lines of attack.

References

Chapter 1

- 1.1 J. Hoshen, R. Kopelman: *Phys. Rev. B* 14, 3428 (1976)
- 1.2 H. Müller-Krumbhaar: In *Monte Carlo Methods in Statistical Physics*, 2nd ed., ed. by K. Binder, Topics Curr. Phys., Vol.7 (Springer, Berlin, Heidelberg 1986)
- 1.3 M. Eden: In *Proc. 4th Berkeley Symp. Math. Statist. and Probability*, ed. by F. Neyman (Univ. California Press, Berkeley 1960) Vol.4

Chapter 2

- 2.1 E. Friedberg, J.E. Cameron: *J. Chem. Phys.* 52, 6049 (1970)
- 2.2 E.B. Smith, B.H. Wells: *Mol. Phys.* 52, 709 (1984)
- 2.3 D. Fincham: Daresbury Lab. Information Quarterly for Computer Simulation of Condensed Phases 17, 43 (1985)
- 2.4 K. Binder, D.W. Heermann: *The Monte Carlo Method in Statistical Physics*, Springer Ser. Solid-State Sci., Vol.80 (Springer, Berlin, Heidelberg 1988)
- 2.5 W.G. Hoover: *Molecular Dynamics* (Springer, Berlin, Heidelberg 1986)
- 2.6 M.H. Kalos, P.A. Whitlock: *Monte Carlo Methods*, Vol.1 (Wiley, New York 1986)
- 2.7 D. Stauffer, F.W. Hehl, V. Winkelmann, J.G. Zabolitzky: *Computational Physics* (Springer, Berlin, Heidelberg 1988)
- 2.8 D. Rapaport: *Comp. Phys. Rep.* 9, 1 (1988)
- 2.9 J.L. Lebowitz, J.K. Percus: *Phys. Rev.* 124, 1673 (1961)
- 2.10 J.L. Lebowitz, J.K. Percus, L. Verlet: *Phys. Rev.* 124, 1673 (1967)

Chapter 3

- 3.1 B.J. Alder, T.E. Wainwright: *J. Chem. Phys.* 27, 1208 (1957)
- 3.2 T.E. Wainwright, B.J. Alder: *Nuovo cimento* 9, Suppl. Sec. 116 (1958)
- 3.3 B.J. Alder, T.E. Wainwright: *J. Chem. Phys.* 31, 456 (1959)
- 3.4 B.J. Alder, T.E. Wainwright: *J. Chem. Phys.* 33, 1439 (1960)
- 3.5 J.R. Beeler, Jr.: In *Physics of Many-Particle Systems*, ed. by C. Meeron (Gordon & Breach, New York 1964)
- 3.6 A. Rahman: *Phys. Rev.* 136, A405 (1964)
- 3.7 L. Verlet: *Phys. Rev.* 159, 98 (1967)
- 3.8 K. Binder, M.H. Kalos: In *Monte Carlo Methods in Statistical Physics*, 2nd edn., ed. by K. Binder, Topics Curr. Phys., Vol.7 (Springer, Berlin, Heidelberg 1986)
- 3.9 P. Turq, F. Lantelme, H.L. Friedman: *J. Chem. Phys.* 46, 3039 (1977)
- 3.10 J.P. Ryckaert, G. Ciccotti, H.J.C. Berendsen: *J. Comput. Phys.* 23, 327 (1977)
- 3.11 G. Ciccotti, M. Ferrario, J.P. Ryckaert: *Mol. Phys.* 47, 1253 (1982)
- 3.12 J.P. Ryckaert, G. Ciccotti: *J. Chem. Phys.* 78, 7368 (1983)
- 3.13 W.F. van Gunsteren, H.J.C. Berendsen: *Mod. Phys.* 34, 1311 (1977)
- 3.14 D.J. Tidlesley, P.A. Madden: *Mol. Phys.* 42, 1137 (1981)
- 3.15 R.M. Stratt, S.L. Holmgreen, D. Chandler: *Mol. Phys.* 46, 1233 (1981)
- 3.16 M.K. Memon, R.W. Hockney, S.K. Mitra: *J. Comput. Phys.* 43, 345 (1982)

- 3.17 S. Nose, M.L. Klein: *Mol. Phys.* **50**, 1055 (1983)
- 3.18 H.C. Anderson: *J. Comput. Phys.* **52**, 24 (1983)
- 3.19 L.D. Landau, E.M. Lifshitz: *Statistical Physics* Vol.5, 3rd edn. (Pergamon, Oxford 1980) p.42
- 3.20 W.W. Wood, F.R. Parker: *J. Chem. Phys.* **27**, 720 (1957)
- 3.21 J.N. Shaw: Monte Carlo calculations for a system of hard-sphere ions, PhD Thesis, Duke University (1963)
- 3.22 P.P. Ewald: *Ann. Physik* **64**, 253 (1921)
- 3.23 S.G. Brush, H.L. Sahlin, E. Teller: *J. Chem. Phys.* **45**, 2102 (1966)
- 3.24 M. Parinello, A. Rahman: *Phys. Rev. Lett.* **45**, 1196 (1980)
- 3.25 M. Parinello, A. Rahman: *J. Appl. Phys.* **52**, 7182 (1981)
- 3.26 M. Parinello, A. Rahman: *J. Chem. Phys.* **76**, 2662 (1982)
- 3.27 M. Parinello, A. Rahman, P. Vashishtra: *Phys. Rev. Lett.* **50**, 1073 (1983)
- 3.28 G.S. Pawley, G.W. Thomas: *Phys. Rev. Lett.* **48**, 410 (1982)
- 3.29 S. Nose, M.L. Klein: *J. Chem. Phys.* **78**, 6928 (1983)
- 3.30 G. Dahlquist, A. Bjorck: *Numerical Methods* (Prentice Hall, Englewood Cliffs, NJ 1964)
- 3.31 J. Stoer, R. Bulirsch: *Einführung in die Numerische Mathematik II* (Springer, Berlin, Heidelberg 1973)
- 3.32 A. Nordsieck: *Math. Comput.* **16**, 22 (1962)
- 3.33 C.W. Gear: *Numerical Initial Value Problems in Ordinary Differential Equations* (Prentice Hall, Englewood Cliffs, NJ 1971)
- 3.34 C.W. Gear: Report ANL 7126, Argonne National Lab. (1966)
- 3.35 C.W. Gear: *Math. Comput.* **21**, 146 (1967)
- 3.36 D. Beeman: *J. Comput. Phys.* **20**, 130 (1976)
- 3.37 S. Toxvaerd: *J. Comput. Phys.* **47**, 444 (1982)
- 3.38 A. LaBudde, D. Greenspan: *J. Comput. Phys.* **15**, 134 (1974)
- 3.39 D. Greenspan: *J. Comput. Phys.* **56**, 28 (1984)
- 3.40 K. Kanatani: *J. Comput. Phys.* **53**, 181 (1984)
- 3.41 K. Aizu: *J. Comput. Phys.* **55**, 154 (1984)
- 3.42 K. Aizu: *J. Comput. Phys.* **58**, 270 (1985)
- 3.43 J.H. Wilkinson: *Rounding Errors in Algebraic Processes* (HMSO, London 1963)
- 3.44 O. Penrose, J.L. Lebowitz: In *Studies in Statistical Mechanics*, Vol.7, ed. by J.L. Lebowitz, E.W. Montroll (North-Holland, Amsterdam 1979)
- 3.45 D.W. Heermann, W. Klein, D. Stauffer: *Phys. Rev. Lett.* **49**, 1262 (1982)
- 3.46 W.C. Swope, H.C. Andersen, P.H. Berens, K.R. Wilson: *J. Chem. Phys.* **76**, 637 (1982)
- 3.47 L.V. Woodcock: *Chem. Phys. Lett.* **10**, 257 (1970)
- 3.48 B. Quentrec, C. Brot: *J. Comput. Phys.* **1**, 430 (1973)
- 3.49 J.M. Haile, S. Gupta: *J. Chem. Phys.* **79**, 3067 (1983)
- 3.50 W.G. Hoover, D.J. Evans, R.B. Hickman, A.J.C. Ladd, W.T. Ashurst, B. Moran: *Phys. Rev. A* **22**, 1690 (1980)
- 3.51 W.G. Hoover: *Physica A* **18**, 111 (1983)
- 3.52 W.G. Hoover: In *Nonlinear Fluid Behaviour*, ed. by H.J. Hanley (North-Holland, Amsterdam 1983)
- 3.53 W.G. Hoover, A.J.C. Ladd, B. Moran: *Phys. Rev. Lett.* **48**, 3297 (1983)
- 3.54 D.J. Evans, G.P. Morriss: *Chem. Phys.* **77**, 63 (1983)
- 3.55 D.J. Evans: *J. Chem. Phys.* **78**, 3297 (1983)
- 3.56 D.M. Heyes, D.J. Evans, G.P. Morriss: *Daresbury Lab. Information Quarterly for Computer Simulation of Condensed Phases* **17**, 25 (1985)
- 3.57 F. van Swol, L.V. Woodcock, J.N. Cape: *J. Chem. Phys.* **75**, 913 (1980)
- 3.58 J.Q. Broughton, G.H. Gilmer, J.D. Weeks: *J. Chem. Phys.* **75**, 5128 (1981)
- 3.59 D. Brown, J.H.R. Clarke: *Mol. Phys.* **51**, 1243 (1984)
- 3.60 S. Nose: *Mol. Phys.* **52**, 255 (1984)

- 3.61 S. Nose, M.L. Klein: *Mol. Phys.* **50**, 1055 (1983)
- 3.62 S. Nose: *J. Chem. Phys.* **81**, 511 (1984)
- 3.63 A. DiNola, J.R. Haak: *J. Chem. Phys.* **81**, 3684 (1984)
- 3.64 J.R. Ray: *Am. J. Phys.* **40**, 179 (1972)
- 3.65 H.C. Andersen: *J. Chem. Phys.* **72**, 2384 (1980)
- 3.66 J.M. Haile, H.W. Graben: *J. Chem. Phys.* **73**, 2412 (1980)
- 3.67 W. Smith: Unpublished
- 3.68 D. Adams: Daresbury Lab. Information Quarterly for Computer Simulation of Condensed Phases **10**, 30 (1983)
- 3.69 D. Fincham: Daresbury Lab. Information Quarterly for Computer Simulation of Condensed Phases **12**, 43 (1984)

Chapter 4

- 4.1 W. Feller: *An Introduction to Probability Theory and Its Applications* (Wiley, New York 1968)
- 4.2 G.R. Grimmett, D.R. Stirzaker: *Probability and Random Processes* (Oxford Univ. Press, Oxford 1987)
- 4.3 J.G. Kemeny, J.L. Snell: *Finite Markov Chains* (Springer, Berlin, Heidelberg 1976)
- 4.4 K. Kremer, K. Binder: *Comput. Phys. Rep.* **7**, 259 (1988)
- 4.5 P. Turq, F. Lantelme, H.L. Friedman: *J. Chem. Phys.* **66**, 3039 (1977)
- 4.6 G.S. Grest, K. Kremer: *Phys. Rev. Lett.* **33**, 3628 (1986)
- 4.7 S. Chandrasekhar: *Rev. Mod. Phys.* **15**, 1 (1943)
- 4.8 M.C. Wang, G.E. Uhlenbeck: *Rev. Mod. Phys.* **17**, 323 (1945)
- 4.9 A. Ludwig: *Stochastic Differential Equations: Theory and Applications* (Wiley, New York 1974)
- 4.10 S.O. Rice: *Bell Syst. Tech. J.* **23**, (1946)
- 4.11 J.L. Doob: *Ann. Math.* **43**, 351 (1942)
- 4.12 J.L. Doob: *Ann. Am. Stat.* **15**, 229 (1944)
- 4.13 D.L. Ermak: *J. Chem. Phys.* **62**, 4189 (1974)
- 4.14 D.L. Ermak: *J. Chem. Phys.* **62**, 4197 (1974)
- 4.15 J.D. Doll, D.R. Dion: *J. Chem. Phys.* **65**, 3762 (1976)
- 4.16 T. Schneider, E. Stoll: *Phys. Rev. B* **13**, 1216 (1976)
- 4.17 T. Schneider, E. Stoll: *Phys. Rev. B* **17**, 1302 (1978)
- 4.18 W.F. van Gunsteren, H.J.C. Berendsen: *Mol. Phys.* **45**, 637 (1982)
- 4.19 M.P. Allen: *Stochastic Dynamics* (CECAM Workshop Report 1978)
- 4.20 M.P. Allen: *Mol. Phys.* **40**, 1073 (1980)
- 4.21 M.P. Allen: *Mol. Phys.* **47**, 599 (1982)
- 4.22 H.J.C. Berendsen, J.P.M. Postma, W.F. van Gunsteren, A. DiNola, J.R. Haak: *J. Chem. Phys.* **81**, 3684 (1984)
- 4.23 H.C. Andersen: *J. Chem. Phys.* **72**, 2384 (1980)
- 4.24 A. Greiner, W. Strittmatter, J. Honerkamp: *J. Stat. Phys.* **51**, 95 (1988)
- 4.25 B. Dünweg, W. Paul: To be published
- 4.26 J.H. Halton: *SIAM Rev.* **12**, 4 (1970)
- 4.27 J.M. Hammersley, D.C. Handscomb: *Monte Carlo Methods* (Chapman & Hall, New York 1964)
- 4.28 F. James: *Rep. Prog. Phys.* **43**, 73 (1980)
- 4.29 A. Papoulis: *Probability, Random Variables and Stochastic Processes* (McGraw-Hill, Tokyo 1965)
- 4.30 N. Metropolis, A.W. Rosenbluth, M.N. Rosenbluth, A.H. Teller, E. Teller: *J. Chem. Phys.* **21**, 1087 (1953)
- 4.31 H. Müller-Krumbhaar, K. Binder: *J. Stat. Phys.* **8**, 1 (1973)
- 4.32 K. Binder: *Adv. Phys.* **23**, 917 (1974)

- 4.33 K. Binder: In *Phase Transitions and Critical Phenomena*, ed. by C. Domb, M.S. Green (Academic, New York 1976)
- 4.34 L.D. Fosdick: *Methods Comput. Phys.* 1, 245 (1963)
- 4.35 I.Z. Fisher: *Sov. Phys.-Usp.* 2, 783 (1959)
- 4.36 K. Binder: In *Monte Carlo Methods in Statistical Physics*, 2nd ed., ed. by K. Binder, *Topic Curr. Phys.*, Vol.7 (Springer, Berlin, Heidelberg 1986)
- 4.37 K. Kawasaki: In *Phase Transitions and Critical Phenomena*, Vol.2, ed. by C. Domb, M.S. Green (Academic, New York 1972)
- 4.38 W.W. Wood: In *Physics of Simple Liquids*, ed. by H.N.V. Temperley, J.S. Rushbrooke (North-Holland, Amsterdam 1968)
- 4.39 H.E. Stanley: *Introduction to Phase Transitions and Critical Phenomena* (Oxford Univ. Press, London 1971)
- 4.40 O.G. Mouritsen: *Computer Studies of Phase Transitions and Critical Phenomena* (Springer, Berlin, Heidelberg 1984)
- 4.41 T.L. Hill: *Thermodynamics of Small Systems* (Benjamin, New York 1963)
- 4.42 G.S. Pawley, G.W. Thomas: *Phys. Rev. Lett.* 48, 410 (1982)
- 4.43 R.G. Palmer: *Adv. Phys.* 31, 669 (1982)
- 4.44 D. Stauffer: Private communication
- 4.45 A.E. Ferdinand, M.E. Fisher: *Phys. Rev.* 185, 832 (1969)
- 4.46 P.C. Hohenberg, B.I. Halperin: *Rev. Mod. Phys.* 49, 435 (1977)
- 4.47 K. Binder, D. Stauffer: *Adv. Phys.* 25, 343 (1976)
- 4.48 K. Binder: *J. Stat. Phys.* 24, 69 (1981)
E. Stoll, K. Binder, T. Schneider: *Phys. Rev. B* 8, 3266 (1973)
- 4.49 N.Y. Choi, B.A. Huberman: *Phys. Rev. B* 29, 2796 (1984)
- 4.50 K. Binder: *J. Comput. Phys.* 59, 1 (1985)
- 4.51 B.K. Chabrabarty, H.G. Baumgärtel, D. Stauffer: *Z. Phys. B* 44, 333 (1981)
- 4.52 C. Kalle: *J. Phys. A* 17, (1984)
- 4.53 M. Creutz: *Phys. Rev. Lett.* 50, 1411 (1983)
- 4.54 G. Bahnot, M. Creutz, H. Neuberger: *Nucl. Phys. B* 235 [FS11], 417 (1984)
- 4.55 D.W. Heermann, R.C. Desai: *Comput. Phys. Commun.* 50, 297 (1988)
- 4.56 R.C. Tolman: *The Principles of Statistical Mechanics* (Dover, New York 1979)
- 4.57 E. Ising: *Z. Phys.* 31, 253 (1925)
- 4.58 K. Binder, H. Müller-Krumbhaar: *Phys. Rev. B* 7, 3297 (1973)
- 4.59 J.W. Cahn: *J. Chem. Phys.* 66, 3667 (1977)
- 4.60 K. Binder, D.P. Landau: *J. Appl. Phys.* 57, 3306 (1985)
- 4.61 R.J. Glauber: *J. Math. Phys.* 4, 294 (1963)
- 4.62 W. Paul, D.W. Heermann, K. Binder: *J. Phys. A* 22, 325 (1989)
- 4.63 M.N. Barber: In *Phase Transitions and Critical Phenomena*, ed. by C. Domb, J.L. Lebowitz (Academic, New York 1983)
- 4.64 D.P. Landau: *Phys. Rev. B* 13, 2997 (1967)
- 4.65 R.H. Swendsen, S. Krinsky: *Phys. Rev. Lett.* 43, 177 (1979)
- 4.66 K. Binder, D.W. Heermann: *The Monte Carlo Method in Statistical Physics*, Springer Ser. Solid-State Sci., Vol.80 (Springer, Berlin, Heidelberg 1988)
- 4.67 A.B. Bortz, M.H. Kalos, J.L. Lebowitz: *J. Comput. Phys.* 17, 10 (1975)
- 4.68 A. Sadiq: *J. Comput. Phys.* 55, 387 (1984)
- 4.69 Z.W. Salsburg, J.D. Jacobsen, W. Fickett, W.W. Wood: *J. Chem. Phys.* 30, 65 (1959)
- 4.70 J.P. Hansen, L. Verlet: *Phys. Rev.* 184, 151 (1969)
- 4.71 K. Binder: *J. Stat. Phys.* 24, 69 (1981)
- 4.72 K. Binder: *Z. Phys. B* 45, 61 (1981)
- 4.73 T.L. Polgreen: *Phys. Rev. B* 29, 1468 (1984)
- 4.74 S.K. Ma: *J. Stat. Phys.* 26, 221 (1981)
- 4.75 Z. Alexandrowicz: *J. Chem. Phys.* 55, 2765 (1971)
- 4.76 H. Meirovitch, Z. Alexandrowicz: *J. Stat. Phys.* 16, 121 (1977)

- 4.77 H. Meirovitch: Chem. Phys. Lett. 45, 389 (1977)
- 4.78 H. Meirovitch: J. Stat. Phys. 30 681 (1983)
- 4.79 J.P. Valleau, D.N. Cord: J. Chem. Phys. 57, 5457 (1972)
- 4.80 G. Torrie, J.P. Valleau: Chem. Phys. Lett. 28, 578 (1974)
- 4.81 G. Torrie, J.P. Valleau: J. Comput. Phys. 23, 187 (1977)
- 4.82 K. Kawasaki, T. Imaeda, J.D. Gunton: In *Perspectives in Statistical Physics*, ed. by H.J. Raveche (North-Holland, Amsterdam 1981)
- 4.83 J.D. Gunton, M. San Miguel, P. Sahn: In *Phase Transitions and Critical Phenomena*, Vol.8, ed. by C. Domb, J.L. Lebowitz (Academic, New York 1985)
- 4.84 L.P. Kadanoff: Physics 2, 263 (1966)
- 4.85 B. Freedman, P. Smolensky, D. Weingarten: Phys. Lett. 113B, 481 (1982)
- 4.86 A. Milchev, D.W. Heermann, K. Binder: J. Stat. Phys. 44, 749 (1986)
- 4.87 R. Swendsen, Wang: Phys. Rev. Lett. 50, 297 (1988)
- 4.88 U. Wolff: Phys. Rev. Lett. 50, 297 (1988)
- 4.89 A.N. Burkitt, D.W. Heermann: to be published
- 4.90 C.M. Fortuin, P.W. Kastelyn: Physica 57, 536 (1972)
- 4.91 M. DeMeo, D.W. Heermann, K. Binder: J. Stat. Phys. in press
- 4.92 J.D. Doll, R.D. Coalsen, D.L. Freeman: Phys. Rev. Lett. 55, 1 (1985)
- 4.93 S. Duane, A.D. Kennedy, B.J. Pendelton, D. Roweth: Phys. Rev. Lett. 2, 195 (1987)
- 4.94 G.E. Norman, V.S. Filinov: High Temp. (USSR) 7, 216 (1969)
- 4.95 J.P. Valleau, L.K. Cohen: J. Chem. Phys. 72, 5935 (1980)
- 4.96 D.J. Adams: Mol. Phys. 28, 1241 (1974)
- 4.97 D.J. Adams: Mol. Phys. 29, 307 (1975)
- 4.98 L.A. Rowley, D. Nicholson, N.G. Parsonage: J. Comput. Phys. 17, 401 (1975)
- 4.99 L.A. Rowley, D. Nicholson, N.G. Parsonage: Mol. Phys. 31, 365 (1976)
- 4.100 J. Yao: PhD Dissertation, Purdue University (1981)
- 4.101 J. Yao, R.A. Greenkorn, K.C. Chao: Mol. Phys. 46, 587 (1982)
- 4.102 A.Z. Panagiatopoulos: Mol. Phys. 61, 813 (1987)
- 4.103 A.Z. Panagiatopoulos, N. Quirke, M. Stapleton, D. Tildesley: Mol. Phys. 63, 527 (1988)
- 4.104 H. Herrmann: J. Stat. Phys. 45, 145 (1986)
- 4.105 G. Grinstein, C. Jayaprakash, Y. He: Phys. Rev. Lett. 55, 2527 (1987)
- 4.106 Kauffman: Phys. Rev. Lett. (1987)
- 4.107 M. Rovere, D.W. Heermann, K. Binder: Europhys. Lett. 6, 585 (1988)

Appendix

- A1.1 J. Lach: Unpublished (1962)
- A1.2 M.N. Barber, R.B. Pearson, D. Toussaint, J.L. Richardson: Phys. Rev. B 32, 1720 (1985); cf. C. Kalle, S. Wansleben: J. Stat. Phys., in print
- A1.3 R.B. Pearson, J.L. Richardson, D. Toussaint: J. Comput. Phys. 51, 241 (1983)
R.B. Pearson: J. Comput. Phys. 49, 478 (1983)
A. Hoogland, J. Spaa, B. Selman, A. Compagner: J. Comput. Phys. 51, 250 (1983)
- A1.4 A. Hoogland: Unpublished
- A1.5 A. Margolina: Unpublished
- A1.6 J.H. Ahrens, U. Dieter: *Pseudo Random Numbers* (Wiley, New York 1979); Math. Comput. 27, 927 (1973)
- A1.7 D. Knuth: *The Art of Computer Programming*, Vol.2 (Addison-Wesley, Reading, MA 1969)
- A1.8 D.H. Lehmer: Proc. 2nd Symposium on Large-Scale Digital Computing Machinery (Harvard University, Cambridge 1951) p.142
- A1.9 M.D. MacLaren, G. Marsaglia: J. ACM 12, 83 (1965)

- A1.10 R.D. Carmichael: Bull. Am. Math. Soc. 16, 232 (1910)
- A1.11 D. Stauffer: J. Appl. Phys. 53, 7980 (1982)
- A1.12 G.O. Williams, M.H. Kalos: J. Stat. Phys. 17, 534 (1984)
- A1.13 R.R. Coveyou: J. ACM 7, 72 (1960)
- A1.14 M. Greenberger: Math. Comput. 15, 383 (1961); *ibid.* 16, 126 (1962)
- A1.15 I. Borosh, H. Niederreiter: BIT 23, 65 (1983)
- A1.16 R.R. Coveyou, R.D. MacPherson: J. ACM 14, 100 (1967)
- A1.17 G. Marsaglia: Proc. Natl. Acad. Sci. USA 61, 25 (1968)
- A1.18 R.C. Tausworth: Math. Comput. 19, 201 (1965)
- A1.19 N. Zierler: Inform. Control 15, 67 (1969)
 - N. Zierler, J. Brillhart: Inform. Control 13, 541 (1968); *ibid.* 14, 566 (1969)
 - H. Niederreiter: In *Probability and Statistical Interference*, ed. by W. Grossman, G. Pflug (North-Holland, Amsterdam 1982)
 - M. Fushimi, S. Tezuka: Commun. ACM 26, 516 (1983)
- A1.20 S. Kirkpatrick, E.P. Stoll: J. Comput. Phys. 40, 517 (1981)
 - J.A. Greenwood: Unpublished (1981)
- A1.21 H. Groote: Private communication
- A1.22 T.G. Lewis, W.H. Payne: J. ACM 20, 456 (1973)
- A1.23 J.P.R. Tootill, W.D. Robinson, A.G. Adams: J. ACM 28, 381 (1971)
- A1.24 R.W. Hamming: *Numerical Methods for Scientists and Engineers* (McGraw-Hill, New York 1962)
- A1.25 Collected algorithms of the ACM
- A1.26 J. von Neumann: *Various Techniques Used in Connection with Random Digits*, Collected Works, Vol.5 (Pergamon, New York 1963)
- A1.27 G.E. Forsythe: Math. Comput. 26, 817 (1972)
- A1.28 R.P. Brent: Algorithm 488, in *Collected Algorithms from CACM*
- A1.29 G.E.P. Box, M.E. Muller, G. Marsaglia: Ann. Math. Stat. 28, 610 (1958)

- A2.1 G.S. Grest, B. Dünweg, K. Kremer: Vectorized link cell Fortran code for molecular dynamics simulations for a large number of particles. Preprint (1989)
- A2.2 R. Friedberg, J.E. Cameron: J. Chem. Phys. 52, 6049 (1970)
- A2.3 L. Jacobs, C. Rebbi: J. Comput. Phys. 17, 10 (1975)
- A2.4 C. Kalle, V. Winkelmann: J. Stat. Phys. 28, 639 (1982)
- A2.5 G.O. Williams, M.H. Kalos: J. Stat. Phys. (1984)
- A2.6 A.B. Bortz, M.H. Kalos, J.L. Lebowitz: J. Comput. Phys. 17, 10 (1975)
- A2.7 M.H. Kalos: In Proc. Brookhaven Conf. on Monte Carlo Methods and Future Computer Architecture (1983) (unpublished)
- A2.8 K.E. Schmidt: Phys. Rev. Lett. 51, 2175 (1983)

Subject Index

- Absolute activity 96
- Ad hoc scaling 30, 31, 36
- Antiferromagnetism 75
- Aperiodicity 54
- Argon 30, 60, 113, 121
- Average 9
 - ensemble 9, 22, 62, 70
 - time 9
 - trajectory 22, 36, 62, 70
- Basic time step 26
- Big-oh 17
- Block distribution function 102
- Bond 91
- Boundary conditions
 - free 77
 - helical 102
 - non-standard 77
 - periodic 15, 30, 77
 - self consistent field 77
- Brownian Dynamics 55, 121
- Cellular automata 101
- Central limit theorem 64
- Chemical potential 96
- Cluster 1, 91
- Coarse graining 84
- Compressibility 102
- Computational complexity 35
- Conditional probability 52
- Corrector 19
- Correlation
 - function 57
 - length 82, 85
 - time 10, 58
- Critical point 75
- Critical slowing down 90
- Cycle 105
- Cumulative distribution 87
- Damped force method 36
- Demon 74
- Detailed balance 69, 79
- Deterministic approach 6, 9
- Difference quotient
 - forward 18
 - backward 18
- Diffusion limited aggregation 7
- Discretization 5, 17
- Discretization error 18
 - local 18
 - global 18
- d-space non-uniformity 107
- Eden cluster 7
- Energy
 - drift 20, 23
 - fluctuations 32, 36, 38, 40
- Ensemble
 - canonical 35, 78
 - grand 96
 - isothermal-isobaric 42, 94
 - microcanonical 27, 73
- Entropy 84
- Equations of motion 14, 17, 59
 - Hamilton 59
 - Lagrangian 38
 - Newtonian 27
- Equilibration 26
 - phase 30
- Equipartition theorem 23
- Ergodicity 53, 68, 71, 101
- Euler algorithm 18
- Ewald sum 16
- Exchange coupling 75
- Exclusive-or 108
- Explicit central difference 28
- Feed-back shift register 107
- Ferromagnetism 75

- Finite difference scheme 17
- Finite size effect 10
- Fluctuations 11
- Fourier acceleration 94
- Free energy 70, 83
 - Ginzburg-Landau-Wilson 85
- Gaussian principle of least constraint 40
- Gaussian process 58
- Generalized force 38
- Ghost particles 98
- Glauber function 81, 100
- Heat bath 35, 40, 59, 78, 103
- Image particle 16
- Initial value problem 17
- Intrinsic dynamics 9
- Irreducibility 53
- Ising model 75, 85, 90, 100, 128
- Kauffman model 102
- Kawasaki dynamics 80, 100
- Lagrangian 38
- Langevin equation 14, 56
- Law of large numbers 63
 - strong 64
- Leapfrog algorithm 37
- Lennard-Jones potential 30, 40, 70, 83, 113, 121
- Magnetization 75
 - finite size dependence 82
 - spontaneous 75, 81
- Markov chain 51
- Markov process 10, 51
- Master equation 68
- Mean field 85
- Mean recurrence time 54
- Mersenne number 106
- Metropolis 68
 - function 81, 100
- Microscopic reversibility 69
- Minimum image convention 16, 30
- Modulo generator 105
- Molecular dynamics 6, 13
 - canonical 35
 - cell 15, 22, 30
 - iso-kinetic 36, 41
 - isothermo-isobaric 42
 - Metropolis 81
 - microcanonical 27, 113
 - NVT 38
 - step 20
- Monte-Carlo
 - canonical 78
 - Creutz method 73, 101, 128
 - dynamic interpretation 71
 - grand ensemble 97, 99
 - hybrid 94
 - method 3, 62
 - NPT 95
 - step (MCS) 76
- Multistep method 19
- Non-holonomic constraint 36, 38
- Observation time 9
- One-step method 18
- Order parameter 75, 85
 - finite size dependence 88
- Pair correlation function 24
- Particles
 - creation 96
 - destruction 96
- Partition function 66, 83, 91, 94, 96
- Percolation 1, 91
 - bond 6
- Period 54
- Persistent 54
- Phase space 8
- Phase transition 75, 89
- Polar method 111
- Potential cut-off 22
- Potts model 90
- Predictor 19
- Probability distribution
 - invariant 53
 - stationary 53
- Pseudorandom numbers 105
- Q2R model 101
- R250 108
- Random force 56

Random number generator 104
 Random numbers 104
 — correlations 107
 Random walk 6, 51, 74
 Realization 2
 Relaxation times 26
 Round-off error 21

Sampling
 — importance 65
 — straightforward 63
 Seed 105
 Self-averaging
 — strong 11
 — weak 11
 Self-avoiding walk 51, 74, 101
 Single-site probability 86
 Spin 75
 Spin flip 75, 80
 Spectral density 56
 Spring 4
 Statistical error 10
 Steepest descent 66
 Stochastic approach 1, 9

Stochastic differential equation 15
 Stochastic dynamics 55
 Stochastic integral 56
 Stochastic matrix 53
 Stochastic supplements 36
 Summed form 29
 Susceptibility 85

Tail correction 24
 Taylor expansion 17
 Time integration 17
 Time reversibility 29
 Two-body central force 30
 Two-step method 28
 Trajectory 5, 9, 13, 55
 Transition probability 52, 68, 79, 83,
 97, 100
 — Monte-Carlo 52

Velocity form 29
 Verlet algorithm 28
 Verlet table 35, 50
 Virtual particles 98